

企业级智能客服

1. 企业级智能客服版本1

1.1 al_chat_0.1 项目部署

1.1.1 项目文件和环境说明

开发环境

Pycharm

项目部署环境

<https://www.autodl.com/>

项目功能演示

登录，注册
基于对话模型对话
基于私有模型对话
基于语义相似度缓存功能
多用户功能
联网搜索

1.1.1 项目部署之Ollama部署和私有化模型部署

部署项目	操作
安装Ollama	1. 安装ollama curl -fsSL https://ollama.com/install.sh sh 2. 启动ollama ollama serve

安装思考模型	ollama run deepseek-r1:1.5b
安装Embedding模型	ollama pull bge-m3

1.1.2 项目部署之MySQL数据库安装

安装Mysql	<pre>sudo apt-get update sudo apt-get install mysql-server sudo service mysql start 首次 mysql -u root -p (下次登录输入mysql -u gpt -p)</pre>
创建账户	<pre>CREATE USER 'gpt'@'localhost' IDENTIFIED BY 'gpt';</pre>
给账户赋权限	<pre>GRANT ALL PRIVILEGES ON *.* TO 'gpt'@'localhost'; FLUSH PRIVILEGES;</pre>
创建数据库	<pre>create database al_chat;</pre>

1.1.3 项目部署之Redis数据库安装

下载Redis源码	<pre>apt-get install -y gcc make tcl wget wget http://download.redis.io/releases/redis-7.0.4.tar.gz tar -zxvf redis-7.0.4.tar.gz</pre>
编译源码	<pre>cd redis-7.0.4 cd deps; make -j4 hiredis jemalloc linenoise lua cd .. #回到redis源码目录 make install #安装redis make clean #清理编译文件</pre>
修改配置文件	<pre>cd src vi ../redis.conf #修改redis数据密码为gpt</pre> 1. bind：指定Redis的监听地址，默认是`127.0.0.1`，如果需要监听所有地址，则可以设置为`0.0.0.0`。比如大家使用云服务器部署Redis，想要在自己的`IP`地址上进行访问，则可以设置为`0.0.0.0`，否则会无法访问到`Redis`服务。

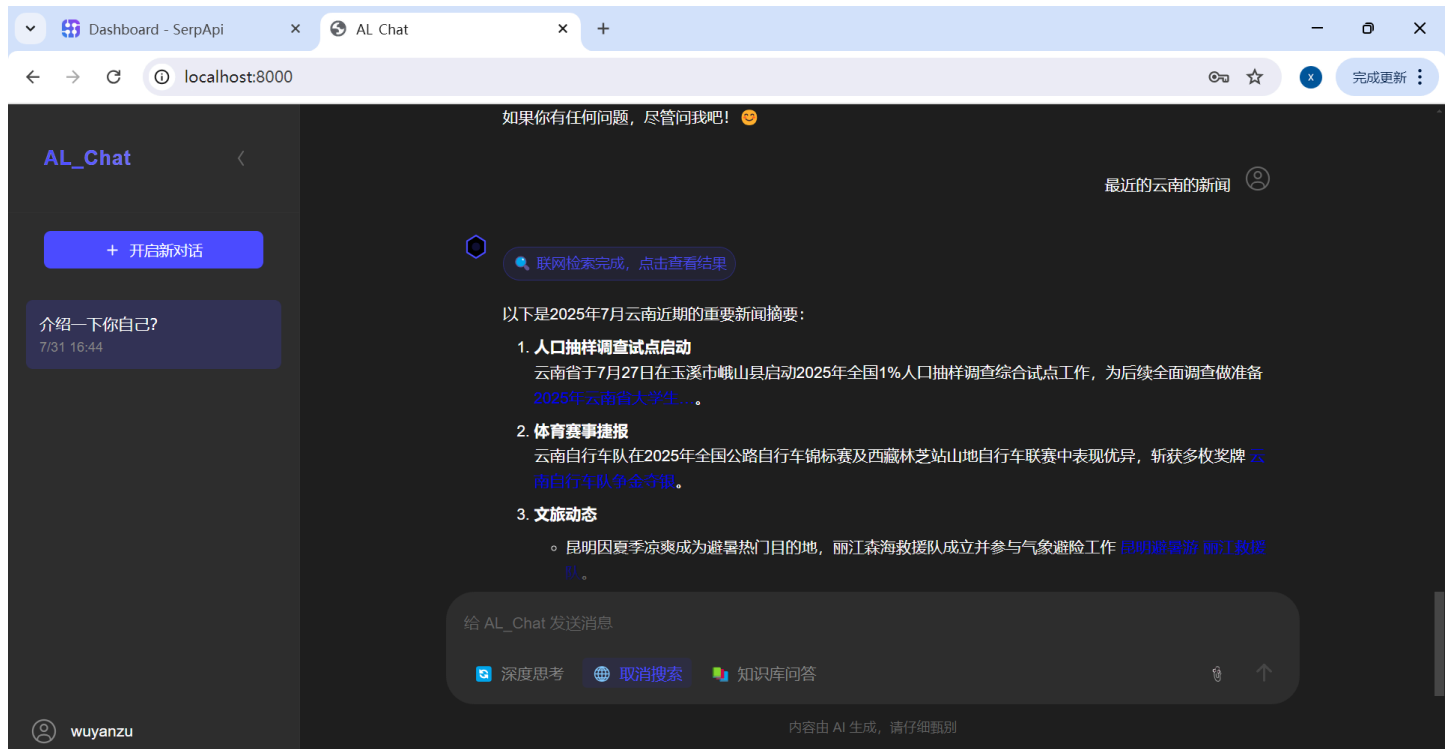
	2. port: 指定Redis的监听端口号, 默认是`6379`。如果大家想要修改端口号, 则可以设置为其他端口号。 3. requirepass: 指定Redis的密码, 默认是`""`, 如果大家想要设置密码, 则可以设置为其他密码。
启动redis	redis-server ../redis.conf
测试redis	代码块 <pre>1 ! pip install redis 2 3 import redis 4 5 r = redis.Redis(host='localhost', 6 password='gpt',port=6379, db=0) 7 8 # 增: 添加数据 9 def add_data(key, value): 10 r.set(key, value) # set: 设置键值对 11 12 print(f"添加数据: {key} -> {value}") 13 14 # 查: 获取数据 15 def get_data(key): 16 value = r.get(key) # get: 获取键值对 17 18 if value is not None: 19 print(f"获取数据: {key} -> {value.decode('utf-8')}") 20 else: 21 print(f"键 {key} 不存在") 22 23 # 添加数据 24 add_data("name", "wuyanzu") 25 add_data("age", "30")</pre>

1.1.4 项目部署之项目启动与功能测试

代码块

```
1  步骤1: 创建项目虚拟环境
2  conda create --name al_chat_0.1 python==3.11
3  conda activate al_chat_0.1
4  步骤2: 安装依赖
5  pip install -r requirements.txt
```

```
6  步骤3：配置 .env文件
7      配置在线模型信息
8      配置离线模型信息
9      配置embedding模型信息
10     配置数据库信息
11     配置redis数据库信息
12  步骤4：初始化数据库表
13  python ./scripts/init_db.py
14  步骤5：启动项目
15  python run.py
```



1.2 al_chat_0.1 技术方案

1.2.1 FastAPI前后端分析

FastAPI 是一个开源项目，其作用是基于标准 Python 类型提示使用 Python 构建 REST API，使用 ASGI 的标准来构建 Python Web 框架和服务端。所有简单理解：FastAPI 是一个 ASGI Web 框架。官方Github: <https://github.com/fastapi/fastapi>

对 FastAPI 的两个概念理解：

1. 它可以把一个 Python 函数，封装成一个 REST API 接口，并提供给前端调用。类似于 /api/generate 接口。
2. 它是一个 ASGI Web 框架，可以构建 Python Web 框架和服务端，解决前端和后端之间的通信问题。也就是可以让前端通过 URL 地址来访问后端服务，并根据请求的参数来执行对应的后端服务，并且前端还可以获取到后端服务的返回结果。

几个流行的基于ASGI的Web框架包括：

1. Starlette：一个轻量级的ASGI框架，它旨在成为构建高性能异步服务的基础框架。Starlette 可以独立使用，也可以作为其他框架的基础组件。
2. FastAPI：建立在Starlette之上的一个现代、快速（高性能）的Web框架，用于构建API。FastAPI 强调快速开发、类型安全和自动API文档生成。
3. Uvicorn：一个轻量级、超快的ASGI服务器，常用于运行ASGI应用。虽然Uvicorn自身不是一个Web框架，但它是运行基于ASGI的框架（如Starlette和FastAPI）的推荐服务器。

简单理解：ASGI 框架（FastAPI 和 Starlette）负责定义应用程序的逻辑和结构，服务器（Uvicorn）负责处理网络请求和响应。其中，Uvicorn 是 FastAPI 的默认服务器。

接下来，我们通过一个示例，来介绍如何使用 FastAPI 来构建一个 REST API 接口。

- 下载 fastapi 依赖包

使用 `pip install fastapi` 命令下载 fastapi 依赖包。

- 安装 uvicorn 依赖包

接下来，我们需要使用一个 ASGI 服务器来运行 FastAPI 应用程序。这里我们使用 Uvicorn 作为 ASGI 服务器。需要使用 `pip install uvicorn` 命令下载 uvicorn 依赖包。

- 创建一个 FastAPI 应用程序

创建一个 FastAPI 应用程序，并使用 Uvicorn 作为 ASGI 服务器来运行。这里需要说明的是，jupyter notebook 中是无法直接运行 FastAPI 应用程序的，需要使用 Uvicorn 作为 ASGI 服务器来运行。因此，我们在 test_fastapi.py 文件中，来创建一个 FastAPI 应用程序，并使用 Uvicorn 作为 ASGI 服务器来运行。代码如下：

代码块

```
1  from fastapi import FastAPI
2  from pydantic import BaseModel
3
4  app = FastAPI() # 创建 FastAPI 应用实例
5  class Item(BaseModel):
6      name: str # 项目的名称
7      description: str = None # 项目的描述，默认为 None
8  @app.get("/") # 定义 GET 请求的根路径
9  async def read_root():
10     """返回欢迎消息"""
11     return {"message": "Hello, World"}
12  @app.post("/items/") # 定义 POST 请求的 /items 路径
13  async def create_item(item: Item):
```

```
14         """创建一个新项目并返回其数据"""
15         return item # 返回创建的项目数据
```

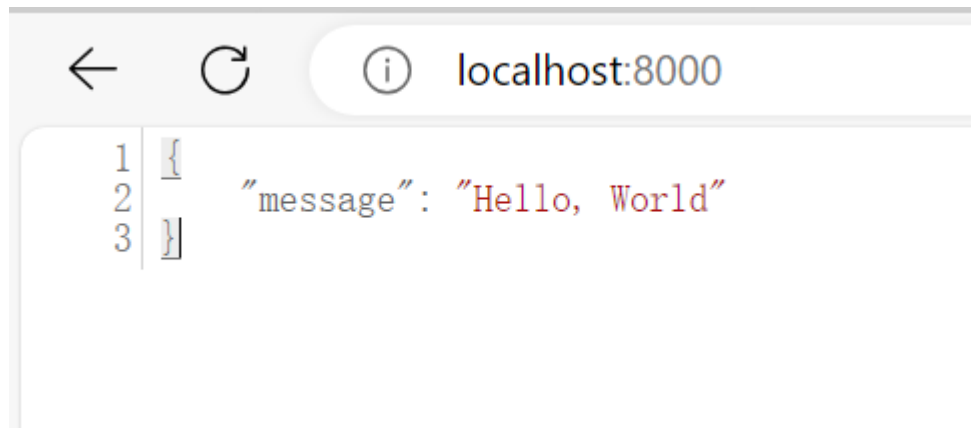
- 启动项目

在 `test_fastapi.py` 文件中，我们创建了一个 `FastAPI` 应用程序，并使用 `Uvicorn` 作为 `ASGI` 服务器来运行。接下来，我们启动项目，并访问 `http://127.0.0.1:8000/items/` 地址，来访问 `FastAPI` 应用程序。启动命令如下：

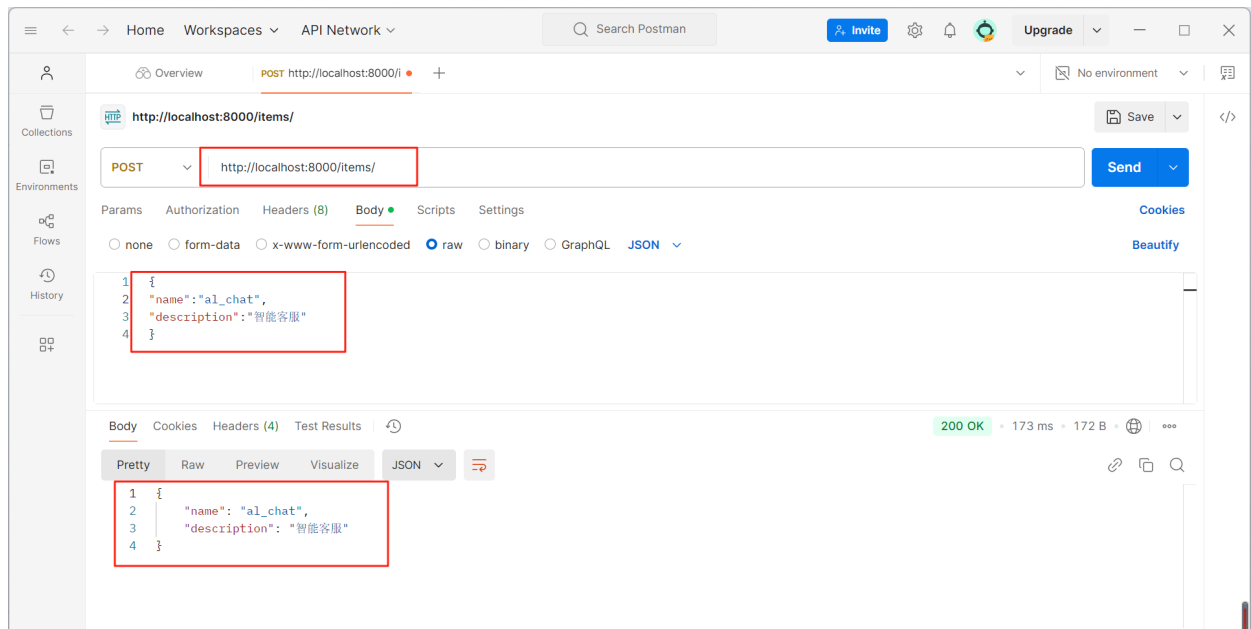
代码块

```
1 uvicorn test_fastapi:app --reload
```

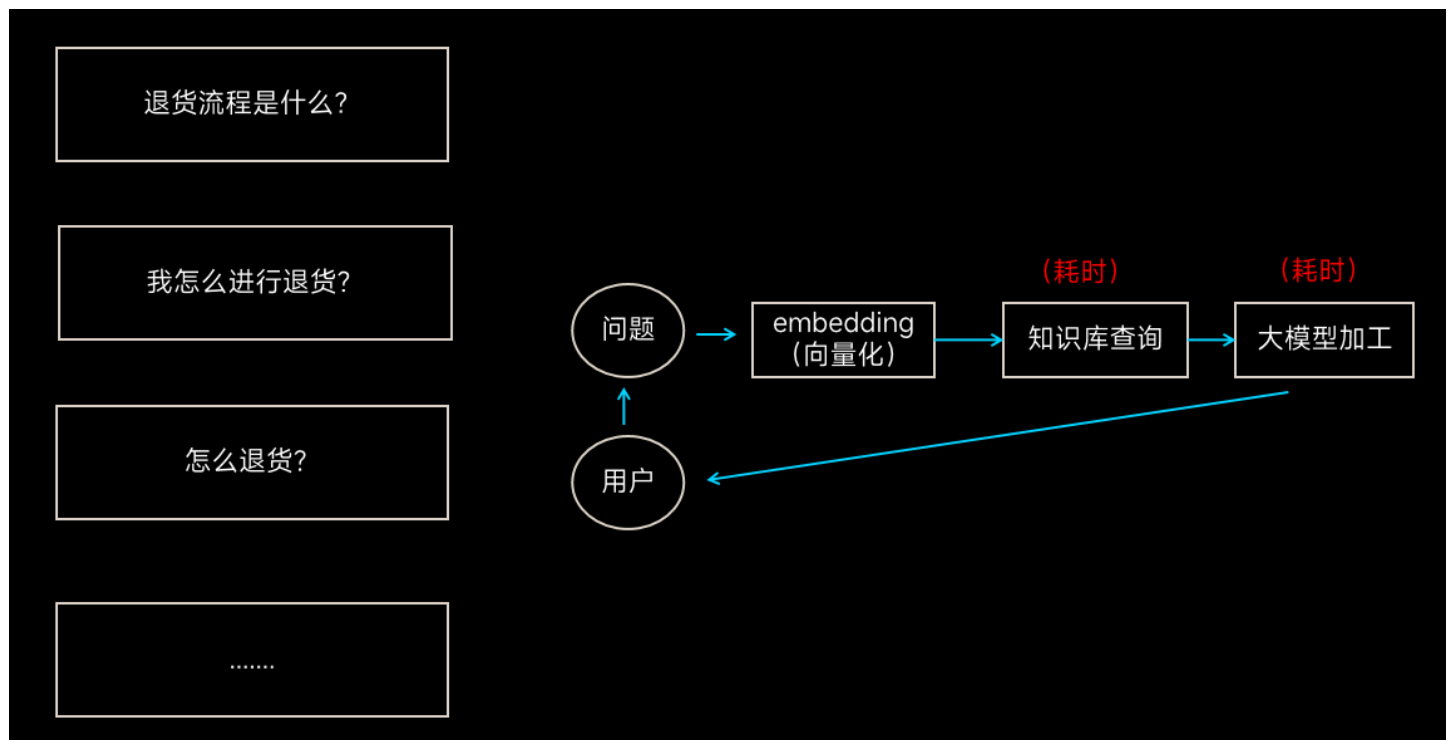
启动项目后，根据代码中定义的请求，当通过 `localhost:8000/` 地址访问时，会调用 `read_root` 函数，返回 `Hello, World` 消息。

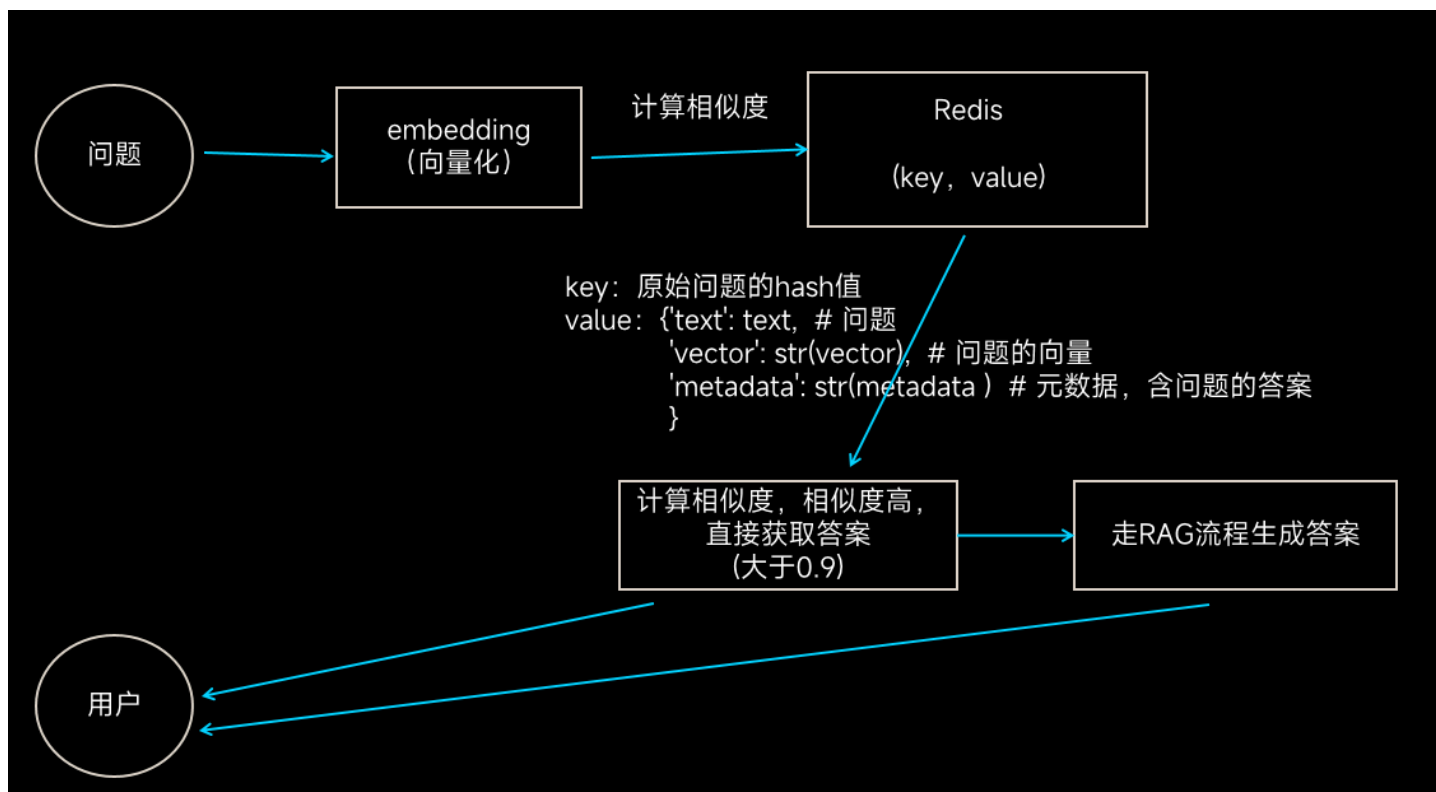


而当通过 `localhost:8000/items/` 地址访问时，会调用 `create_item` 函数，返回创建的项目数据。但是这里需要注意到是：`POST` 请求是无法在浏览器的地址栏中直接访问的，因为 `POST` 请求需要发送请求体，而浏览器地址栏无法发送请求体。所有我们需要使用 `curl` 命令或者 `Postman` 工具来访问。

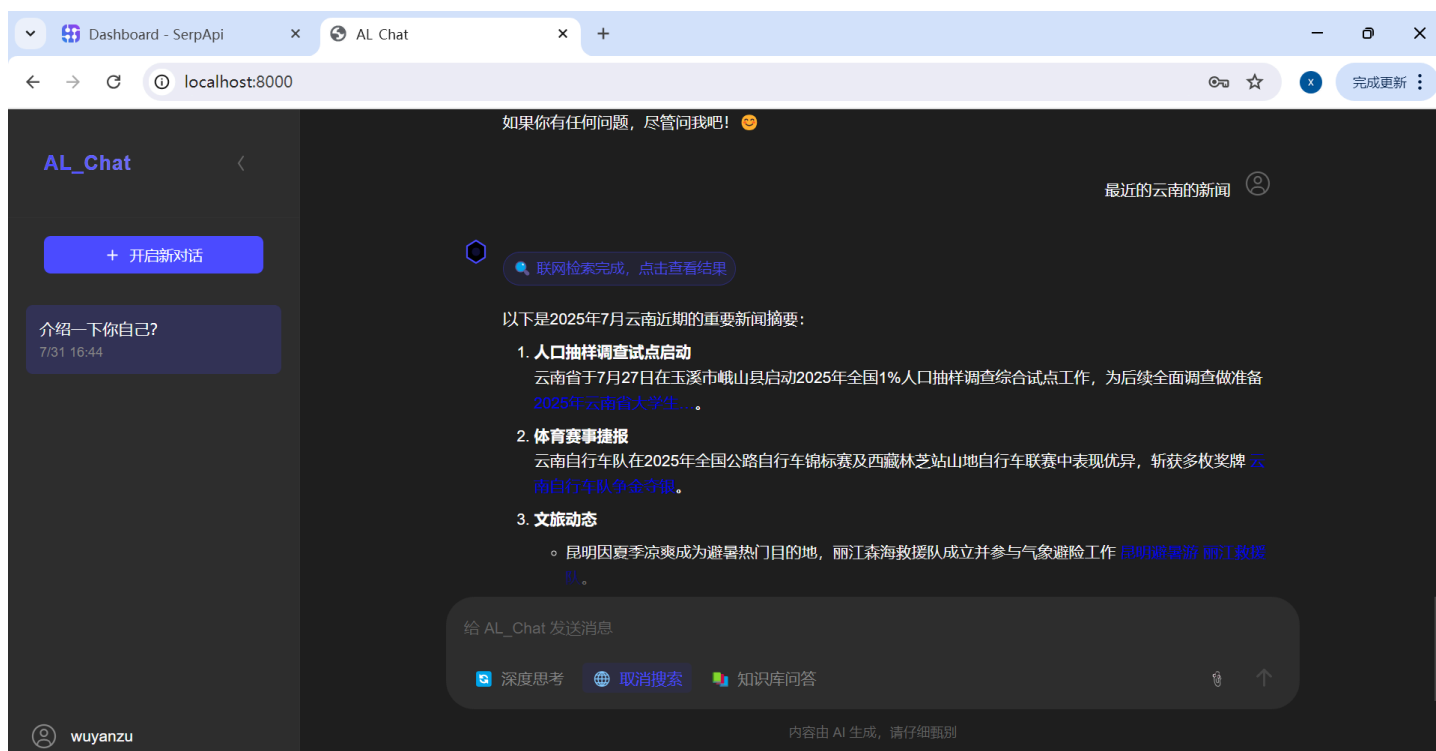


1.2.2 语义相似度缓存方案





1.2.3 Function Call外部工具



1.3 al_chat_0.1 代码实现

1.3.1 代码实现之数据库表设计

SQLAlchemy官网: <https://www.sqlalchemy.org/>

用户表	字段名	类型	约束	说明
	id	Integer	PRIMARY KEY	用户唯一标识
	username	String(50)	UNIQUE, NOT NULL	用户名
	email	String(100)	UNIQUE, NOT NULL	电子邮件
	password_hash	String(255)	NOT NULL	密码哈希值
	created_at	DateTime	DEFAULT NOW()	创建时间
	updated_at	DateTime	DEFAULT NOW()	更新时间
	last_login	DateTime	NULL	最后登录时间
	status	String(20)	DEFAULT 'active'	用户状态

对话表	字段名	类型	约束	说明
	id	Integer	PRIMARY KEY	对话唯一标识
	user_id	Integer	FOREIGN KEY	关联用户ID
	created_at	DateTime	DEFAULT NOW()	创建时间
	updated_at	DateTime	DEFAULT NOW()	更新时间
	status	String(20)	DEFAULT 'ongoing'	对话状态
	dialogue_type	Enum	NOT NULL	对话类型

消息表	字段名	类型	约束	说明
	id	Integer	PRIMARY KEY	消息唯一标识
	conversation_id	Integer	FOREIGN KEY	关联对话ID
	sender	String(50)	NOT NULL	发送者
	content	Text	NOT NULL	消息内容
	created_at	DateTime	DEFAULT NOW()	创建时间
	message_type	String(20)	DEFAULT 'text'	消息类型

1.3.2 代码实现之登录模块设计

代码块

```
1  ## 用户注册逻辑
2  @router.post("/register", response_model=UserResponse)
3  # Depends(get_db) 表示 db 参数是一个依赖项，它会调用 get_db 函数来获取一个数据库会话 (AsyncSession) 。
4  # FastAPI 会在处理请求时自动执行 get_db 函数，并将返回的结果传递给 register 函数的 db 参数。
5  async def register(user_data: UserCreate, db: AsyncSession = Depends(get_db)):
6      try:
7          user_service = UserService(db)
8          user = await user_service.create_user(user_data)
9          return user
10     except ValueError as e:
11         raise HTTPException(status_code=400, detail=str(e))
12  ## 用户登录逻辑
13  @router.post("/token", response_model=Token)
14  async def login(user_data: UserLogin, db: AsyncSession = Depends(get_db)):
15      user_service = UserService(db)
16      user = await user_service.authenticate_user(user_data.email,
17      user_data.password)
18      if not user:
19          raise HTTPException(
20              status_code=401,
21              detail="Incorrect email or password",
22              headers={"WWW-Authenticate": "Bearer"},
23          )
24      access_token_expires =
25      timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
26      access_token = create_access_token(
27          data={"sub": user.email}, expires_delta=access_token_expires
28      )
29      return {"access_token": access_token, "token_type": "bearer"}
30  ## 登录以后，显示用户信息
31  @router.get("/users/me", response_model=UserResponse)
32  async def get_current_user_info(current_user: User =
33      Depends(get_current_user)):
34      """获取当前登录用户的信息"""
35      return current_user
```

1.3.3 代码实现之对话模块设计



1.3.4 代码实现之联网模块设计

提示词模板设计

代码块

```
1  """搜索服务的提示词管理"""
2  from datetime import datetime
3
4  # 基础系统提示
5  SEARCH_SYSTEM_PROMPT = """你是一个智能助手, 可以通过调用外部的工具获取实时信息。
6  你可以使用的工具及使用方法如下:
7
8  {tools_description}
9
10  当你遇到以下情况时, 请调用联网检索工具:
11  1. 问题涉及实时数据 (如天气、新闻、股票)。
12  2. 问题涉及最新事件或动态信息 (如 '今天发生了什么? ')。
13  3. 问题涉及特定领域的外部知识 (如法律、医疗、科技)。
14  4. 问题中包含 '最新'、'当前'、'今天' 等时间敏感关键词。
15  其他情况下, 请直接回答用户的问题。"""
16
17  # 搜索结果总结提示
18  SEARCH_SUMMARY_PROMPT = """# 以下内容是基于用户发送的消息的搜索结果:
19
20  {context}
21
22  每个搜索结果包含标题、链接和内容。在引用来源时, 请使用以下固定格式:
23  - 引用格式: [标题] (链接)
24  - 示例: 根据最新研究显示, 人工智能正在快速发展 [2024年AI发展报告] (http://example.com)
25  - 引用必须放在相关内容的末尾, 不要集中在最后
```

26 - 如果一段内容来自多个来源，请在末尾列出所有来源，用空格分隔，如：[标题1] (链接1) [标题2]
(链接2)

27

28 在回答时，请注意以下几点：

29

30 今天是{cur_date}。

31 1. 并非搜索结果的所有内容都与用户的问题密切相关，你需要结合问题，对搜索结果进行甄别、筛
选。

32 2. 对于列举类的问题，尽量将答案控制在10个要点以内，并告诉用户可以查看搜索来源、获得完整信
息。

33 3. 对于创作类的问题，请务必在每个段落末尾使用规定格式引用来源。

34 4. 如果回答很长，请尽量结构化、分段落总结。如果需要分点作答，尽量控制在5个点以内。

35 5. 对于客观类的问答，如果答案非常简短，可以适当补充相关信息。

36 6. 你需要根据用户要求和回答内容选择合适、美观的回答格式，确保可读性强。

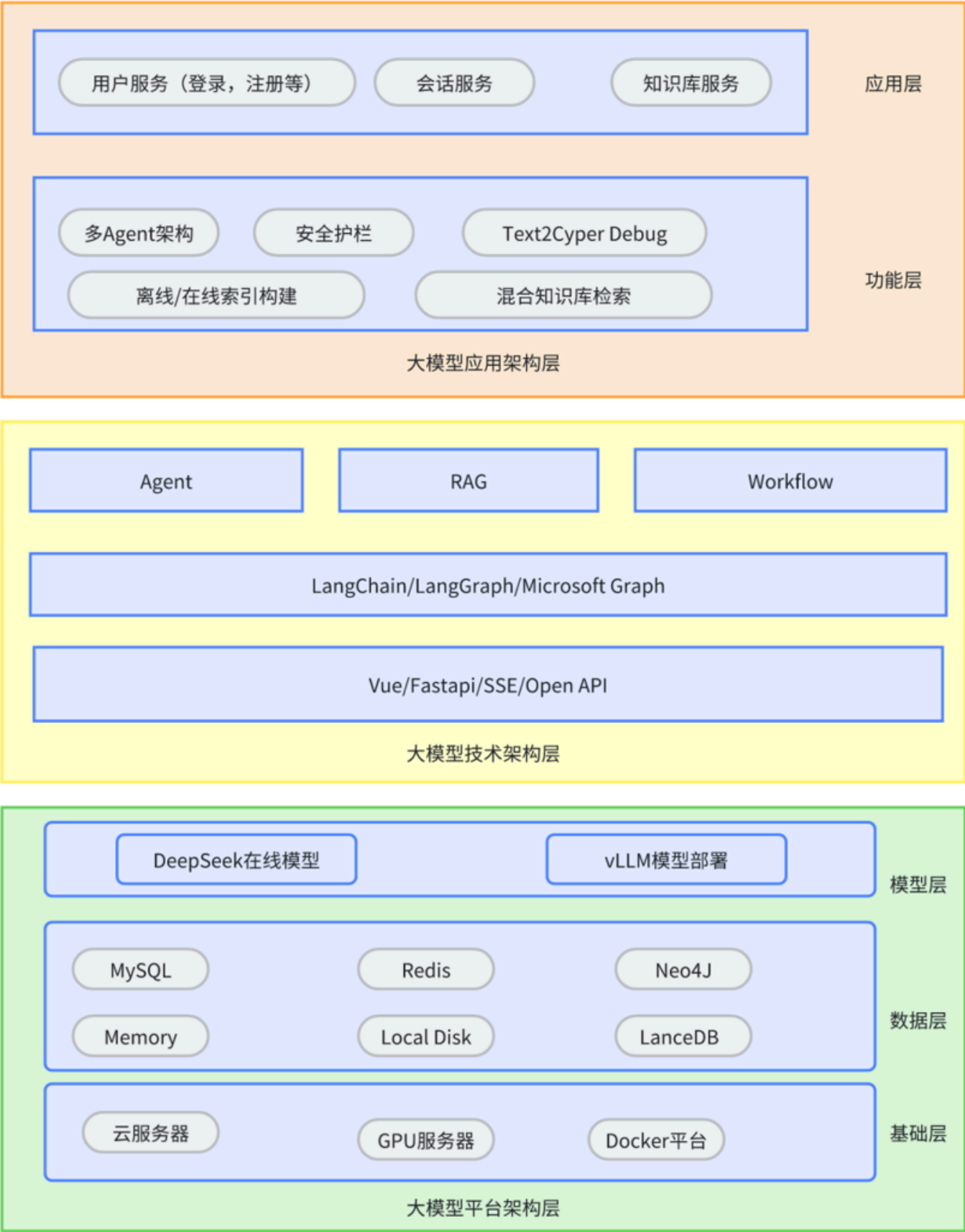
37 7. 你的回答应该综合多个相关网页来回答，不要重复引用同一个来源。

38

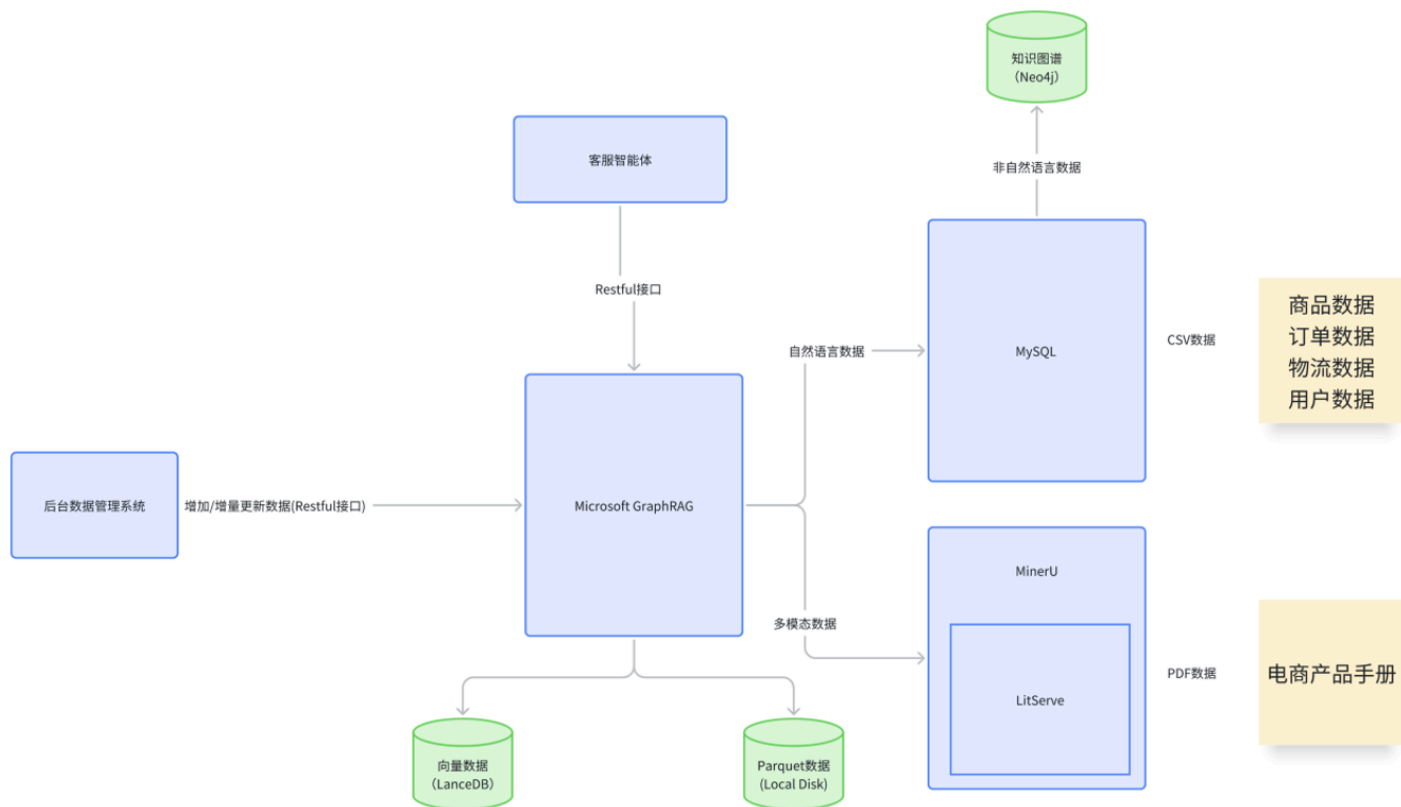
39 用户问题：{query}""

2. 企业级智能客服版本2

2.1 企业智能客服整体架构设计



2.2 企业智能客服数据架构方案



2.3 企业智能客服CSV数据方案设计与落地

2.3.1 电商客服数据准备

智能客服场景使用的数据往往是企业内部非常私密的数据，是不太可能在公网上获取到相关的隐私数据的。因此，我们接下来给大家选择的数据集是来自 Kaggle 上的一个公开数据集 -

Northwind Traders。其地址为：<https://www.kaggle.com/datasets/jeetahirwar/northwind-traders>

该数据集包含了一个小型商业公司的各种业务数据，如产品、订单、客户、员工等信息。它是一个关系型数据库的经典示例，包含了多个相互关联的表格。以下是 Northwind Traders 数据库中通常包含的一些主要表格及其字段说明：

Customers (客户表)	<table><tr><th>字段名称</th><th>数据类型</th><th>描述</th></tr><tr><td>CustomerID</td><td>nchar(5)</td><td>客户ID (主键)</td></tr><tr><td>CompanyName</td><td>nvarchar(40)</td><td>公司名称</td></tr><tr><td>ContactName</td><td>nvarchar(30)</td><td>联系人姓名</td></tr><tr><td>ContactTitle</td><td>nvarchar(30)</td><td>联系人职务</td></tr><tr><td>Address</td><td>nvarchar(60)</td><td>地址</td></tr><tr><td>City</td><td>nvarchar(15)</td><td>城市</td></tr></table>	字段名称	数据类型	描述	CustomerID	nchar(5)	客户ID (主键)	CompanyName	nvarchar(40)	公司名称	ContactName	nvarchar(30)	联系人姓名	ContactTitle	nvarchar(30)	联系人职务	Address	nvarchar(60)	地址	City	nvarchar(15)	城市
字段名称	数据类型	描述																				
CustomerID	nchar(5)	客户ID (主键)																				
CompanyName	nvarchar(40)	公司名称																				
ContactName	nvarchar(30)	联系人姓名																				
ContactTitle	nvarchar(30)	联系人职务																				
Address	nvarchar(60)	地址																				
City	nvarchar(15)	城市																				
Employees (员工表)	<table><tr><th>字段名称</th><th>数据类型</th><th>描述</th></tr><tr><td>EmployeeID</td><td>int</td><td>员工ID (主键)</td></tr><tr><td>employeeName</td><td>nvarchar(20)</td><td>员工名字</td></tr><tr><td>Title</td><td>nvarchar(30)</td><td>职位</td></tr><tr><td>City</td><td>nvarchar(15)</td><td>城市</td></tr><tr><td>Country</td><td>nvarchar(15)</td><td>国家</td></tr><tr><td>ReportsTo</td><td>int</td><td>上级ID (外键)</td></tr></table>	字段名称	数据类型	描述	EmployeeID	int	员工ID (主键)	employeeName	nvarchar(20)	员工名字	Title	nvarchar(30)	职位	City	nvarchar(15)	城市	Country	nvarchar(15)	国家	ReportsTo	int	上级ID (外键)
字段名称	数据类型	描述																				
EmployeeID	int	员工ID (主键)																				
employeeName	nvarchar(20)	员工名字																				
Title	nvarchar(30)	职位																				
City	nvarchar(15)	城市																				
Country	nvarchar(15)	国家																				
ReportsTo	int	上级ID (外键)																				
Products (产品表)	<table><tr><th>字段名称</th><th>数据类型</th><th>描述</th></tr><tr><td>ProductID</td><td>int</td><td>产品ID (主键)</td></tr><tr><td>ProductName</td><td>nvarchar(40)</td><td>产品名称</td></tr><tr><td>CategoryID</td><td>int</td><td>类别ID (外键)</td></tr><tr><td>QuantityPerUnit</td><td>nvarchar(20)</td><td>单位数量</td></tr><tr><td>UnitPrice</td><td>money</td><td>单价</td></tr><tr><td>Discontinued</td><td>bit</td><td>是否停产</td></tr></table>	字段名称	数据类型	描述	ProductID	int	产品ID (主键)	ProductName	nvarchar(40)	产品名称	CategoryID	int	类别ID (外键)	QuantityPerUnit	nvarchar(20)	单位数量	UnitPrice	money	单价	Discontinued	bit	是否停产
字段名称	数据类型	描述																				
ProductID	int	产品ID (主键)																				
ProductName	nvarchar(40)	产品名称																				
CategoryID	int	类别ID (外键)																				
QuantityPerUnit	nvarchar(20)	单位数量																				
UnitPrice	money	单价																				
Discontinued	bit	是否停产																				
Categories (分类表)																						

	<table><tr><th>字段名称</th><th>数据类型</th><th>描述</th></tr><tr><td>CategoryID</td><td>int</td><td>类别ID (主键)</td></tr><tr><td>CategoryName</td><td>nvarchar(15)</td><td>类别名称</td></tr><tr><td>Description</td><td>ntext</td><td>类别描述</td></tr></table>	字段名称	数据类型	描述	CategoryID	int	类别ID (主键)	CategoryName	nvarchar(15)	类别名称	Description	ntext	类别描述															
字段名称	数据类型	描述																										
CategoryID	int	类别ID (主键)																										
CategoryName	nvarchar(15)	类别名称																										
Description	ntext	类别描述																										
Shippers (货运公司表)	<table><tr><th>字段名称</th><th>数据类型</th><th>描述</th></tr><tr><td>ShipperID</td><td>int</td><td>货运公司ID (主键)</td></tr><tr><td>CompanyName</td><td>nvarchar(40)</td><td>公司名称</td></tr></table>	字段名称	数据类型	描述	ShipperID	int	货运公司ID (主键)	CompanyName	nvarchar(40)	公司名称																		
字段名称	数据类型	描述																										
ShipperID	int	货运公司ID (主键)																										
CompanyName	nvarchar(40)	公司名称																										
Orders (订单表)	<table><tr><th>字段名称</th><th>数据类型</th><th>描述</th></tr><tr><td>OrderID</td><td>int</td><td>订单ID (主键)</td></tr><tr><td>CustomerID</td><td>nchar(5)</td><td>客户ID (外键)</td></tr><tr><td>EmployeeID</td><td>int</td><td>员工ID (外键)</td></tr><tr><td>OrderDate</td><td>datetime</td><td>订单日期</td></tr><tr><td>RequiredDate</td><td>datetime</td><td>要求交货日期</td></tr><tr><td>ShippedDate</td><td>datetime</td><td>实际发货日期</td></tr><tr><td>ShippedID</td><td>int</td><td>运输方式ID (外键)</td></tr><tr><td>Freight</td><td>money</td><td>运费</td></tr></table>	字段名称	数据类型	描述	OrderID	int	订单ID (主键)	CustomerID	nchar(5)	客户ID (外键)	EmployeeID	int	员工ID (外键)	OrderDate	datetime	订单日期	RequiredDate	datetime	要求交货日期	ShippedDate	datetime	实际发货日期	ShippedID	int	运输方式ID (外键)	Freight	money	运费
字段名称	数据类型	描述																										
OrderID	int	订单ID (主键)																										
CustomerID	nchar(5)	客户ID (外键)																										
EmployeeID	int	员工ID (外键)																										
OrderDate	datetime	订单日期																										
RequiredDate	datetime	要求交货日期																										
ShippedDate	datetime	实际发货日期																										
ShippedID	int	运输方式ID (外键)																										
Freight	money	运费																										
Order Details (订单详情表)																												

字段名称	数据类型	描述
OrderID	int	订单ID (主键部分/外键)
ProductID	int	产品ID (主键部分/外键)
UnitPrice	money	单价
Quantity	smallint	数量
Discount	real	折扣

以上是 `Northwind Traders` 数据库中主要表格的关联关系，大家可以直接在其地址为：
<https://www.kaggle.com/datasets/jeetahirwar/northwind-traders> 下载到本地。不过，我们这里仿照该数据集，通过脚本生成了类似结构的符合电商领域客服场景的数据集，构建脚本存储位置在：
`create_sql_data.py`，大家可以直接运行该脚本，生成符合我们需求的数据集。

生成数据步骤：

1. 到MySQL数据库中提前创建好数据库：

create database graphrag
2. 下载数据集：

<https://www.kaggle.com/datasets/jeetahirwar/northwind-traders> 下载到本地
3. 执行脚本：

python `create_sql_data.py`

执行前需要安装如下依赖：

pip install pymysql pandas numpy faker sqlalchemy
4. 观察MySQL库中的数据表：

```
mysql> show tables;
+-----+
| Tables_in_graphrag |
+-----+
| Categories          |
| Customers           |
| Employees           |
| Order Details       |
| Orders              |
| Products            |
| Reviews             |
| Shippers            |
| Suppliers            |
+-----+
```

5. 观察csv文件

在 `graphrag\origin_data\exported_data` 目录下，还可以看到生成的一系列数据表的 `.csv` 文件

📁 / ... / origin_data / exported_data /

名称

- 📄 categories.csv
- 📄 customers.csv
- 📄 employees.csv
- 📄 merged_reviews.csv
- 📄 order_details.csv
- 📄 orders.csv
- 📄 processed_reviews.csv
- 📄 products.csv
- 📄 reviews.csv
- 📄 sales_report.csv
- 📄 shippers.csv
- 📄 suppliers.csv

这些就是我们接下来要操作的数据

2.3.2 数据处理方案分析

1. 对于非自然语言的结构化数据，直接定义明确的规范构建知识图谱，导入到如 `Neo4j` 中进行统一管理。比如以客服场景为例，能从结构化数据中梳理出来的明确节点和边如下所示：

代码块

1 - 节点类型

- 2 - Product - 产品节点
- 3 - Supplier - 供应商节点
- 4 - Customer - 客户节点
- 5 - Employee - 员工节点
- 6 - Shipper - 物流公司节点
- 7 - Order - 订单节点
- 8 - 边类型
- 9 - SUPPLIED_BY - 产品由供应商提供
- 10 - PLACED_BY - 订单由客户下单
- 11 - PROCESSED_BY - 订单由员工处理
- 12 - SHIPPED_VIA - 订单通过物流公司配送
- 13 - CONTAINS - 订单包含产品
- 14 - REPORTS_TO - 员工上下级关系

2. 对于自然语言的非结构化数据，根据场景的特定信息做文本聚合。比如如果想要提取一些商品的评论信息，可以按照商品的名称进行聚合，然后对聚合后的内容进行实体和关系的提取。比如：

ReviewID	ProductID	ProductName	CustomerID	CustomerName	Rating	Text	ReviewDate	CategoryName
22	5	LG 智能门铃	NI721	泰麒麟科技有限公司数字科技有限公司	4	智能门铃的设计很人性化，夜视效果好，值得推荐。	2025-03-16	智能门铃

聚合后的格式为：

代码块

```
1 "客户 {CustomerName} 对 {C
2 ategoryName} 产品 {ProductName} 的评价（评分：{Rating} 星）： \"{Text}\"（评价日期：{ReviewDate}）"
```

后者就是典型的 `Microsof GraphRAG` 比较适合的解析领域，而在前面的课程中，我们已经介绍过如何把 `Microsof GraphRAG` 生成的一系列 `parquet` 文件导入到 `Neo4j` 中，所以混合检索的策略也非常多，主要有如下几种：

1. 将直接构建的 `Neo4j` 知识图谱与 `Microsof GraphRAG` 构建出来的图谱进行合并，用统一的检索流程进行问答检索；

2. 分离直接可以构建的 Neo4j 知识图谱和 Microsoft GraphRAG 构建出来的图谱，用不同的检索流程进行问答检索；

- 直接可以构建的 Neo4j 知识图谱，使用 TEXT2CYPHER 的流程进行问答检索；
- Microsoft GraphRAG 构建出来的图谱，使用 Global Search 或者 Local Search 的流程进行问答检索；

实测效果比较好的是第二种策略，其一是可以按照业务场景分离数据库，各自建立对应的检索优化流程；其二在知识库的后期维护中，可以按照业务场景分离数据库，隔离数据。同时，借助 Agent 技术做好第一轮意图识别，根据用户需求选择对应的数据库进行针对性的检索，也可以有效降低检索的响应时长。

2.3.3 自然语言数据处理

2.3.3.1 CSV文件数据预处理

我们需要根据自己的业务需求，在偌大的业务数据中构建出较为完整的语义文本。以智能客服场景为例，我们可以抽象出如下几种业务需求通常是以自然语言存储的：

1. 客户评论与反馈数据：

代码块

```
1      review_id,product_id,customer_id,rating,review_text,date
2      101,P234,C567,4,"这款智能音箱音质非常好，但连接蓝牙设备时偶尔会断开。语音助手理解能力还需提高。",2023-05-12
```

其业务价值是从评论文本中提取产品特性、问题点和情感，构建产品-特性-问题的知识网络，帮助客服快速了解产品常见问题。

2. 客服聊天记录

代码块

```
1      conversation_id,timestamp,sender_type,message,intent,order_id
2      CS1001,2023-06-15 14:23:45,customer,"我上周五买的小米智能灯泡无法连接到我的华为手机，我已经按照说明书重置了三次",product_issue,ORD8976
```

其业务价值是提取产品问题、解决方案和兼容性信息，构建问题-故障-解决方案的知识图谱，训练客服系统自动回答类似问题。

3. 退货和投诉记录

代码块

```
1      case_id,order_id,product_id,return_reason,customer_description,resolution
```

2 RET345,ORD7891,P456,"产品损坏","收到的智能插座有明显的外壳裂痕,插入电源后指示灯闪烁红色,无法正常工作","退款并更换新品"

其业务价值是提取产品问题模式和解决方案， 构建产品-问题类型-解决方案关系。帮助客服系统自动处理退货和投诉。

原始数据：

ReviewID	ProductID	ProductName	CustomerID	CustomerName	Rating	Text	ReviewDate	CategoryName
10	5	索尼 智能体重秤 Basic	MH412	创联世纪传媒有限公司智能科技	4.9	这款智能体重秤质量很好,反应灵敏。总体来说很满意。	2024-06-12	智能体重秤

代码块

```
1  处理后的 text 字段中的内容：
2
3      客户ID: MH412
4      客户公司: 创联世纪传媒有限公司智能科技
5      客户所在地: 文市, 中国
6      产品信息: 索尼 智能体重秤 Basic (类别: 智能体重秤)
7      生产商: 索尼智能科技
8      产品价格: 2042.82
9      评分: 4.9星
10     评价日期: 2024年06月12日
11     评价内容: "这款智能体重秤质量很好,反应灵敏。总体来说很满意。"
```

构建完整的 text 语义信息是第一步，接下来结合 Microsof GraphRAG 的 Workflow 流程，我们还需要进行策略上的优化，具体来说：就是把单条语义信息合并成更大的文本块，从而能在构建索引的单次处理中，尽可能的包含更多的信息，降低构建成本的同时，大幅提升索引构建的效率。在执行这一步的时候，需要考虑的是尽可能的把语义相近的文本块合并在一起，因此相关的策略可参考如下：

- 1. 分组：通过 groupby 分组，例如，所有属于“电子产品”的评论、产品或数据可以被分在一起。这种分组有助于在分析时保持上下文的一致性。
- 2. 设置最大Token：在每个类别内，根据 token 数量动态决定每批次包含的评论数量，避免某些未知的评论过长，超出最大上下文限制，同时太长的文本框会在读取时导致 OOM 错误。

3. 设置分隔符：在合并多行时，通过特殊字符如 `<ROW_SEP>` 来标记单独行之间的边界，用于在 `Microsof GraphRAG` 的 `text_unit` 切分中，能够正确识别出单独的行，制定单独的切分策略。

进行处理的脚本：`preprocess_data.py`，最终生成的文件为 `merged_reviews.csv`

我们看一下里面的内容：

text	ReviewID	ProductID	CustomerID	Rating	ReviewDate	token_count	CategoryName	ProductName	CompanyName
客户ID: VQ899 客户公司: 图龙信息网络有限公司虚拟现实 客户所在地: 银川市, 中国 产品信息: 格力 智能体重秤 Ultra (类别: 智能体重秤) 生产商: 格力智能电器 产品价格: 5991.31 评分: 4.7星 评价日期: 2024年05月30日 评价内容: "这是我用过的最好的智能体重秤之一, 性价比高, 性价比很高。"									
客户ID: YL244 客户公司: 新格林耐特科技有限公司人工智能 客户所在地: 东莞市, 中国 产品信息: 格力 智能体重秤 Ultra (类别: 智能体重秤) 生产商: 格力智能电器 产品价格: 5991.31 评分: 4.3星 评价日期: 2024年05月18日 评价内容: "在多次对比之后选择了这款, 这款智能体重秤真的很不错。尤其是在早晨起床时表现尤为出色。如果非要缺点的话, 配件可以再多一些。便宜没好货, 这句话在这里很适用。"	4	2	VQ899	4.7	2024/5/30	316	智能体重秤	格力 智能	图龙信息网络有限公司虚拟现实

微软GraphRAG对 `.csv` 文件中的每一行 `row` (行), 会看做一个 `Document`, 按照设置的 `size` 和 `overlap` 进行切分, 所以这种切分方法是存在一定问题的, 主要在于: 为了增加索引效率, 我们手动将多行合并成一个 `text`, 如果按照这种切分方法, 极大概率会把某一行原始的 `text` 切分开, 导致这一行的信息丢失, 从而影响索引构建的效果, 基于此, 我们需要手动实现对 `.csv` 文件的切分策略优化。

2.3.3.2 CSV文件切割策略优化

接下来我们要看GraphRAG的源码, 大家在听这个项目之前, 需要学习《GraphRAG深度剖析》的课程, 我这儿默认认为大家学习完了这个课, 所以我这儿就直接基于GraphRAG的源码进行讲解了。

结合 `Microsof GraphRAG` 的内置流程和我们在前一阶段对 `.csv` 文件的预处理, 一种非常行之有效的切分优化策略如下:

- 1. 首先使用分隔符拆分文本, 每个分隔后的部分作为一个完整的评论块处理;
- 2. 对于正常长度的评论块, 按照 `config.size` 进行分组。假设一个text中有两个评论块, 每个token数为20, 而config.size设置为30, 切分策略如下:
 - 首先检查第一个评论块:
 - 当前chunk为空, 所以直接添加第一个评论块 (20 tokens)
 - `current_chunk_texts = [评论块1]`

- `current_chunk_size = 20`
 - 然后检查第二个评论块：
 - 检查添加后是否会超过限制： $\text{current_chunk_size} + \text{row_tokens} = 20 + 20 = 40$
 - 因为 $40 > 30$ (`config.size`)，所以会先保存当前chunk
 - 创建一个包含第一个评论块的chunk并添加到结果中
 - 然后重置当前chunk，并添加第二个评论块
 - `current_chunk_texts = [评论块2]`
 - `current_chunk_size = 20`
 - 最后处理：
 - 处理完所有评论块后，保存最后一个chunk（包含第二个评论块）
 - 所以最终结果是两个chunk，每个包含一个评论块。
3. 当单个评论块超过 `config.size` 时，借助 `chunk_overlap` 参数尽可能的保留完整的语义信息。

述优化策略的实现代码已经集成到 `Microsoft GraphRAG` 源码中，存储位置为：

`graphrag\graphrag\index\operations\chunk_text\csv_strategy.py`。同时，如果要启用我们自定义的切分策略，仅需要修改 `settings.yaml` 文件中 `chunks` 下的 `strategy` 参数为 `csv` 即可，如下所示：

```
82
83 input:
84     type: file # or blob
85     file_type: csv
86     base_dir: "input"
87     file_encoding: utf-8
88     file_pattern: ".*\\.csv$$"
89     text_column: "text" # 如果原始数据中没有text字段，则需要指定一个包含文本内容的字段
90
91
92 chunks:
93     strategy: csv
94     size: 300
95     overlap: 100
96     group_by_columns: [id]
97
98 ### Output settings ###
99 ## If blob storage is specified in the following four sections,
100 ## connection_string and container_name must be provided
101
102 cache:
103     type: file # [file, blob, cosmosdb]
104     base_dir: "cache"
```

```
1 """A module containing run_csv function for CSV text chunking."""
2
3 > import ...
4
5
6
7
8
9
10
11
12
13 def run_csv(
14     texts: Any,
15     config: ChunkingConfig,
16     tick: ProgressTicker,
17 ) -> List[TextChunk]:
18     """
19     按CSV行切分文本，确保不会切断单行内容，并处理<ROW_SEP>分隔符
20
21     参数:
22     | texts: 要切分的文本列表
23     | config: 切分配置
24     | tick: 进度条
25
26     返回:
27     | 切分后的文本块列表
28     """
29     results = []
30
```

至此，我们现在已经优化了 `.csv` 格式文件的预处理和 `text_unit` 的切分，接下来的 `Workflow` 流程将是对每一个 `text_unit` 进行实体和关系的提取。因此，我们要进一步优化的就是 `Prompt` 模板，从而让大模型可以更加精准的提取出我们需要的实体和关系。

执行源码，验证切割思路

代码块

```
1 conda create --name graph python=3.11
2 conda activate graph
3 pip install graphrag==2.1.0
4
5 pip install poetry
6 pip install poethepoet
7 ## 在项目的根目录下执行
8 poetry run poe index --root ./data
```

Poe => `python -m graphrag index --root ./data`

Logging enabled at `/root/autodl-tmp/kefu/rag/graphrag_2.1.0/graphrag/data/logs/indexing-e`

🚀 LLM Config Params Validated

🚀 Embedding LLM Config Params Validated

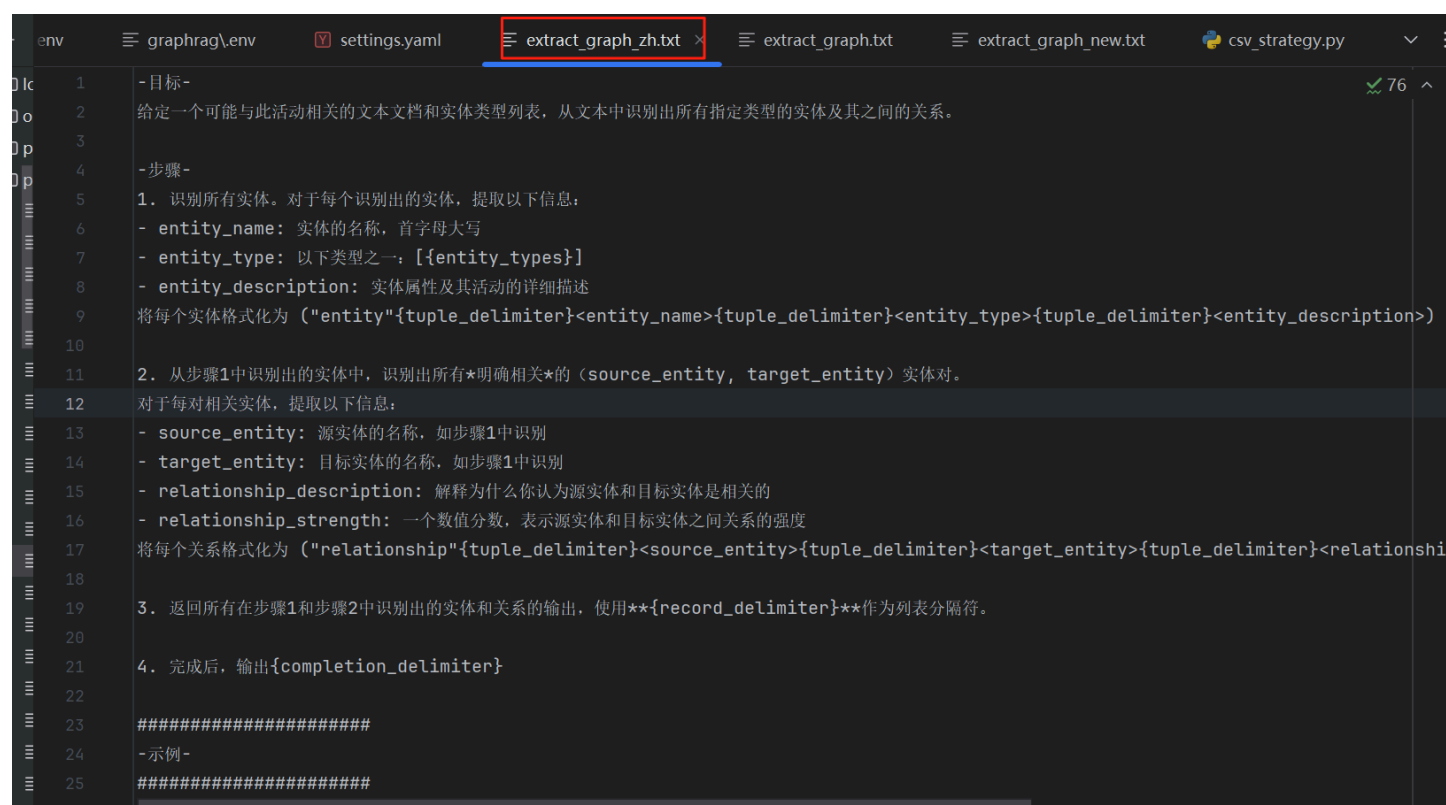
Running standard indexing.

🚀 `create_base_text_units`

	id	n_tokens
0	f0180afb48694508502bbdeae77440a4889c3b2cfc89e3...	292
1	9231c27419f1eb9f945df1613037ffbc656cb369167c4d...	141
2	3757ed3d9681318e0f4a568f6b9f03530700b78558ece3...	192
3	b72d70f75f6c102851539bb59e7663ae00050b69338526...	140
4	22d011473839a756d7a1b7d81427042e3b71fed72cc64f...	182
5	4ebe8af17fb87fc3806c1bc3639c2f48f60a397f18a7c6...	131
6	7bcf21ffdl7c8a768f2c3f51bc2cb4a968b12ec400778...	288
7	9273c8f04e504bd399bbe686aba2ac6390bb9cdd28584d...	159
8	b2a4e0d73fc8alea05c342c4eaad71f32b331d13c0cd80...	148
9	e1b80dbd4d40076f41d3e1b49e26449e574dd4e6eb505d...	140
10	f23a189b272b63dfeee642cc6fc9ad81e7af79d2d2a117...	185
11	1f9854cfb68f3a7b2da3306e03103b79366dff5649c92e...	151
12	4edfe512a184927f0ae83a6cc0361f0179d666dbb2fbea...	152
13	84c8dbe03aa3e57d95fbb01e266dbb5a50984dd4a92aa1...	158
14	2532ad7a3c4d46b4c61c2716269d83e37bfe62e15226a8...	165
15	e6db2c1401ed8669857d2fcc3aa93c94e52108ff2edbec...	198
16	61e55b85ac7695cd72c0d4f3a5fc7e6bde8a2f8e3af1fb...	281
17	04fae203c35f588a2014f8c5de147a7f8b8a8688c734f2...	282
18	d037f6a4144e927274daeceb03bb39a5d945e1e4fec30b...	194
19	cd4d30d65d6ae2eb5fdf6642fd09adc49c7846bd580c81...	172
20	a427fe70ca8c068a2a04ee5beafc725b75ca6226f06168...	297
21	48ab6648ed4f3087cb038c33d207657590687831fe54ec...	152
22	468dc7702a0051eae5b09a4badca69b3356f24fd6f308c...	180
23	3347b90794fb48c9b217d96efa4c0e745d8524842b33a8...	141
24	61e7f6223740113101e4a43ff905dbc99116eba294df05...	175

2.2.3.3 动态提示词生成

在 `Microsoft GraphRAG` 中，实体、关系、社区报告、摘要等信息的提取都是依赖 `Prompt` 模板 + 大模型来生成的。其中，用于实体关系提取的 `Prompt` 模板为 `extract_graph.txt`，其内容如下所示：



```
env  graphrag\env  settings.yaml  extract_graph_zh.txt  extract_graph.txt  extract_graph_new.txt  csv_strategy.py
1  -目标-
2  给定一个可能与此活动相关的文本文档和实体类型列表，从文本中识别出所有指定类型的实体及其之间的关系。
3
4  -步骤-
5  1. 识别所有实体。对于每个识别出的实体，提取以下信息：
6  - entity_name: 实体的名称，首字母大写
7  - entity_type: 以下类型之一: [{entity_types}]
8  - entity_description: 实体属性及其活动的详细描述
9  将每个实体格式化为 ("entity"{tuple_delimiter}<entity_name>{tuple_delimiter}<entity_type>{tuple_delimiter}<entity_description>)
10
11  2. 从步骤1中识别出的实体中，识别出所有*明确相关*的 (source_entity, target_entity) 实体对。
12  对于每对相关实体，提取以下信息：
13  - source_entity: 源实体的名称，如步骤1中识别
14  - target_entity: 目标实体的名称，如步骤1中识别
15  - relationship_description: 解释为什么你认为源实体和目标实体是相关的
16  - relationship_strength: 一个数值分数，表示源实体和目标实体之间关系的强度
17  将每个关系格式化为 ("relationship"{tuple_delimiter}<source_entity>{tuple_delimiter}<target_entity>{tuple_delimiter}<relationship_description>{tuple_delimiter}<relationship_strength>)
18
19  3. 返回所有在步骤1和步骤2中识别出的实体和关系的输出，使用**{record_delimiter}**作为列表分隔符。
20
21  4. 完成后，输出{completion_delimiter}
22
23  #####
24  -示例-
25  #####
```

同时，仅使用 `extract_graph.txt` 模板其默认提取的实体和关系类型为：`organization`、`person`、`geo`、`event`。这个很明显是不太符合我们传入的智能客服场景的，因此，我们需要设计一个更加符合我们业务需求的 `Prompt` 模板。

`Microsoft GraphRAG` 提供了两种方式来调整 `Prompt` 模板，分别是手动调整和自动调整。其中：手动调整指的是通过以纯文本形式编写自定义提示文件来覆盖每个提示，但有一定的限制，即需要包含指定的关键词，稍有不慎就会导致 `Workflow` 流程中断。另外一种方式是自动调整，即通过 `Microsoft GraphRAG` 内置实现的 `Prompt` 模板优化器来调整 `Prompt` 模板，这种方式也是我们推荐的方式。

我们接下来重点介绍启动调整的方式

首先，`Auto Prompt Tuning` 的工作入口函数仍然是在 `graphrag\graphrag\cli\main.py` 文件中的 `_prompt_tune_cli` 函数，位置如下所示：

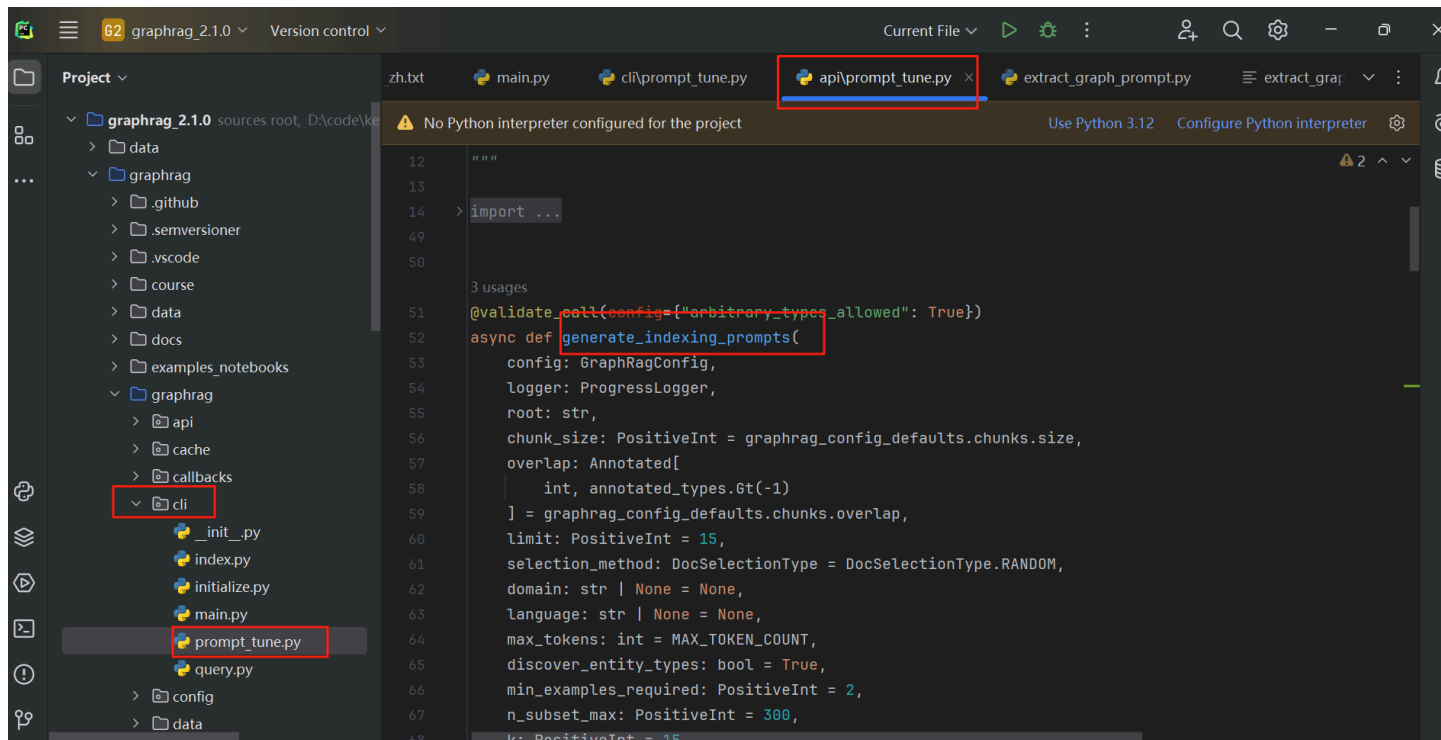
```

243     )
244
245
246     # 这里是进入prompt提示模版领域适配流程的入口
247     @app_command("prompt-tune")
248     def _prompt_tune_cli(
249         root: Annotated[
250             Path,
251             typer.Option(
252                 help="The project root directory.",
253                 exists=True,
254                 dir_okay=True,
255                 writable=True,
256                 resolve_path=True,
257                 autocompletion=path_autocomplete(
258                     file_okay=False, dir_okay=True, writable=True, match_wildcard="*"
259                 ),
260             ),
261         ] = Path(), # set default to current directory
262         config: Annotated[
263             Path | None,
264             typer.Option(
265                 help="The configuration to use.",

```

参数名	描述	状态	默认值
--config	配置文件的路径。这是加载数据和模型设置所必需的。	必需	无
--root	数据项目根目录，包括配置文件（YML、JSON 或 .env）。	可选	当前目录
--domain	与输入数据相关的域，例如“空间科学”、“微生物学”或“环境新闻”。如果留空，则将从输入数据中推断出域。	可选	无
--selection-method	选择文档的方法。选项包括 all、random、auto 或 top。	可选	random
--limit	使用随机或顶部选择时要加载的文本单元的限制。	可选	15
--language	用于输入处理的语言。如果它与输入的语言不同，则将进行翻译。默认值为“”，表示将自动从输入中检测。	可选	自动检测
--max-tokens	提示生成的最大令牌数。	可选	2000
--chunk-size	用于从输入文档生成文本单元的标记大小。	可选	200
--n-subset-max	使用自动选择方法时嵌入的文本块数量。	可选	300
--k	使用自动选择方法时要选择的文档数量。	可选	15
--min-examples-required	实体提取提示所需的最小示例数。	可选	2
--discover-entity-types	允许自动发现和提取实体。当数据涵盖大量主题或高度随机化时，建议使用此选项。	可选	无
--output	保存生成的提示的文件夹。	可选	prompts

接下来我们依然借助源码来详细介绍 `Auto Prompt Tuning` 的实现流程，以及上述各个参数的实际应用价值。其中 `prompt tune` 的源码核心函数位于：`graphrag\graphrag\api\prompt_tune.py` 文件中的 `generate_indexing_prompts` 函数，如下所示：



其核心流程包含以下 11 个步骤：

1. 解析 `settings.yaml` 中配置的参数，通过 `--config` 参数；
2. 找到在 `input` 文件夹中存放的文档，通过 `--root` 参数指定根目录；
3. 将读取到的文档切分成 `text_unit`（块），通过 `--chunk_size` 参数指定切分策略中每个块的大小；
4. 限制用于后续生成提示的原始文档的数量，通过 `--limit` 参数指定；
5. 根据 `--k` 参数，进一步选择具体的 `text_unit` 块，包含三个策略：
 - 如果 `--selection-method` 参数为 `top`，则选择前 `k` 个文本单元；
 - 如果 `--selection-method` 参数为 `random`，则随机选择 `k` 个文本单元；
 - 如果 `--selection-method` 参数为 `auto`，则使用 `k` 和 `n_subset_max` 选择文本单元，具体的实现逻辑是：
 - `--n-subset-max` 用来限制随机采样的取值，表示从 `text_unit` 个文本单元中随机采样 `n_subset_max` 个文本单元，转化成 `Embedding` 向量；
 - 计算采样文本单元的 `Embedding` 向量平均值，即中心点；
 - 计算每个采样文本块到中心点的距离，选择出 `k` 个距离中心点最近的文本单元；
 - 最终选择出来的文本最能代表整个文档集合的一般特征，同时可以过滤掉一些极端的文本块；
6. 设置输入数据所属的场景领域，比如 `电商客服`、`算法分析` 等，用于填充 `Prompt` 模板中的 `domain` 参数，如果未指定，则通过内置提示词 + 语言模型自动生成，最后填充 `Prompt` 模板中的 `domain` 参数；

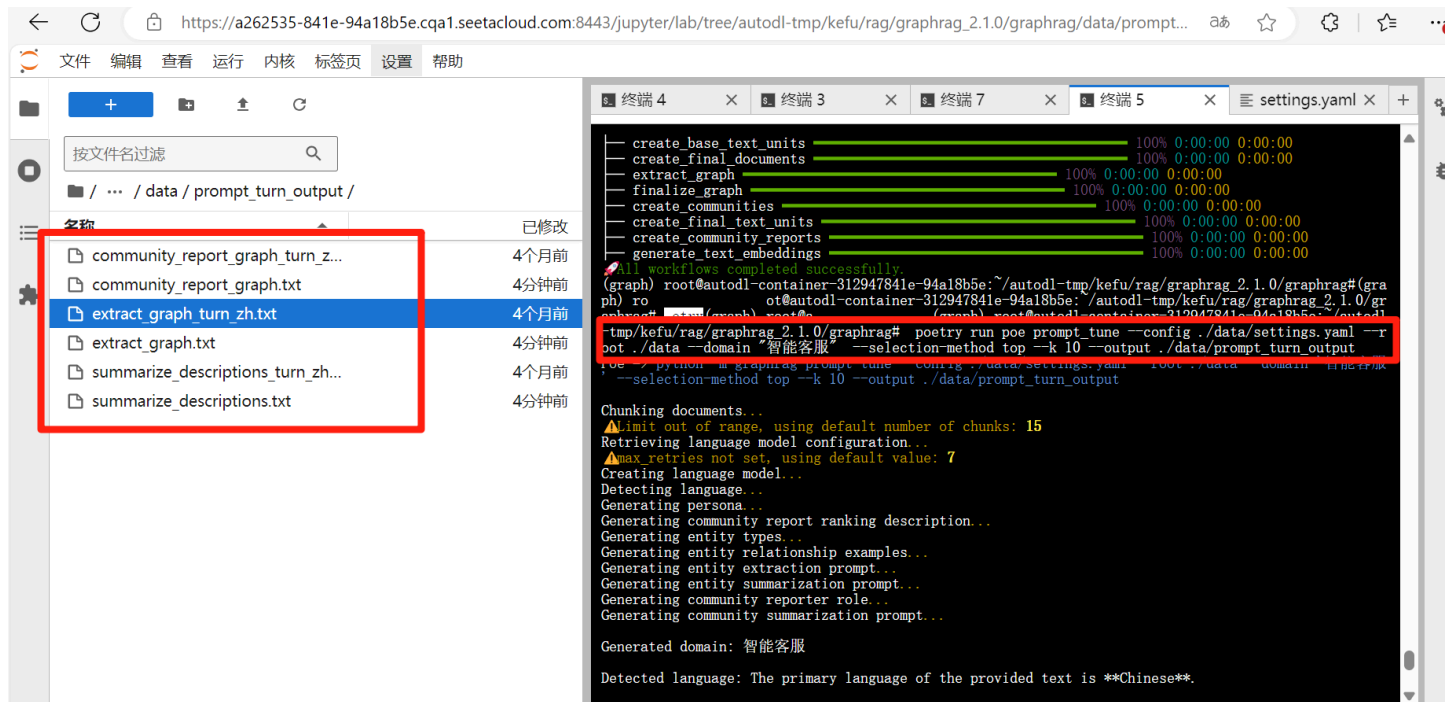
7. 设置最终输出的提示词模版的语言，通过 `--language` 参数指定，如果未指定，则通过内置提示词 + 语言模型自动生成，最后填充 `Prompt` 模板中的 `language` 参数；
8. 根据设置的领域 `domain`，生成模型的系统身份背景信息；
9. 如果设置 `--discover_entity-types`，则会通过提示词 + 语言模型自动发现符合当前输入数据的实体类型；
10. 生成用于生成实体配置的实体/关系示例列表，用来填充 `Prompt` 模板中的 `examples` 参数，即 `Few-shot` 示例；
11. 最后，依次生成用于提取实体、关系、社区报告、摘要的提示词，可以通过 `--min_examples-required` 参数指定每个提示词需要的最小示例数量，以及通过 `--output` 参数指定输出文件夹。

通过以上的源码解读，我们则可以非常清楚的了解如何执行 `Auto Prompt Tuning` 流程。因此在源码环境下，我们可以使用如下命令来执行 `Auto Prompt Tuning` 流程：

代码块

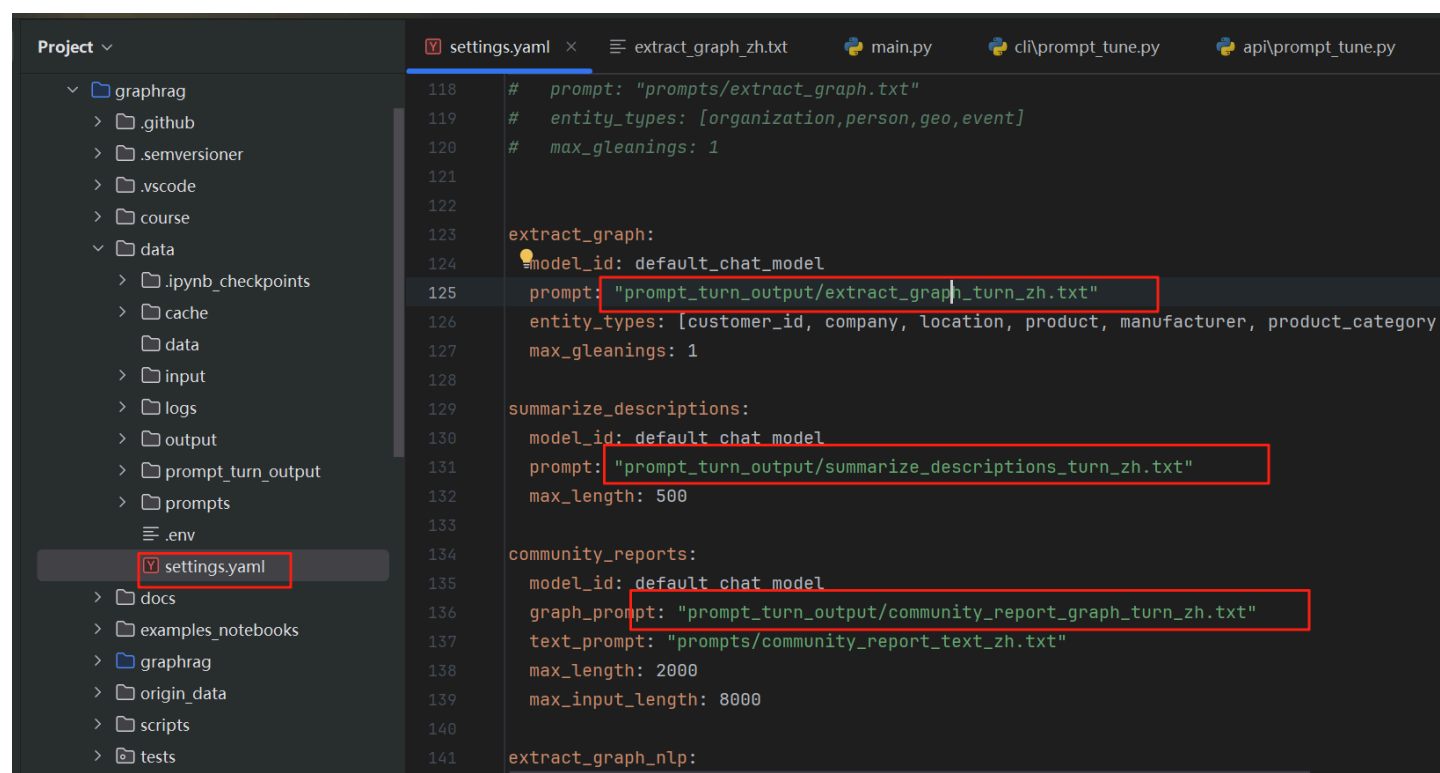
```
1 poetry run poe prompt_tune --config ./data/settings.yaml --root ./data --  
domain "智能客服" --selection-method top --k 10 --output  
./data/prompt_turn_output
```

如下图所示：



这里生成的 `Prompt` 模板文件就是针对我们特定智能客服场景下以及具体的数据，借助大模型生成的符合 `Microsoft GraphRAG` 索引构建 `Workflow` 的提示词。如果想要使用最新的 `Prompt` 模

板文件，则需要将在 `settings.yaml` 文件中配置的 `prompt_template_path` 参数指向最新的 `Prompt` 模板文件。即如下图所示：



修改好配置后，我们就可以使用最新的 `Prompt` 模板文件来构建 `Microsoft GraphRAG` 索引了。仍然是使用 `poetry run poe index` 命令来构建 `Microsoft GraphRAG` 索引，如下所示：

代码块

```
1 poetry run poe index --root ./data/
```

全部执行结束后，依然会正常在 `./data/output` 文件夹中生成对应的 `.parquet` 文件，可以直接用于 `Microsoft GraphRAG` 的检索过程。

2.3.4 非自然语言数据处理

上述的流程我们操作的是非结构化数据，构建的方式是通过 `提示工程+大模型` 借助 `Microsoft GraphRAG` 的 `Workflow` 来构建的。如果大家想操作结构化数据，则需要结合实际的业务场景，将数据集拆分为节点和边，并按照节点类型和边类型分类，符合Neo4j等图数据库的标准导入方式。

这个过程其实没什么技巧可谈，一切以业务需求为驱动，考验的就是大家的业务理解能力和数据处理能力，以及知识图谱的构建能力。比如，我们以 `电商智能客服` 为例，可以先分析一下智能客服场景的需求，主要的点包括：

1. 需要快速检索产品信息
2. 需要了解客户的基本信息和订单历史
3. 需要了解产品与供应商、类别的关系

4. 需要追踪订单状态和物流信息

基于以上需求，我们就可以开始着手划分 Neo4j 的节点和边。其中：

- 节点类型
 - Product - 产品节点
 - Category - 产品类别节点
 - Supplier - 供应商节点
 - Customer - 客户节点
 - Employee - 员工节点
 - Shipper - 物流公司节点
 - Order - 订单节点
- 边类型
 - BELONGS_TO - 产品属于类别
 - SUPPLIED_BY - 产品由供应商提供
 - PLACED_BY - 订单由客户下单
 - PROCESSED_BY - 订单由员工处理
 - SHIPPED_VIA - 订单通过物流公司配送
 - CONTAINS - 订单包含产品
 - REPORTS_TO - 员工上下级关系

节点类型

节点类型	标识	属性
Product	ProductID	ProductName, QuantityPerUnit, UnitPrice, UnitsInStock, UnitsOnOrder, ReorderLevel, Discontinued
Category	CategoryID	CategoryName, Description
Supplier	SupplierID	CompanyName, ContactName, ContactTitle, Address, City, Region, PostalCode, Country, Phone, Fax, HomePage
Customer	CustomerID	CompanyName, ContactName, ContactTitle, Address, City, Region, PostalCode, Country, Phone, Fax
Employee	EmployeeID	LastName, FirstName, Title, BirthDate, HireDate, Address, City, Country, HomePhone, Notes
Shipper	ShipperID	CompanyName, Phone
Order	OrderID	OrderDate, RequiredDate, ShippedDate, Freight, ShipName, ShipAddress, ShipCity, ShipRegion, ShipPostalCode, ShipCountry

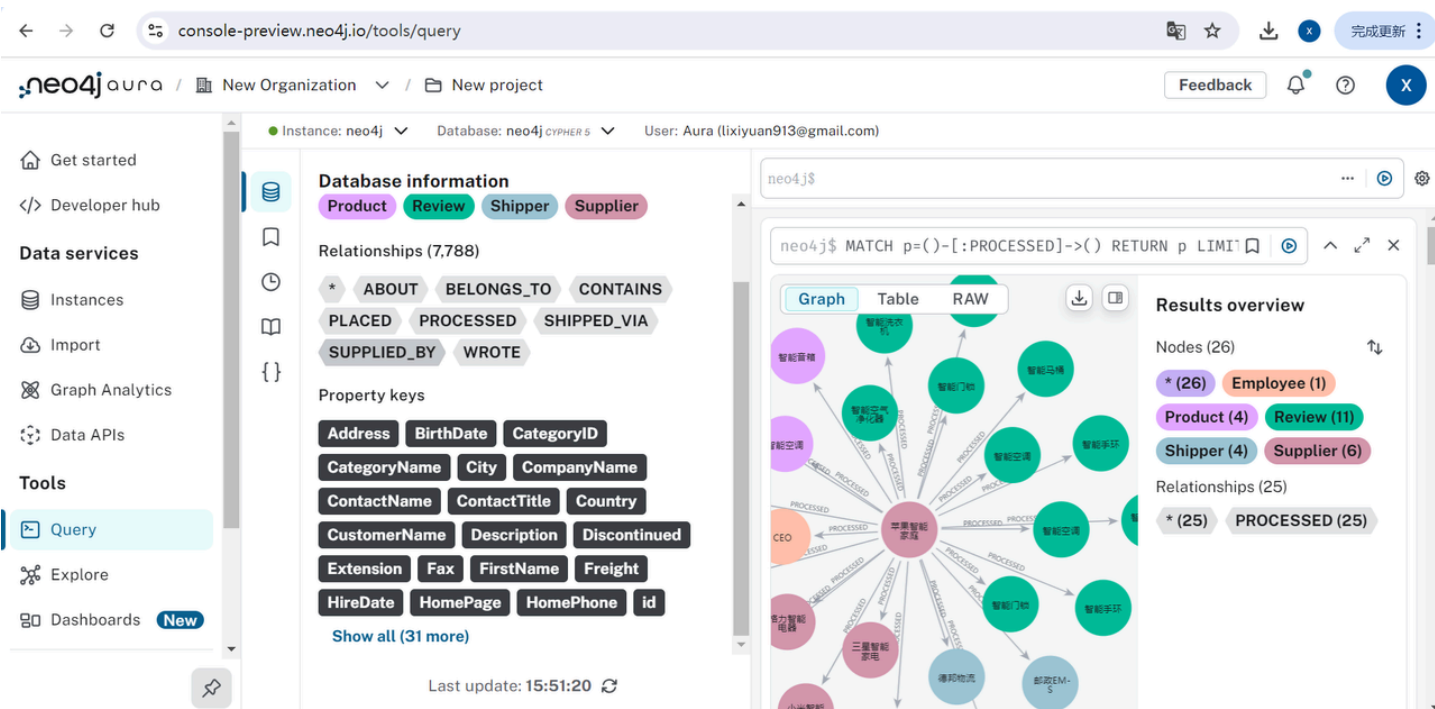
关系类型

边类型	源节点	目标节点	数据源
BELONGS_TO	Product	Category	products.csv (ProductID -> CategoryID)
SUPPLIED_BY	Product	Supplier	products.csv (ProductID -> SupplierID)
PLACED_BY	Customer	Order	orders.csv (CustomerID -> OrderID)
PROCESSED_BY	Employee	Order	orders.csv (EmployeeID -> OrderID)
SHIPPED_VIA	Order	Shipper	orders.csv (OrderID -> ShipVia)
CONTAINS	Order	Product	order_details.csv (OrderID -> ProductID)
REPORTS_TO	Employee	Employee	employees.csv (EmployeeID -> ReportsTo)

先用脚本对CSV文件进行处理，处理完，直接用Python脚本把内容写到neo4j即可：

附件中的csv文件，名字中带有nodes的文件代表是知识图谱中的实体节点数据，请读取对应文件，理解文件里面表示的含义；名字中带有edges的文件代表是知识图谱中的关系数据。请读取上传的csv的文件，生成一个Python脚本，把所有的文件数据都导入到neo4j图数据库中，需要生成节点数据的就生成节点数据，需要生成关系数据的就生成关系数据。导出数据前先删除原来的数据。

图数据库使用在线的neo4j：<https://www.neo4j.com/>



2.4 企业智能客服PDF数据方案设计与落地

2.4.1 电商客服数据准备



此商品为
NR-B21S6

特长

●松下原配压缩机

采用松下原配压缩机，为冰箱的长寿命高效运行提供保障。

●上冷藏下冷冻，使用方便

把平时经常使用的冷藏室配置在上层，内置抽屉式冷冻室置于下层，让您使用起来更加方便。

●时尚外观设计

采用波浪形外观设计，彰显时尚与尊贵。

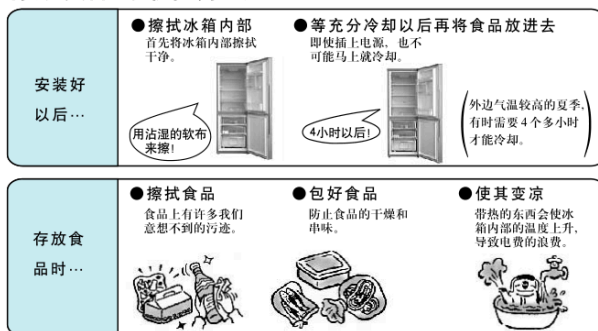
●钢化玻璃搁板

冷藏室内采用钢化玻璃搁板，美观、结实、耐用。

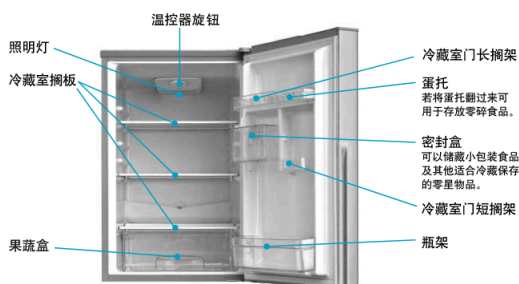
●分类保鲜冷冻室

抽屉式冷冻室，冷冻食品可以分类保存，存取方便。

存放食品的最佳方法



冷藏室的使用方法(零部件名称)



2.4.2 minerU项目安装部署

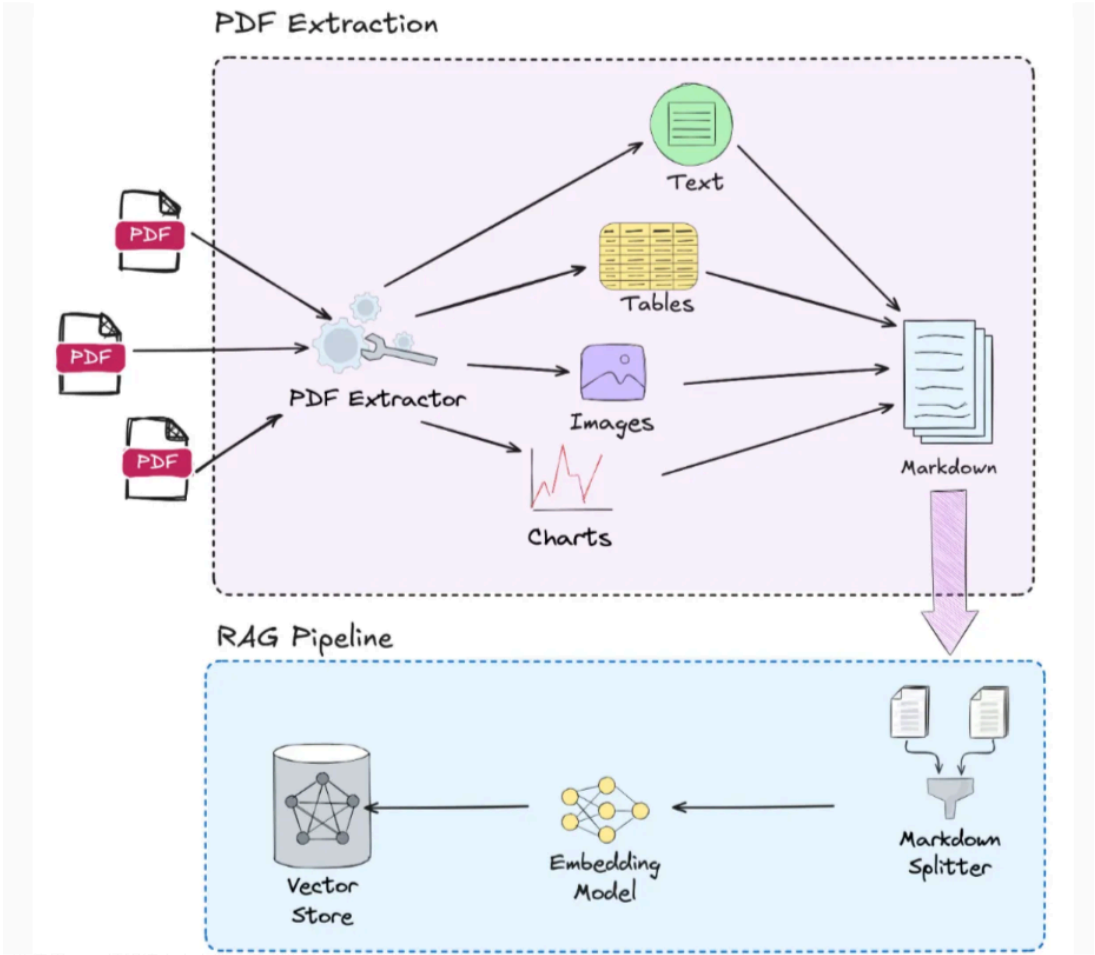
2.4.2.1 PDF格式与Markdown格式

无论是传统 RAG 还是 GraphRAG，PDF 格式文件都是最难处理的文件类型，没有之一。主要原因在于 PDF 格式文件的结构非常复杂，包含文本、图像、表格等多种类型的数据，这几种不同类型的每一部分数据对于 RAG 来说都非常重要，因为很多上下文信息都是通过标题、图片、图表和格式等来传达的，我们需要保留这些信息并进行有效的存储，才能够保证在检索阶段正确识别出有效的文本，从而使大模型能够更好地确定如何“思考”给定内容中提供的信息。

我们在实际构建 RAG 的第一步，往往是先做 PDF Parsing，即 PDF 解析。所谓的 PDF Parsing，指的是提取 PDF 文件中的内容并将其转换为结构化格式的过程，对于 PDF 格式的这种文件特点，就不能像处理 .txt、.csv、.json 等文件那样，直接使用 Python 的 open 函数打开文件，然后逐行读取文件内容，然后进行处理。它涉及分析 PDF 文件的结构和内容以提取有意

义的信息，即把 PDF 文件中的文本、图像、表格和元数据等正确的识别出来，同时还需要解析出 PDF 文件的结构信息，比如：页眉、页脚、页码、章节、段落、标题等。

RAG 系统往往是需要依靠高质量的结构化数据来生成准确且与文本相关的输出。PDF 通常用于官方文件、业务报告和法律合同，包含大量信息，但其布局复杂且数据结构不合理。如果做不到精确的 PDF 解析，关键数据就会丢失，从而导致结果不准确并直接影响 RAG 应用程序的有效性。因此，PDF 格式文件的索引和检索的流程，在 RAG 框架中的实现过程如下图所示：



在 RAG 处理 PDF 格式文件的解析过程中，将其先转换为 Markdown 文件再做后处理是目前通用的做法。Markdown 格式文件自 2023 年起就一直在大模型领域的最流行格式，像 ChatGPT、DeepSeek 等聊天机器人格式化其响应的方式都是使用的 Markdown 语法，采取的做法都是后端实现流式输出，通过 SSE 协议将结果返回给前端，前端再通过 Markdown 语法进行展示。

代码块

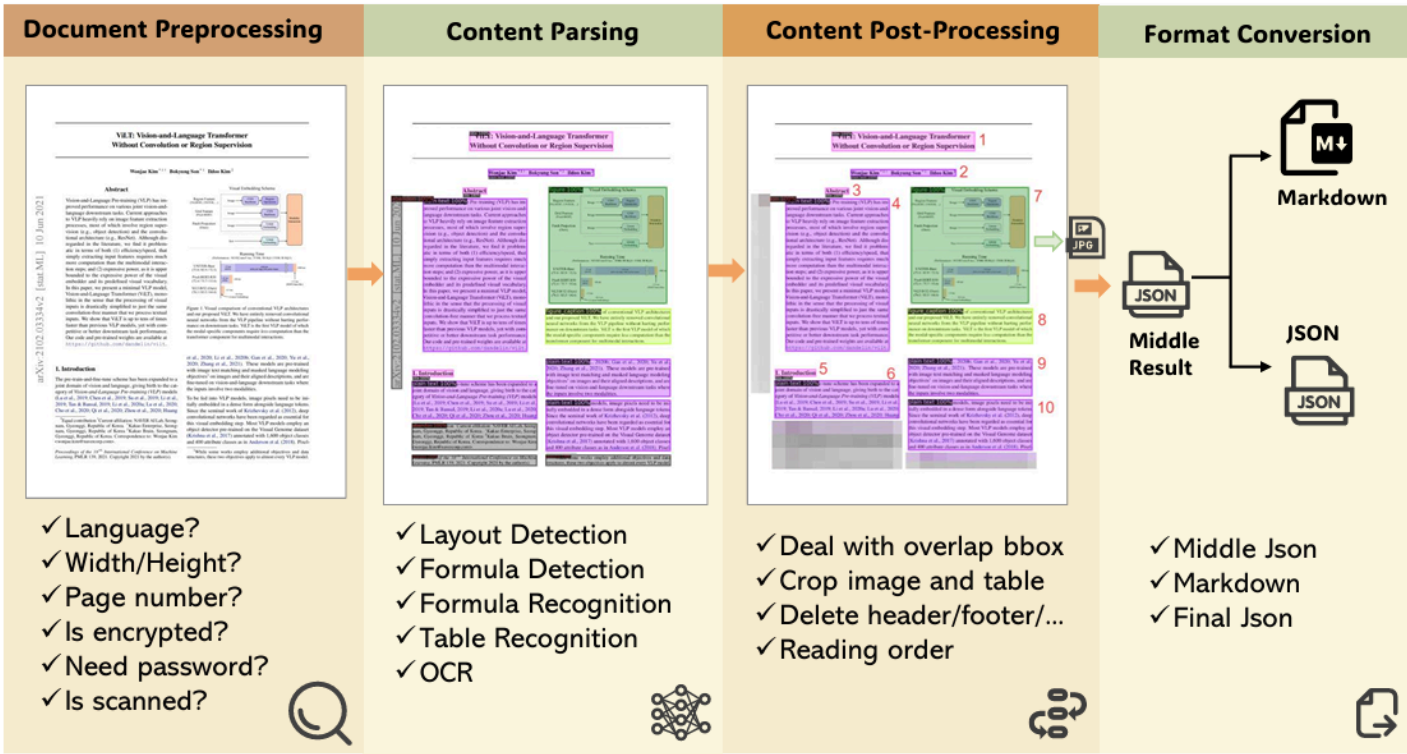
```
1 Markdown 语法的一些基本示例：
2     # 一级标题
3     ## 二级标题
4     ### 三级标题
5
6     **加粗文本**
7
8     *斜体文本*
9
```

```
10      > 引用文本
11
12      [链接文本] (https://www.example.org)
13
14      ...
15      代码文本
16      ...
17
18      | 表头1    | 表头2    |
19      |-----|-----|
20      | 表格数据 | 表格数据 |
```

这种结构优势很明显，比如能给标题、表格、列表、链接等提供结构化的信息，添加了印刷强调元素，例如粗体或斜体，还能提供代码块、数学公式、图片、图表等，即易于编写，又易于阅读，即可以保留文档的原始结构，这包括保留布局、顺序以及不同部分（例如页眉、脚注、表格）之间的连接。对大模型来说，阅读和理解这种类型的输入文档上下文中是非常有效的。也正是因为 Markdown 格式文件的这种优势，所以现在主流的处理 PDF 格式文件的库、框架、工具，都是优先针对 PDF --> Markdown 提取策略展开研究和优化。

2.4.2.2 MinerU项目介绍

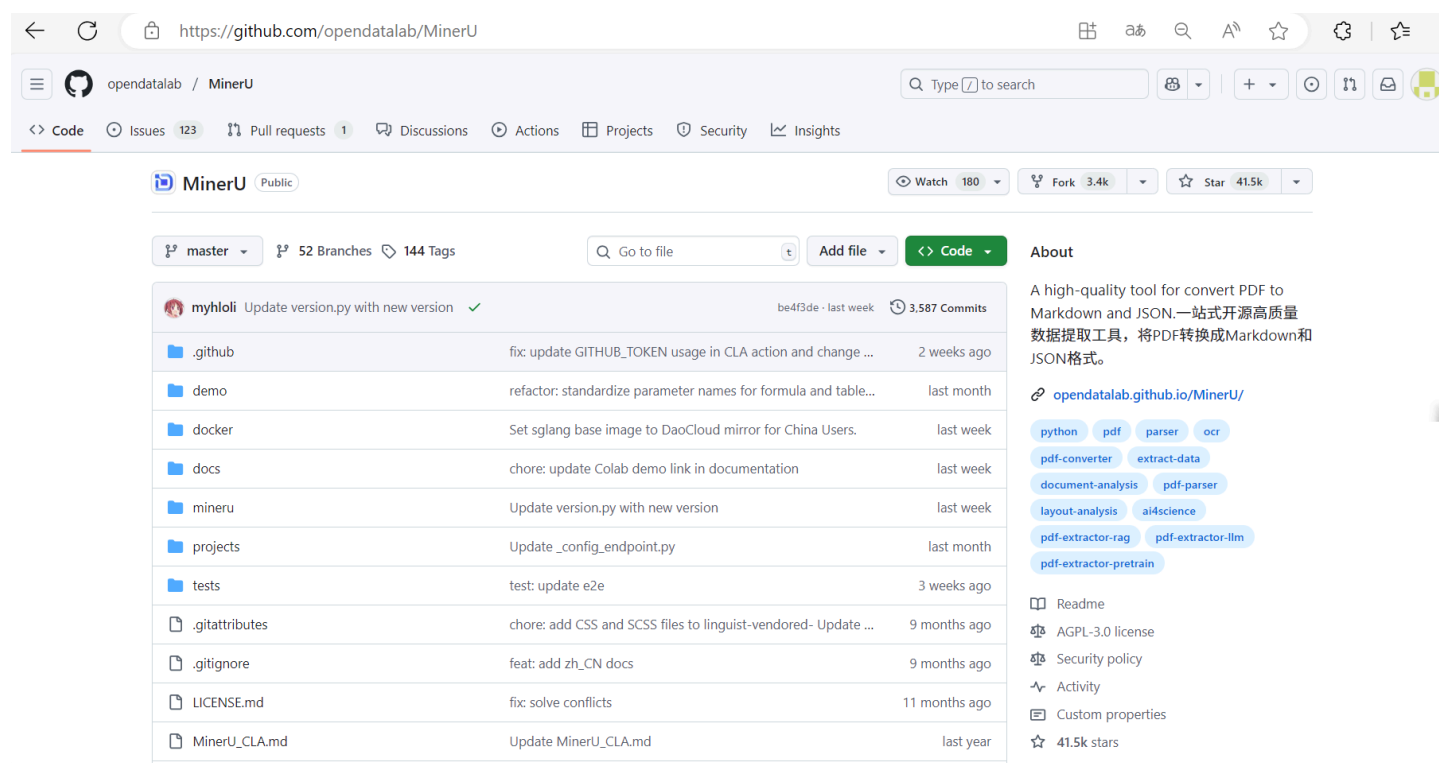
MinerU 是一个非常典型的基于管道的解决方案 (Pipeline-based solution)，并且是一个开源文档解析项目，一共四个核心组件，通过 Pipeline 的设计无缝衔接，实现比较高效、准确的文档解析。如下图所示：（下图来源官方论文：<https://arxiv.org/pdf/2409.18839v1>）



MinerU 的主要工作流程分为以下几个阶段：

- a. **输入**：接收 PDF 格式文本，可以是简单的纯文本，也可以是包含双列文本、公式、表格或图等多模态 PDF 文件；
- b. **文档预处理（Document Preprocessing）**：检查语言、页面大小、文件是否被扫描以及加密状态；
- c. **内容解析（Content Parsing）**：
 - 局分析：区分文本、表格和图像。
 - 公式检测和识别：识别公式类型（内联、显示或忽略）并将其转换为 LaTeX 格式。
 - 表格识别：以 HTML/LaTeX 格式输出表格。
 - OCR：对扫描的 PDF 执行文本识别。
- d. **内容后处理（Content Post-processing）**：修复文档解析后可能出现的问题。比如解决文本、图像、表格和公式块之间的重叠，并根据人类阅读模式重新排序内容，确保最终输出遵循自然的阅读顺序。
- e. **格式转换（Format Conversion）**：以 Markdown 或 JSON 格式生成输出。
- f. **输出（Output）**：高质量、结构良好的解析文档。

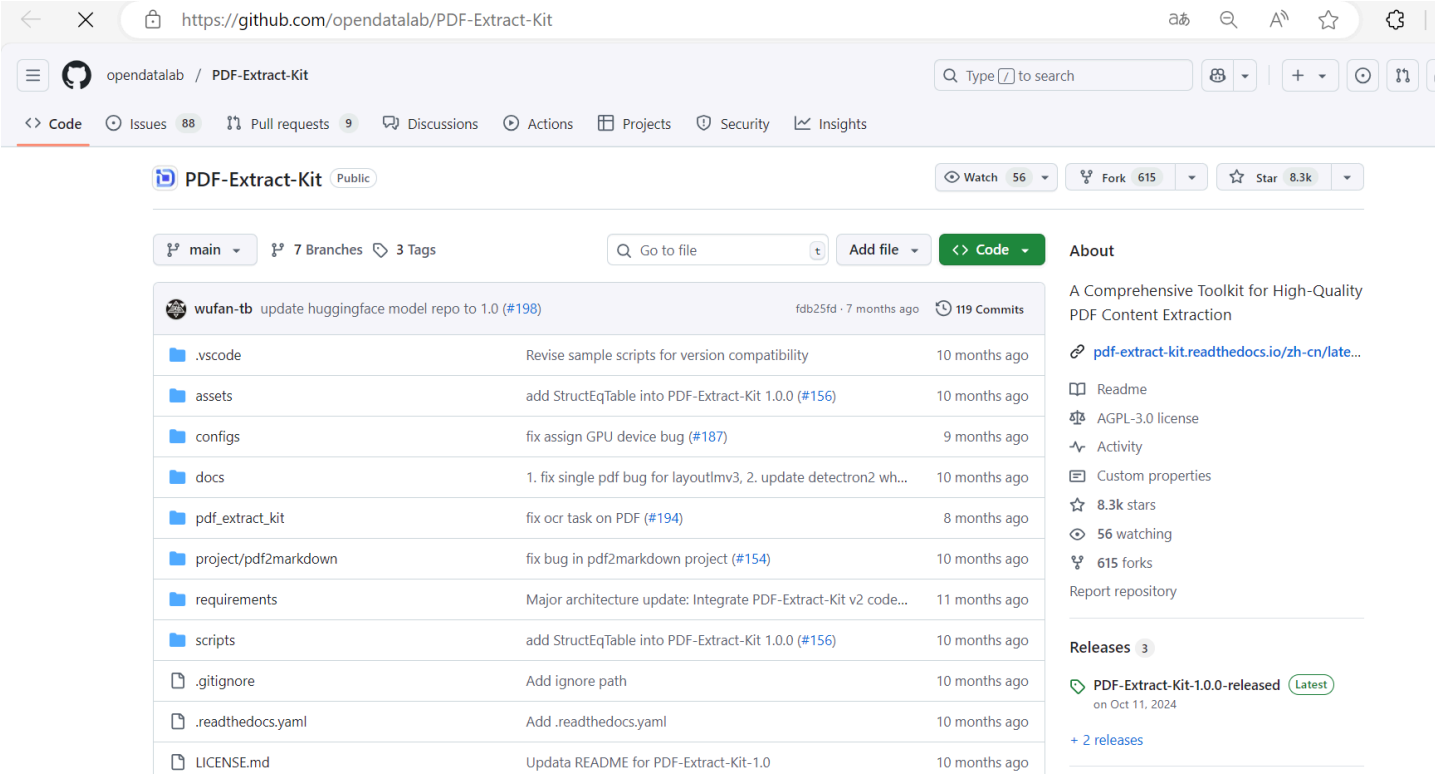
MinerU是Github上面非常火的一个开源项目：<https://github.com/opendatalab/MinerU>



minerU也提供了在线的试用平台：<https://mineru.net/OpenSourceTools/Extractor?source=github>



同时，MinerU 项目于 2024年07月05日 首次开源，底层主要是集成 PDF-Extract-Kit 开源项目做 PDF 的内容提取，PDF-Extract-Kit 也是一个开源项目：
<https://github.com/opendatalab/PDF-Extract-Kit>



PDF-Extract-Kit 这个项目主要针对的是 PDF 文档的内容提取，通过集成众多 SOTA 模型对 PDF 文件实现高质量的内容提取，其中应用到的模型主要包括：

模型类型	模型名称	GitHub 链接	模型下载链接	任务描述
布局检测模型	LayoutLMv3 / DocLayout-YOLO	GitHub / GitHub	模型下载 / 模型下载	定位文档中不同元素位置：包含图像、表格、文本、标题、公式等
公式检测模型	YOLO	GitHub	模型下载	定位文档中公式位置：包含行内公式和行间公式
公式识别模型	UniMERNet	GitHub	模型下载	识别公式图像为latex源码
表格识别模型	StructEqTable	GitHub	模型下载	识别表格图像为对应源码 (Latex/HTML/Markdown)
OCR模型	PaddleOCR	GitHub	模型下载	提取图像中的文本内容（包括定位和识别）

MinerU 与 PDF-Extract-Kit 的关系是：MinerU 结合 PDF-Extract-Kit 输出的高质量预测结果，进行了专门的工程优化，使得文档内容提取更加便捷高效，处理底层原理的优化细节外，主要提升点在以下几点：

- 加入了自研的 `doclayout_yolo(2501)` 模型做布局检测，在相近解析效果情况下比原方案提速 10 倍以上，可以通过配置文件与 `layoutlmv3` 自由切换使用；
- 加入了自研的 `unimernet(2501)` 模型做公式识别，针对真实场景下多样性公式识别的算法，可以对复杂长公式、手写公式、含噪声的截图公式均有不错的识别效果；
- 增加 OCR 的多语言支持，支持 84 种语言的检测与识别，支持列表：
https://paddlepaddle.github.io/PaddleOCR/latest/ppocr/blog/multi_languages.html#5
- 重构排序模块代码，使用 `layoutreader` 进行阅读顺序排序，确保在各种排版下都能实现极高准确率：<https://github.com/ppaanngggg/layoutreader>
- 表格识别功能接入了 `StructTable-InternVL2-1B` 模型，大幅提升表格识别效果，模型下载地址：<https://huggingface.co/U4R/StructTable-InternVL2-1B>

部署和使用方面，MinerU 支持 Linux、Windows、MacOS 多平台部署的本地部署，并且其中用到的布局识别模型、OCR 模型、公式识别模型、表格识别模型都是开源的，我们可以直接下载到本地进行使用。而且，MinerU 项目是完全支持华为昇腾系列芯片的，可适用性非常广且符合国内用户的使用习惯。

因此，接下来我们将重点介绍如何在 Linux 系统下部署 MinerU 项目并进行 PDF 文档解析流程实战。注意：这里强烈建议大家使用 Linux 系统进行部署，其支持性和兼容性远高于 Windows 系统。

2.4.3 minerU工程化落地方案设计

2.4.3.1 minerU安装部署

部署minerU的环境要求

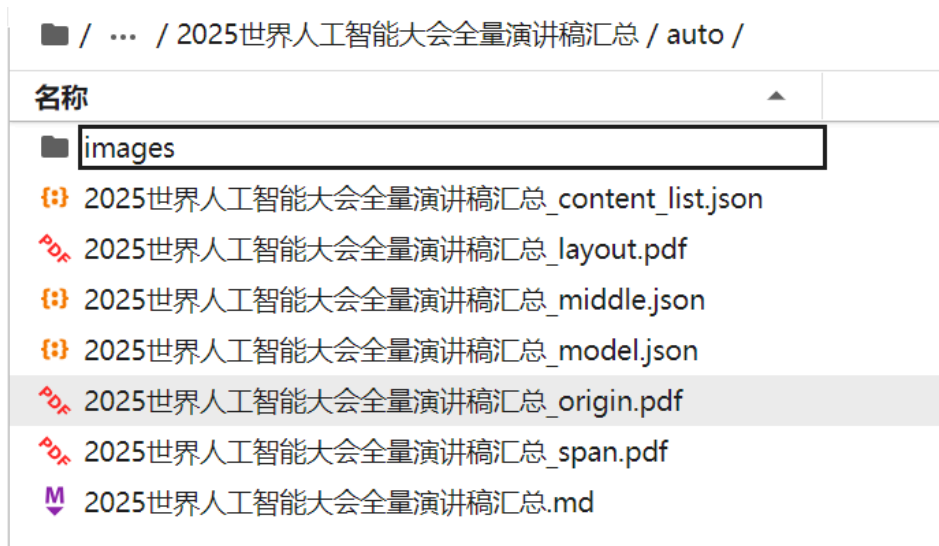
Parsing Backend	pipeline	vlm-transformers	vlm-sglang
Operating System	Linux / Windows / macOS	Linux / Windows	Linux / Windows (via WSL2)
CPU Inference Support	✔	✘	
GPU Requirements	Turing architecture and later, 6GB+ VRAM or Apple Silicon	Turing architecture and later, 8GB+ VRAM	
Memory Requirements	Minimum 16GB+, recommended 32GB+		
Disk Space Requirements	20GB+, SSD recommended		
Python Version	3.10-3.13		

代码块

```
1  ##构建虚拟环境，执行环境路径（空间较大），避免依赖过大
2  conda create -p /root/autodl-tmp/data/mineru python=3.10
3  conda activate /root/autodl-tmp/data/mineru
4
5  ## 用git下载代码
6  git clone https://github.com/opendatalab/MinerU.git
7  cd MinerU
8  ## 项目用uv工具管理，先下个uv
9  pip install uv
10 ## 安装依赖
11 uv pip install -e .[core] -i https://mirrors.aliyun.com/pypi/simple
12
13 ## 下载模型(也可以不执行，执行了会下载较多的模型，占用空间)
14 mineru-models-download
15
16 ## 执行命令
17 mineru -d cuda -p ../2025世界人工智能大会全量演讲稿汇总.pdf -o ../output --source modelscope
```

mineru --help

选项	描述
<code>-v, --version</code>	显示版本并退出
<code>-p, --path PATH</code>	本地文件路径或目录。支持 PDF、PPT、PPTX、DOC、DOCX、PNG、JPG 文件 [必需]
<code>-o, --output-dir PATH</code>	输出本地目录 [必需]
<code>-m, --method ocr、txt、auto</code>	解析 PDF 的方法。ocr: 使用 OCR 技术从 PDF 中提取信息。txt: 适用于仅基于文本的 PDF, 性能优于 OCR。auto: 自动选择最佳解析方法。未指定方法时, 默认使用 auto。
<code>-l, --lang TEXT</code>	输入 PDF 中的语言 (如果已知) 以提高 OCR 准确性。可选。您应该输入“缩写”, 语言形式请参见 PaddleOCR 多语言支持
<code>-d, --debug BOOLEAN</code>	在执行 CLI 命令期间启用详细调试信息。
<code>-s, --start INTEGER</code>	PDF 解析的起始页, 从 0 开始。
<code>-e, --end INTEGER</code>	PDF 解析的结束页, 从 0 开始。
<code>--help</code>	显示此帮助信息并退出。



这里得到的解析文件，就是将用于我们后续构建 `GraphRAG` 时用来作为分块的原始文件，即 `xxx.md`，同时搭配其他的 `.json` 文件，可以用来构建丰富的上下文语义信息及语义增强。因此，我们需要了解每个文件中存储的核心结构及内容，分别如下：

1. `images`: 这是一个文件夹，里面存储了 `PDF` 文档中出现的所有图片；
2. `.origin.pdf`: 进行解析的原始 `PDF` 文件；
3. `.markdown`: `PDF` 文档的解析结果，输出为 `xxx.md` 的 `Markdown` 格式；
4. `.layout.pdf`: 这个文件用不同的背景色块圈定不同的内容块，以此来区分整体的布局识别结果；
5. `.model.json`: 这个文件的主要作用是将 `PDF` 从像素级别的展示转换为结构化的数据，使计算机能够“理解”文档的组成部分。主要存储的信息是：
 - 文档布局识别：存储文档中的各种元素（标题、正文、图表等）及其在页面上的精确位置

- 内容分类：将识别出的元素分类为不同类型（如标题、表格、公式等）
- 特殊内容解析：对于公式、表格等特殊内容，提供其结构化表示（如LaTeX、HTML格式）

其示例数据如下：

代码块

```
1 [{"layout_dets": [{"category_id": 2,  
2 "poly": [99.1906967163086,  
3 100.3119125366211,  
4 730.3707885742188,  
5 100.3119125366211,  
6 730.3707885742188,  
7 245.81326293945312,  
8 99.1906967163086,  
9 245.81326293945312]},  
10 "score": 0.9999997615814209}],  
11 "page_info": {"page_no": 0,  
12 "height": 2339,  
13 "width": 1654}},  
14 {"layout_dets": [{"category_id": 5,  
15 "poly": [99.13092803955078,  
16 2210.680419921875,  
17 497.3183898925781,  
18 2210.680419921875,  
19 497.3183898925781,  
20 2264.78076171875,  
21 99.13092803955078,  
22 2264.78076171875]},  
23 "score": 0.9999997019767761}],  
24 "page_info": {"page_no": 1,  
25 "height": 2339,  
26 "width": 1654}}}]
```

其中每一个 `layout_dets` 中存储了 PDF 文档中每一页的布局识别结果，`poly` 是四边形坐标，分别是左上、右上、右下、左下四点的坐标，形式为：`[x0, y0, x1, y1, x2, y2, x3, y3]`，分别表示左上、右上、右下、左下四点的坐标，`page_info` 是 PDF 文档中每一页元数据，包含页码、高度、宽度信息，而 `category_id` 为 PDF 文档中每一页的布局识别结果的类型，分别为：

`category_id` 的类别

类别 ID	类别名称	描述
0	title	标题
1	plain_text	文本
2	abandon	包括页眉页脚页码和页面注释
3	figure	图片
4	figure_caption	图片描述
5	table	表格
6	table_caption	表格描述
7	table_footnote	表格注释
8	isolate_formula	行间公式
9	formula_caption	行间公式的标号
13	embedding	行内公式
14	isolated	行间公式
15	text	OCR 识别结果

这个 `.json` 文件主要可以内容提取，比如精确提取特定类型的内容（如所有表格或公式），搜索，比如实现对文档内容的结构化搜索，文档分析，比如分析文档的结构特征（如有多少图表、公式密度等）等需求场景。

6. `_middle.json`: 这个文件的核心作用是存储 `PDF` 文档解析过程中多层次结构化数据，它比 `model.json` 提供了更详细的层次结构和内容信息, 其示例数据是这样的：

代码块

```
1  {"pdf_info": [{"preproc_blocks": [{"type": "text",
2      "bbox": [52,
3      61.956024169921875,
4      294,
5      82.99800872802734]},
6      "lines": [{"bbox": [52,
7      61.956024169921875,
8      294,
9      72.0000228881836]},
10     "spans": [{"bbox": [54.0,
11     61.956024169921875,
12     296.2261657714844,
13     72.0000228881836]},
14     "content": "dependent on the service
    headway and the reliability of the departure ",
15     "type": "text",
16     "score": 1.0}}]}],
17  "layout_bboxes": [{"layout_bbox": [52,
18      61,
```

```

19             294,
20             731],
21             "layout_label": "V",
22             "sub_layout": []}],
23         "page_idx": 0,
24         "page_size": [612.0,
25             792.0],
26         "_layout_tree": [],
27         "images": [],
28         "tables": [],
29         "interline_equations": [],
30         "discarded_blocks": [],
31         "para_blocks": [{"type": "text",
32             "bbox": [52,
33                 61.956024169921875,
34                 294,
35                 82.99800872802734],
36             "lines": [{"bbox": [52,
37                 61.956024169921875,
38                 294,
39                 72.00000228881836],
40                 "spans": [{"bbox": [54.0,
41                     61.956024169921875,
42                     296.2261657714844,
43                     72.00000228881836],
44                     "content": "dependent on the service
45 headway and the reliability of the departure ",
46                     "type": "text",
47                     "score": 1.0}}]}],
48         "_parse_type": "txt",
49         "_version_name": "0.6.1"}

```

解析结果是一个多层嵌套，从大到小依次为：PDF 文档 → 包含多个页面 → 包含多个区块(blocks) → 分为一级区块和二级区块 → 每个区块包含多行 → 每行包含多个最小单元，其中 `para_blocks` 内存储的元素为区块信息，`span` 是所有元素的最小存储单元。大家理解了这个结构后，可以结合下图对每个字段进行理解，我们这里不再赘述。

一级

字段名	解释
pdf_info	list，每个元素都是一个 dict，这个 dict 是每一页 PDF 的解析结果，详见下表
_parse_type	ocr
_version_name	string，表示本次解析使用的 magic-pdf 的版本号

二级

字段名	解释
preproc_blocks	PDF 预处理后，未分段的中间结果
layout_bboxes	布局分割的结果，含有布局的方向（垂直、水平），和 bbox，按阅读顺序排序
page_idx	页码，从 0 开始
page_size	页面宽度和高度
_layout_tree	布局树状结构
images	list，每个元素是一个 dict，每个 dict 表示一个 img_block
tables	list，每个元素是一个 dict，每个 dict 表示一个 table_block
interline_equations	list，每个元素是一个 dict，每个 dict 表示一个 interline_equation_block
discarded_blocks	list，模型返回的需要 drop 的 block 信息
para_blocks	将 preproc_blocks 进行分段之后的结果

三级（最外层block）

字段名	解释
type	block 类型（table 或 image）
bbox	block 矩形框坐标
blocks	list，里面的每个元素都是一个 dict 格式的二级 block

四级（内层block）

字段名	解释
type	block 类型
bbox	block 矩形框坐标
lines	list，每个元素都是一个 dict 表示的 line，用来描述一行信息的构成
----- --	-----
其中 type 类型	描述
image_body	图像的本体
image_caption	图像的描述文本
image_footnote	图像脚注
table_body	表格本体
table_caption	表格的描述文本
table_footnote	表格脚注
text	文本块
title	标题块
index	目录块
list	列表块
interline_equation	行间公式块

五级（lines）

字段名	解释
bbox	line 的矩形框坐标
spans	list，每个元素都是一个 dict 表示的 span，用来描述一个最小组成单元的构成

六级（spans）

字段名	解释
bbox	span 的矩形框坐标
type	span 的类型
content	img_path
-----	-----
--	
其中 type 类型	描述
type	描述
image	图片
table	表格
text	文本
inline_equation	行内公式
interline_equation	行间公式

该 `.json` 文件除了能像 `model.json` 一样提取特定类型的内容，因为其内容的丰富程度，可以做更精细化的文本分析，比如段落级别的文本分析，建立更精确的搜索索引，支持按内容类型搜索等。

7. `span.pdf`: 根据 `span` 类型的不同，采用不同颜色线框绘制页面上所有 `span`。该文件可以用于质检，可以快速排查出文本丢失、行间公式未识别等问题。

8. 最后一个 `.content_list.json` 文件，是 `PDF` 文档中所有内容的列表，使用 `JSON` 格式存储数据，每个元素都是一个字典，包含了不同类型的内容（文本、图像等）。

代码块

```
1  [{
2      // 文本内容块
3      "type": "text",
4      "text": "...",
5      "text_level": N, // 可选字段
6      "page_idx": N
7  },
8  {
9      // 图像内容块
10     "type": "image",
11     "img_path": "...",
12     "img_caption": [],
13     "img_footnote": [],
14     "page_idx": N
15 },
16 {
17     // 表格内容块
```

```

18         "type": "table",
19         "img_path": "...",
20         "table_caption": [],
21         "table_footnote": [],
22         "table_body": "...",
23         "page_idx": N
24     }
]
```

字段说明

字段名	出现位置	数据类型	说明
type	所有内容块	字符串	内容块的类型，可能的值有："text"、"image"、"table"
page_idx	所有内容块	整数	内容所在的页码，从0开始计数
text	文本类型块	字符串	文本内容
text_level	部分文本类型块	整数	文本的层级，通常用于标题，1表示一级标题
img_path	图像和表格类型块	字符串	图像或表格截图的文件路径
img_caption	图像类型块	数组	图像的说明文字，通常为空数组
img_footnote	图像类型块	数组	图像脚注，通常为空数组
table_caption	表格类型块	数组	表格的说明文字，通常为空数组
table_footnote	表格类型块	数组	表格脚注，通常为空数组
table_body	表格类型块	字符串	表格的HTML内容，包含完整的表格结构

以上是对 MinerU 解析结果的说明，通过这些文件，我们基本上可以获取到 PDF 文档中所有内容的位置、类型、内容等信息，理解各个文件的结构，也是我们后续进行自定义处理的关键，所以这里大家务必明确文件及其内部结构的组成。除此以外，除了 PDF 格式，该流程也可以支持图像（.jpg 及.png）、PDF、Word（.doc及.docx）、以及PowerPoint（.ppt及.pptx）在内的多种文档格式的解析，大家可以进行尝试。

实践到这里，我们通过 CLI 工具快速实现了 MinerU 对 PDF 文档的解析过程，这个过程并不是很复杂。除此之外，如果想更灵活的控制解析过程，一种更主流的方式是通过 Python API 接口来实现，这可以在官方的 API Docs 中找到详细的说明：

https://mineru.readthedocs.io/en/latest/user_guide/usage/api.html，大家可以在 Python IDE 环境中根据官方的示例进行接口级别的解析测试。

2.4.3.2 minerU封装 RESTful API 接口

MinerU 作为一个比较重的工具，其解析过程涉及调用多个模型，所以其解析速度相对较慢，所以需要在解析效果和响应速度之间进行平衡。除此以外，在 Microsoft GraphRAG 项目中，我们主要将其作为 PDF 文档解析工具，用于将 PDF 文档解析为 Markdown 格式，然后作为

GraphRAG 的输入，进行知识图谱的构建。而如果集成到 Microsoft GraphRAG 项目中，则会产生项目上的冗余，不利于后续的维护，所以这里我们需要采用服务化的方式，将 MinerU 作为一个独立的服务，在 Microsoft GraphRAG 项目中只保留 MinerU 的调用接口。

因此，我们需要将 MinerU 封装成一个服务，并提供 RESTful API 接口，以便 Microsoft GraphRAG 项目中直接进行调用。在这个封装的过程中，我们再去结合具体使用的框架综合考虑效率优化等问题。

这里我们采用的是一个 LitServe 框架去做多 GPU 的并行解析，并基于该框架提供 RESTful API 接口。

我们之前的项目经常使用 FastAPI 框架去封装后端服务的 RESTful API 接口，而 LitServe 框架适用于基于 FastAPI 构建的模型后端服务，主要通过批处理、流式处理和 GPU 自动扩缩等功能增强了 FastAPI，并且其评测结果显示会比普通的 FastAPI 快至少2倍。其开源地址如下：<https://github.com/Lightning-AI/LitServe>



LitServe

From the creators of PyTorch Lightning

Most serving engines serve one model with rigid abstractions. LitServe lets you serve any model (vision, audio, text) and build full AI systems - agents, chatbots, MCP servers, RAG, pipelines - with full control, batching, multi-GPU, streaming, custom logic, multi-model support, and zero YAML.

Self host or deploy in one-click to [Lightning AI](#).

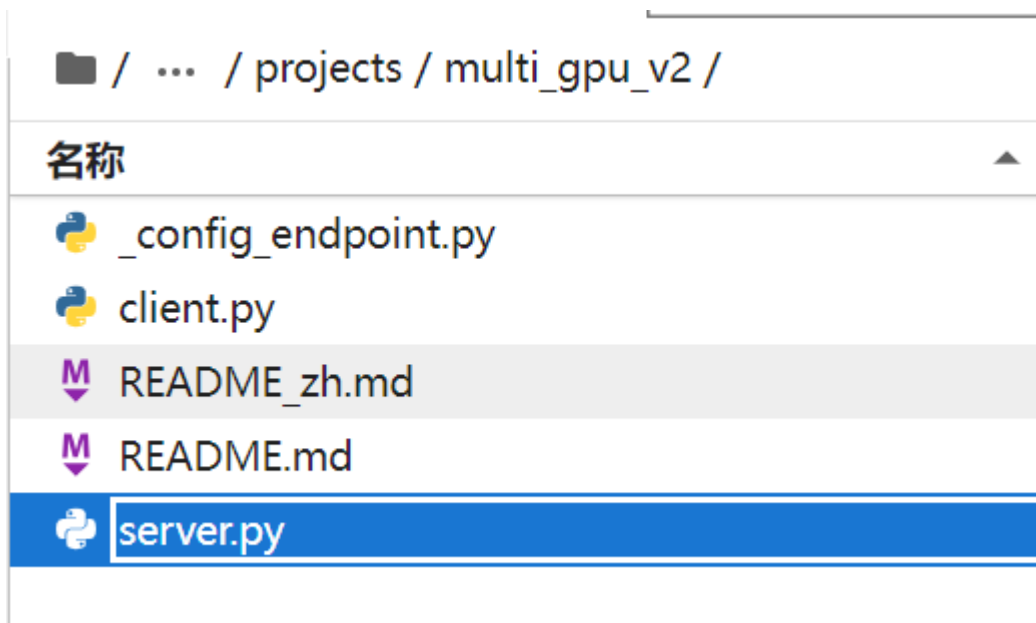
- | | | |
|--------------------------|--------------------------|--------------------------------|
| ✓ Build full AI systems | ✓ 2× faster than FastAPI | ✓ Agents, RAG, pipelines, more |
| ✓ Custom logic + control | ✓ Any PyTorch model | ✓ Self-host or managed |
| ✓ Multi-GPU autoscaling | ✓ Batching + streaming | ✓ BYO model or vLLM |
| ✓ No MLOps glue code | ✓ Easy setup in Python | ✓ Serverless support |

downloads 419k Get help on Discord 810 online CI testing passing codecov 85% License Apache 2.0

[Quick start](#) • [Examples](#) • [Features](#) • [Performance](#) • [Hosting](#) • [Docs](#)

 Get started

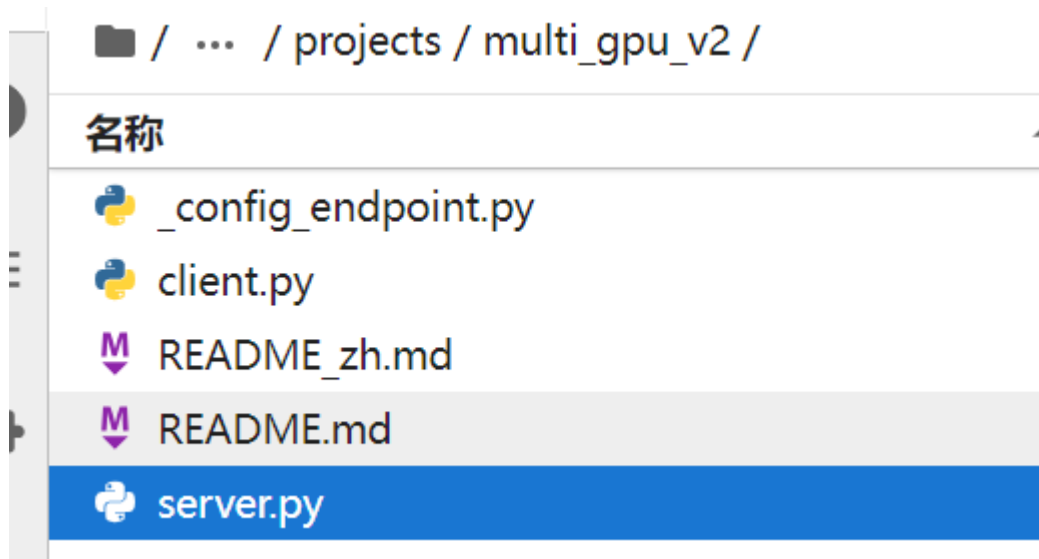
MinerU 的源码中提供了一个 LitServe 的基础服务，其源码存放位置为 MinerU-master/projects 中的 multi_gpu_v2 文件夹。



代码块

```
1 cd /root/autodl-tmp/MinerU/projects/multi_gpu_v2
```

我们进入 `multi_gpu_v2` 文件夹，可以看到 `LitServe` 的基础服务，其提供了三个文件，分别是 `server.py`、`client.py` 和 `README.md`。其中 `server.py` 文件是 `LitServe` 的基础服务，`client.py` 文件是 `LitServe` 的客户端，`README.md` 文件是 `LitServe` 的说明文档。而我们需要调整的是 `server.py` 文件，其内部会定义 `LitServe` 的 `RESTful` API 接口，并提供 `LitServe` 的 `RESTful` API 接口的调用方法。



启动服务之前，需要修改`server.py`文件

终端3 × 2025世界 × 6. Micros × 终端1 × 终端2 × server.py × +

```
1 import os
2 import base64
3 import tempfile
4 from pathlib import Path
5 import litserve as ls
6 from fastapi import HTTPException
7 from loguru import logger
8
9 from mineru.cli.common import do_parse, read_fn
10 from mineru.utils.config_reader import get_device
11 from mineru.utils.model_utils import get_vram
12 from _config_endpoint import config_endpoint
13
14 class MinerUAPI(ls.LitAPI):
15     def __init__(self, output_dir='/root/autodl-tmp/mineruoutput'):
16         super().__init__()
17         self.output_dir = output_dir
18
19     def setup(self, device):
20         """Setup environment variables exactly like MinerU CLI does"""
21         logger.info(f"Setting up on device: {device}")
22
```

```
83         end_page_id=inputs['end_page_id']
84     )
85
86     return str(output_dir)
87
88     except Exception as e:
89         logger.error(f"Processing failed: {e}")
90         raise HTTPException(status_code=500, detail=str(e))
91     finally:
92         # Cleanup temp file
93         if Path(input_path).exists():
94             Path(input_path).unlink()
95
96     def encode_response(self, response):
97         return {'output_dir': response}
98
99 if __name__ == '__main__':
100     server = ls.LitServer(
101         MinerUAPI(output_dir='/root/autodl-tmp/mineruoutput'),
102         accelerator='auto',
103         devices='auto',
104         workers_per_device=1,
105         timeout=False
106     )
107     logger.info("Starting MinerU server on port 8000")
108     server.run(port=8899, generate_client_file=False)
109
```

行 108, 列 25 空格: 4 server.py 1

代码块

- 1 #完成以上自定义配置后，回到服务器终端，依次执行如下三条命令安装LitServe服务运行所需要的依赖环境：
- 2 `pip install -U litserve python-multipart filetype`
- 3 `pip install paddlepaddle-gpu==3.0.0b1 -i`
`https://www.paddlepaddle.org.cn/packages/stable/cu118`
- 4
- 5 # 启动脚本
- 6 `python server.py`

如果遇到下面的报错：

```

File /root/autodl-tmp/data/my_mineru/lib/python3.10/site-packages/modelscope/utils/ast_utils.py, line 24, in <module>
    from modelscope.utils.registry import default_group
File "/root/autodl-tmp/data/my_mineru/lib/python3.10/site-packages/modelscope/utils/registry.py", line 11, in <module>
    logger = get_logger()
File "/root/autodl-tmp/data/my_mineru/lib/python3.10/site-packages/modelscope/utils/logger.py", line 49, in get_logger
    from modelscope.utils.torch_utils import is_dist, is_master
File "/root/autodl-tmp/data/my_mineru/lib/python3.10/site-packages/modelscope/utils/torch_utils.py", line 13, in <module>
    import torch
File "/root/autodl-tmp/data/my_mineru/lib/python3.10/site-packages/torch/__init__.py", line 416, in <module>
    from torch._C import * # noqa: F403
ImportError: /root/autodl-tmp/data/my_mineru/lib/python3.10/site-packages/torch/lib/libtorch_cuda.so: undefined symbol: ncclGroupSimulat
eEnd

```

解决方案：

升级 NCCL 到兼容版本

#通过 Conda 安装最新 NCCL (推荐)

```
conda install -c nvidia nccl
```

检查并修复 PyTorch 环境

#确保 PyTorch 版本与 CUDA 匹配

```
pip uninstall torch torchvision torchaudio
```

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118 # 替
换为您的 CUDA 版本
```

代码块

```

1  # 再次重启
2  python server.py

```

```

(/root/autodl-tmp/data/my_mineru) root@autodl-container-61bb4e82b9-e3a00b14:~/autodl-tmp/MinerU/projects/multi_gpu_v2# python server.py
2025-08-11 12:15:26.211 | WARNING | mineru.backend.vlm.predictor:<module>:35 - sglang is not installed. If you are not using sglang, yo
u can ignore this warning.
2025-08-11 12:15:26.419 | INFO | __main__:<module>:107 - Starting MinerU server on port 8000
INFO: Uvicorn running on http://0.0.0.0:8899 (Press CTRL+C to quit)
2025-08-11 12:15:39.322 | WARNING | mineru.backend.vlm.predictor:<module>:35 - sglang is not installed. If you are not using sglang, yo
u can ignore this warning.
2025-08-11 12:15:39.421 | INFO | __mp_main__:setup:21 - Setting up on device: cpu
2025-08-11 12:15:39.421 | INFO | __mp_main__:setup:33 - MINERU_VIRTUAL_VRAM_SIZE: 1
ERROR:root:Failed to connect to Hugging Face at https://huggingface.co/models: HTTPConnectionPool(host='huggingface.co', port=443): Max
retries exceeded with url: /models (Caused by ConnectTimeoutError(<urllib3.connection.HTTPSConnection object at 0x7feal62adc0>, 'Conne
ction to huggingface.co timed out. (connect timeout=3)'))
INFO:root:Falling back to 'modelscope' as model source.
2025-08-11 12:15:42.429 | INFO | __mp_main__:setup:37 - MINERU_MODEL_SOURCE: modelscope
Swagger UI is available at http://0.0.0.0:8899/docs
INFO: Started server process [2742]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:33674 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:33674 - "GET /favicon.ico HTTP/1.1" 404 Not Found

```

使用AutoDL平台的代理

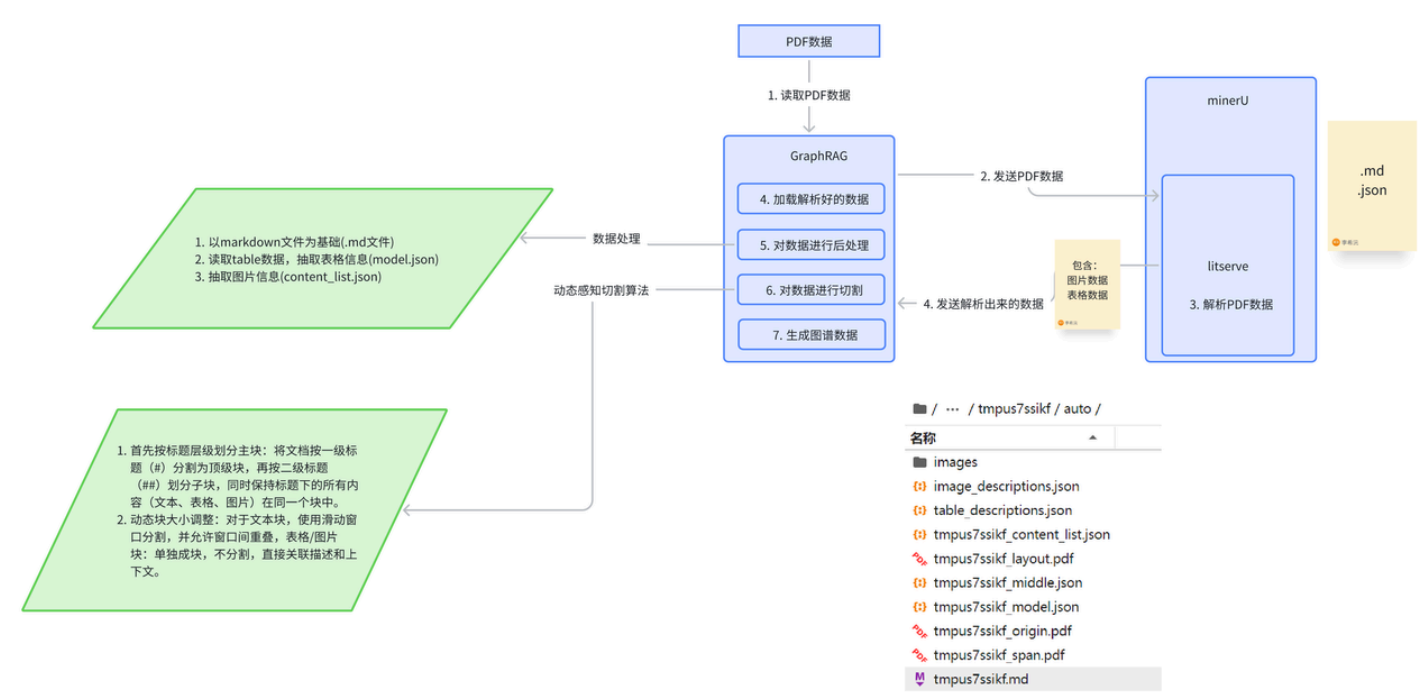


如果浏览器网页中出现 `litserver running` 的提示，则说明 `MinerU` 的 `RESTful` API 接口服务已经正常启动，并监听在 `http://ip:port` 端口上，否则需要检查 `server.py` 文件中的配置是否正确，以及网络连接是否正常。

准备好了 `MinerU` 的 `RESTful` API 接口服务后，接下来的工作就是在 `Microsoft GraphRAG` 项目中进行集成。

2.4.4 GraphRAG和minerU整合

2.4.4.1 PDF多模态数据处理方案整体架构设计



2.4.4.2 GraphRAG加载数据策略设计与实现

GprahRAG项目加载数据

The screenshot shows the GraphRAG project structure in a code editor. The project is named 'graphrag' and is located in the 'index' directory. The 'run_pipeline.py' file is highlighted, showing the following code:

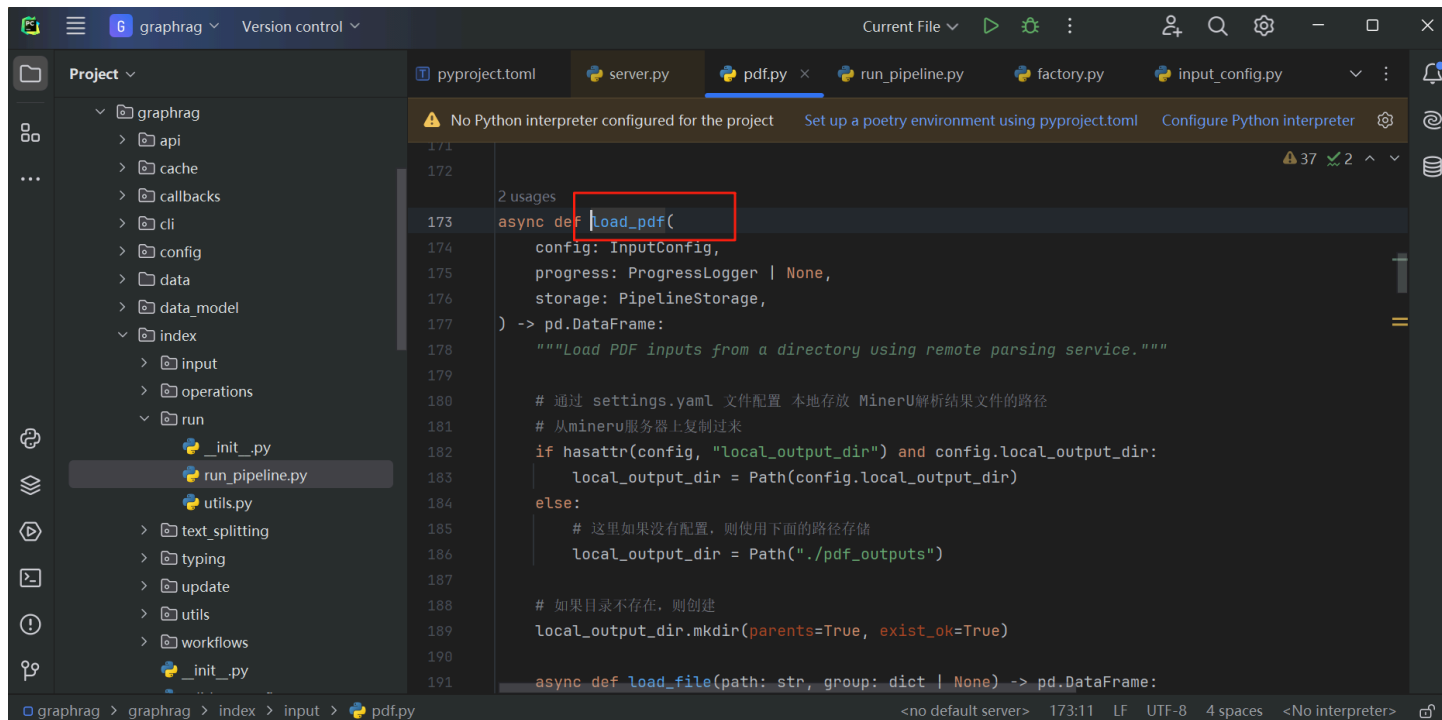
```
38 async def run_pipeline(  
43     is_update_run: bool = False,  
44 ) -> AsyncIterable[PipelineRunResult]:  
45     """Run all workflows using a simplified pipeline."""  
46     root_dir = config.root_dir  
47  
48     storage = create_storage_from_config(config.output)  
49     cache = create_cache_from_config(config.cache, root_dir)  
50  
51     # 关键点: 获取输入数据集  
52     dataset = await create_input(config.input, logger, root_dir)  
53  
54     if is_update_run:  
55         logger.info("Running incremental indexing.")  
56  
57         delta_dataset = await get_delta_docs(dataset, storage)  
58  
59         # warn on empty delta dataset  
60         if delta_dataset.new_inputs.empty:  
61             warning_msg = "Incremental indexing found no new documents, exiting."  
            raise ValueError(warning_msg)
```

This screenshot shows the VS Code editor with the 'graphrag' project open. The file explorer on the left shows the project structure, with 'run_pipeline.py' selected under the 'run' directory. The main editor window displays the 'create_input' function. A red box highlights the line `result = await loader(config, progress, storage)`, with a comment above it: `# 这就是加载器, 不同的输入类型, 加载器不同` (This is the loader, different input types, different loaders).

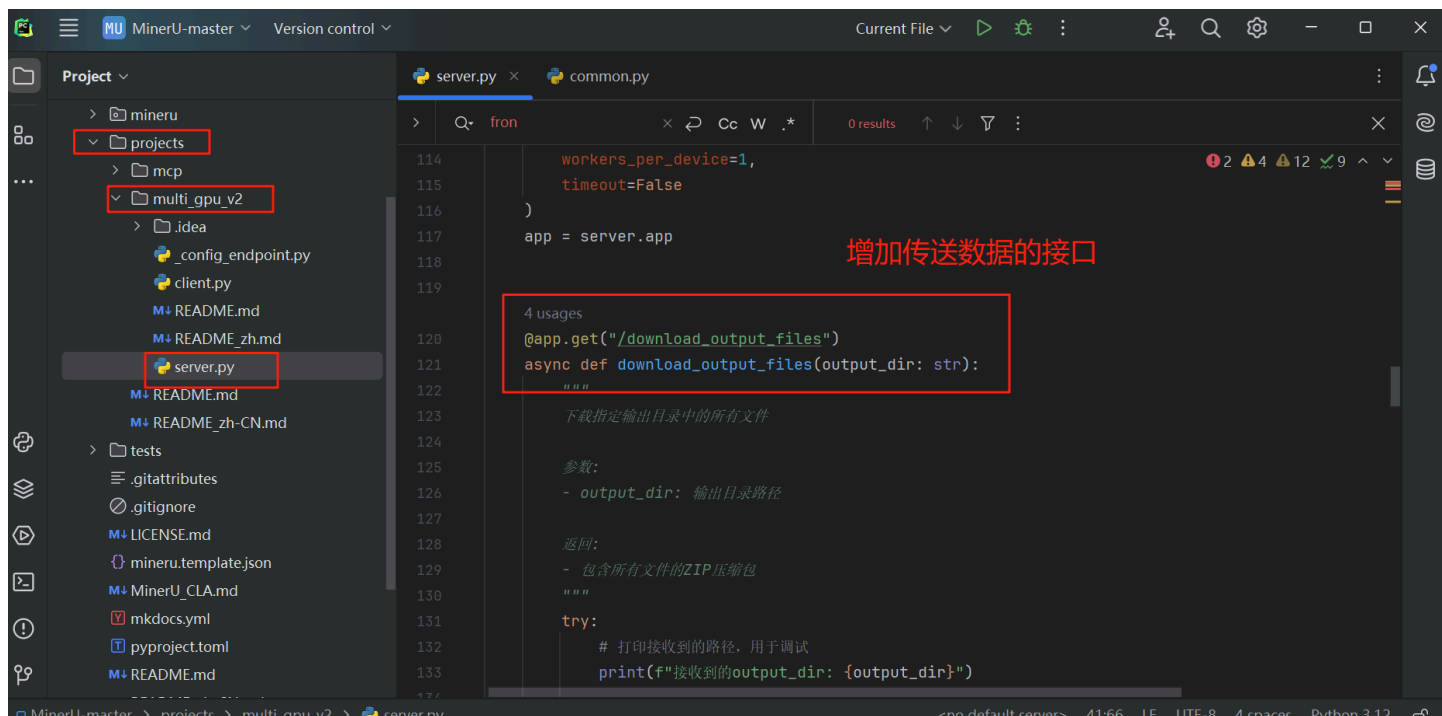
```
33 async def create_input(  
69     root_dir=Path(root_dir) / (config.base_dir or "")  
70 )  
71  
72     # 关键点: 不同的输入加载器  
73     if config.file_type in loaders:  
74         progress = progress_reporter.child(  
75             prefix=f"Loading Input ({config.file_type})", transient=False  
76         )  
77         loader = loaders[config.file_type]  
78  
79         # 这就是加载器, 不同的输入类型, 加载器不同  
80         result = await loader(config, progress, storage)  
81  
82         # Convert metadata columns to strings and collapse them into a JSON object  
83         if config.metadata:  
84             if all(col in result.columns for col in config.metadata):  
85                 # Collapse the metadata columns into a single JSON object column  
86                 result["metadata"] = result[config.metadata].apply(  
87                     lambda row: row.to_dict(), axis=1  
88                 )
```

This screenshot shows the same VS Code editor with 'graphrag' project. The file explorer is the same. The main editor window displays the 'loaders' dictionary definition. A red box highlights the entry `InputFileType.pdf: load_pdf,` in the dictionary.

```
2 # Licensed under the MIT License  
3  
4 """A module containing create_input method definition."""  
5  
6 > import ...  
23  
24 log = logging.getLogger(__name__)  
25 loaders: dict[str, Callable[..., Awaitable[pd.DataFrame]]] = {  
26     InputFileType.text: load_text,  
27     InputFileType.csv: load_csv,  
28     InputFileType.json: load_json,  
29     InputFileType.pdf: load_pdf,  
30 }  
31  
32 19 usages  
33 async def create_input(  
34     config: InputConfig,  
35     progress_reporter: ProgressLogger | None = None,  
36     root_dir: Path | None = None
```



minerU项目传送数据



代码块

```
1 @app.get("/download_output_files")
2 async def download_output_files(output_dir: str):
3     """
4     下载指定输出目录中的所有文件
5
6     参数:
7     - output_dir: 输出目录路径
8
```

```
9     返回:
10     - 包含所有文件的ZIP压缩包
11     """
12     try:
13         # 打印接收到的路径, 用于调试
14         print(f"接收到的output_dir: {output_dir}")
15
16         # 确保路径格式正确
17         output_dir = output_dir.rstrip('/')
18
19         # 关键修复: 处理路径中的auto子目录
20         # 如果路径不存在但路径中包含/auto, 尝试移除它
21         dir_path = Path(output_dir)
22         if not dir_path.exists() and '/auto' in output_dir:
23             output_dir = output_dir.replace('/auto', '')
24             dir_path = Path(output_dir)
25             print(f"尝试移除/auto后的路径: {dir_path}")
26
27         # 如果路径仍然不存在, 尝试添加/auto
28         if not dir_path.exists() and '/auto' not in output_dir:
29             output_dir = f"{output_dir}/auto"
30             dir_path = Path(output_dir)
31             print(f"尝试添加/auto后的路径: {dir_path}")
32
33         # 列出父目录内容, 帮助调试
34         parent_dir = str(Path(output_dir).parent)
35         print(f"父目录: {parent_dir}")
36         if os.path.exists(parent_dir):
37             print(f"父目录内容: {os.listdir(parent_dir)}")
38         else:
39             print(f"父目录不存在: {parent_dir}")
40
41         print(
42             f"转换后的路径: {dir_path}, 存在: {dir_path.exists()}, 是目录:
{dir_path.is_dir() if dir_path.exists() else False}")
43
44         # 直接检查目录是否存在, 如果不存在, 尝试在父目录或其他可能的位置查找
45         if not dir_path.exists() or not dir_path.is_dir():
46             # 尝试在父目录中查找
47             base_tmp_dir = parent_dir
48
49             # 如果父目录不存在, 尝试使用MinerUAPI的output_dir
50             if not os.path.exists(base_tmp_dir):
51                 base_tmp_dir = str(server.api.output_dir)
52                 print(f"使用API的output_dir: {base_tmp_dir}")
53
54             if os.path.exists(base_tmp_dir):
```

```

55         # 获取目录名的关键部分
56         basename = os.path.basename(output_dir)
57         # 移除可能的auto后缀
58         if basename == "auto" and output_dir != f"{base_tmp_dir}/auto":
59             basename = os.path.basename(os.path.dirname(output_dir))
60
61         # 直接列出base_tmp_dir中的所有目录
62         tmp_contents = os.listdir(base_tmp_dir)
63         print(f"{base_tmp_dir}目录内容: {tmp_contents}")
64
65         # 尝试精确匹配
66         if basename in tmp_contents:
67             dir_path = Path(os.path.join(base_tmp_dir, basename))
68             print(f"找到精确匹配的目录: {dir_path}")
69         else:
70             # 尝试部分匹配
71             parts = basename.split('_')
72             if len(parts) > 0:
73                 # 尝试匹配时间戳部分
74                 timestamp_part = parts[0]
75                 matching_dirs = [d for d in tmp_contents if
timestamp_part in d]
76                 if matching_dirs:
77                     dir_path = Path(os.path.join(base_tmp_dir,
matching_dirs[0]))
78                     print(f"找到匹配时间戳的目录: {dir_path}")
79                 elif len(parts) > 1:
80                     # 尝试匹配UUID部分
81                     uuid_part = parts[1].split('/')[0]
82                     matching_dirs = [d for d in tmp_contents if
uuid_part in d]
83                     if matching_dirs:
84                         dir_path = Path(os.path.join(base_tmp_dir,
matching_dirs[0]))
85                         print(f"找到匹配UUID的目录: {dir_path}")
86                     else:
87                         # 最后尝试: 按修改时间排序, 取最新的
88                         tmp_dirs = [os.path.join(base_tmp_dir, d) for
d in tmp_contents if
89 os.path.isdir(os.path.join(base_tmp_dir, d))]
90                         if tmp_dirs:
91                             tmp_dirs.sort(key=lambda x:
os.path.getmtime(x), reverse=True)
92                             dir_path = Path(tmp_dirs[0])
93                             print(f"使用最新的目录: {dir_path}")
94                     else:

```

```
95         # 如果在当前目录找不到, 尝试在MinerUAPI的
output_dir中查找
96         if base_tmp_dir !=
str(server.api.output_dir):
97             print(
98                 f"在{base_tmp_dir}中找不到匹配的目录,
尝试在{server.api.output_dir}中查找")
99             return await download_output_files(f"
{server.api.output_dir}/{basename}")
100         else:
101             raise HTTPException(status_code=404,
102                                 detail=f"找不到匹配的
目录: {output_dir}")
103         else:
104             # 如果无法解析目录名, 尝试在MinerUAPI的output_dir中查找
105             if base_tmp_dir != str(server.api.output_dir):
106                 print(f"无法解析目录名, 尝试在{server.api.output_dir}
中查找")
107                 return await download_output_files(f"
{server.api.output_dir}/{basename}")
108             else:
109                 raise HTTPException(status_code=404, detail=f"无法解
析目录名: {output_dir}")
110         else:
111             # 如果基础目录不存在, 尝试在MinerUAPI的output_dir中查找
112             if base_tmp_dir != str(server.api.output_dir):
113                 print(f"基础目录{base_tmp_dir}不存在, 尝试在
{server.api.output_dir}中查找")
114                 return await download_output_files(f"
{server.api.output_dir}/{os.path.basename(output_dir)}")
115             else:
116                 raise HTTPException(status_code=404, detail=f"基础目录不存
在: {base_tmp_dir}")
117
118         # 再次检查目录是否存在
119         if not dir_path.exists() or not dir_path.is_dir():
120             # 最后尝试: 如果所有尝试都失败, 检查是否可以直接使用MinerUAPI的output_dir
121             if str(dir_path) != str(server.api.output_dir):
122                 print(f"找不到有效的目录{dir_path}, 尝试使用
{server.api.output_dir}")
123                 return await download_output_files(str(server.api.output_dir))
124             else:
125                 raise HTTPException(status_code=404, detail=f"找不到有效的目录:
{output_dir}")
126
127         # 创建临时文件用于存储ZIP
128         temp_file = tempfile.NamedTemporaryFile(delete=False, suffix='.zip')
```

```

129         temp_file.close()
130
131         print(f"将打包目录: {dir_path}")
132
133         # 创建ZIP文件
134         with zipfile.ZipFile(temp_file.name, 'w', zipfile.ZIP_DEFLATED) as
zipf:
135             # 遍历目录中的所有文件和子目录
136             file_count = 0
137             dir_count = 0
138             for root, dirs, files in os.walk(dir_path):
139                 print(f"遍历目录: {root}, 包含文件数: {len(files)}, 子目录数:
{len(dirs)}")
140
141                 # 添加空目录
142                 for dir_name in dirs:
143                     dir_path_full = os.path.join(root, dir_name)
144                     # 计算目录在ZIP中的相对路径
145                     dir_arcname = os.path.relpath(dir_path_full, dir_path) +
'/'
146
147                     # 创建目录条目
148                     zipinfo = zipfile.ZipInfo(dir_arcname)
149                     zipinfo.external_attr = 0o755 << 16 # 设置目录权限
150                     zipf.writestr(zipinfo, '')
151                     dir_count += 1
152                     print(f"添加目录: {dir_arcname}")
153
154                 for file in files:
155                     file_path = os.path.join(root, file)
156                     # 计算文件在ZIP中的相对路径
157                     arcname = os.path.relpath(file_path, dir_path)
158                     # 添加文件到ZIP
159                     zipf.write(file_path, arcname)
160                     file_count += 1
161
162             print(f"总共添加了 {file_count} 个文件和 {dir_count} 个目录到ZIP")
163
164             # 返回ZIP文件供下载
165             zip_size = os.path.getsize(temp_file.name)
166             print(f"生成的ZIP文件大小: {zip_size} 字节")
167             return FileResponse(
168                 path=temp_file.name,
169                 filename=f"{os.path.basename(output_dir)}.zip",
170                 media_type="application/zip",
171                 background=None # 在响应发送后删除临时文件
172             )
except HTTPException:

```

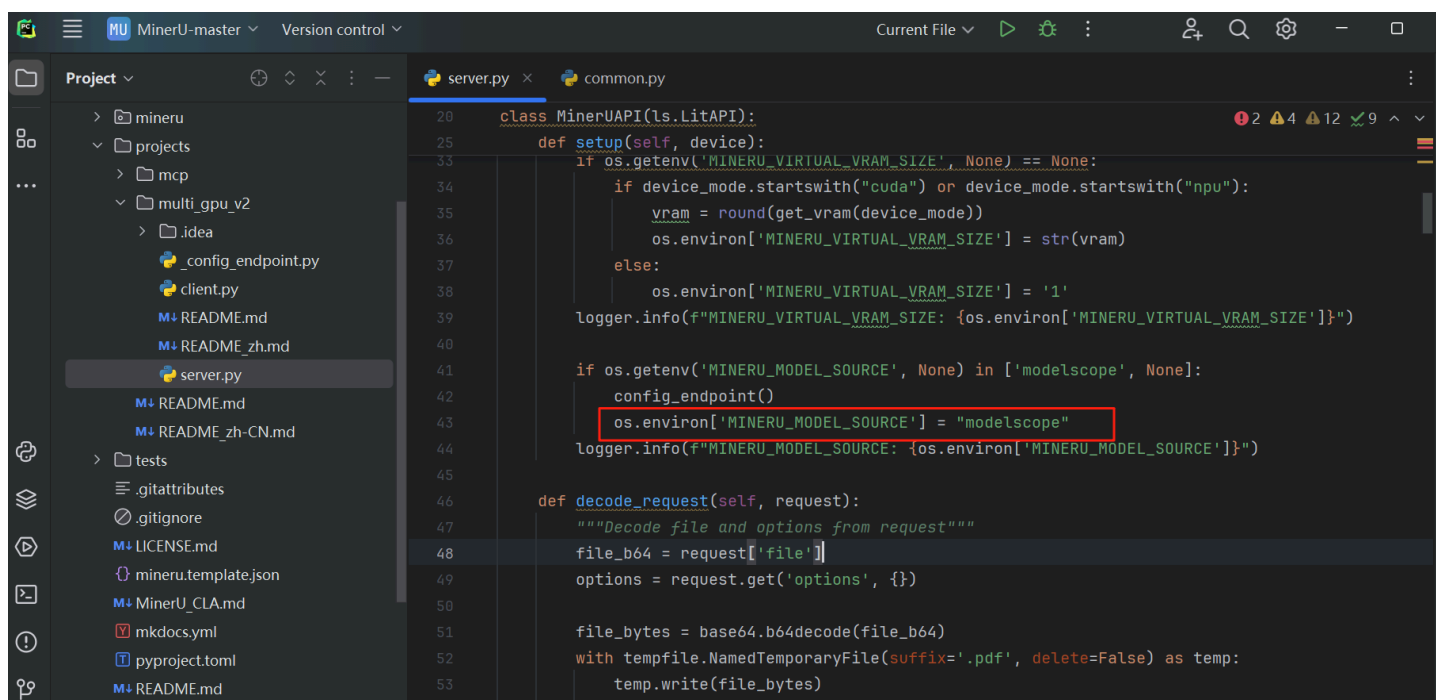
```

173         raise
174     except Exception as e:
175         print(f"下载文件失败, 错误: {str(e)}")
176         import traceback
177         traceback.print_exc()
178         raise HTTPException(status_code=500, detail=f"下载文件失败: {str(e)}")

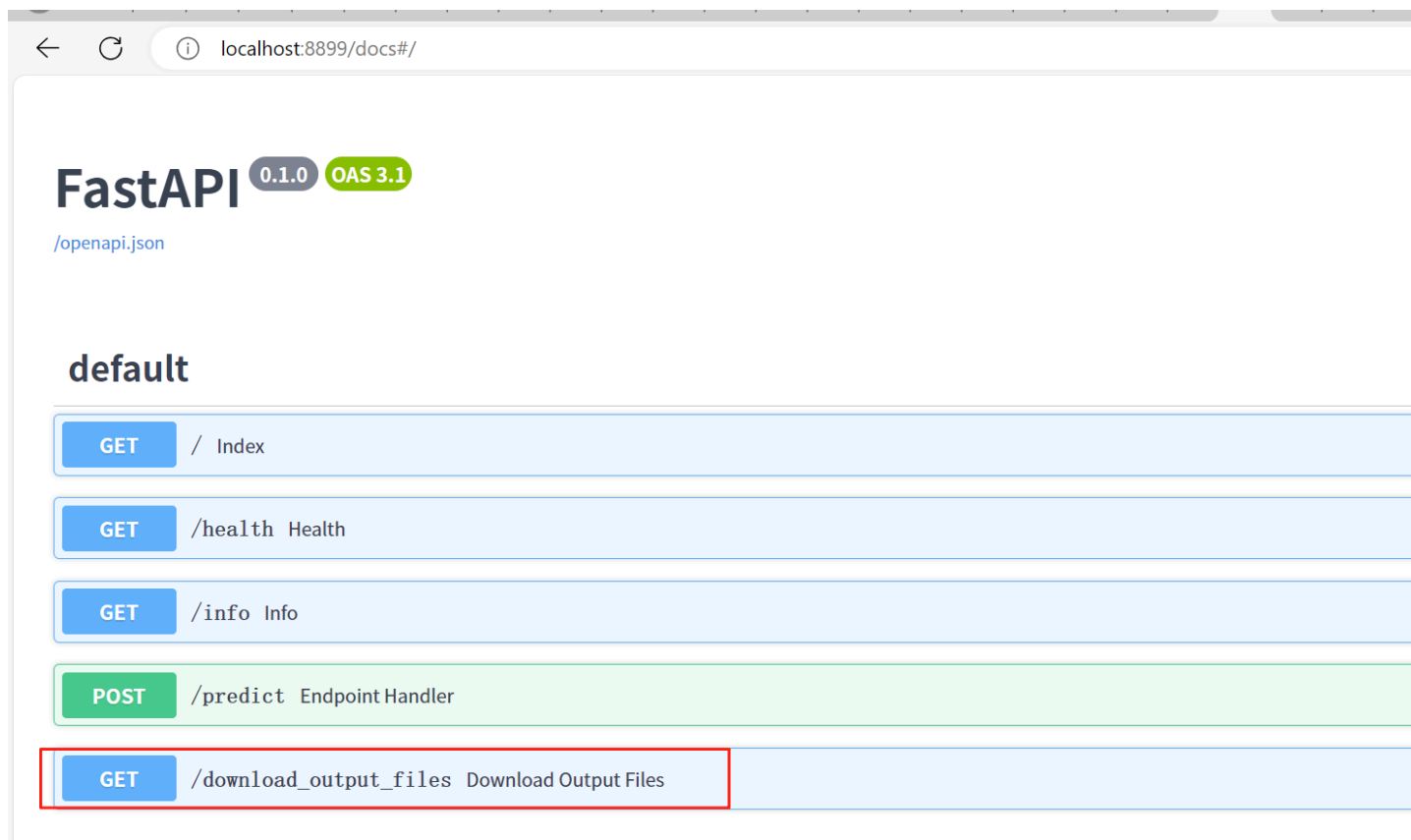
```

加入的新的代码以后，执行：python server.py 之前，有两个步骤需要操作：

1. 安装依赖：pip install --upgrade PyMuPDF
2. 把默认加载模型的网站改为modelscope



运行：python server.py



完整的数据后处理的思路如下：

1. 调用 MinerU 的 `do_parse` RestFul API 接口，将本地的文件上传 PDF 依次传到 MinerU 服务上进行解析；
2. 调用 MinerU 的 `download_output_files` RestFul API 接口，将解析后的所有文件拉取到项目代码的运行环境（一般企业场景需要搭配专业的文件服务器）；
3. 读取 `xxx.md` 文件，获取 PDF 文件的文本内容，这里面包含：纯文本、图片、表格、公式等；
4. 在进行文本分块之前，先进行语义增强和文本优化，主要包括以下几点：
 - o 对于表格数据：
 - i. 首先提取结构化数据，从 `model.json` 文件中，根据 `category_id` 是否等于 5，提取出都有哪些表格；
 - ii. 调用大模型接口，对表格进行总结和摘要，作为额外描述添加到元数据中；
 - o 对于图片数据：
 - i. 从 `content_list.json` 文件中，根据 `type` 找到图片，并提取其前后文，作为图片的背景信息（因为按照一般的逻辑，当出现图片的时候，前文或者后文会对图片有相关的描述或解释性文字）；
 - ii. 从本地对应的 PDF 解析文件中找到 `Images` 文件夹，获取到所有的图片路径；
 - iii. 调用视觉模型对图片生成详细的描述，包括图片中的内容、对象、场景、布局等关键信息，作为额外描述添加的元数据中；

5. 将表格、图片的元数据以注释形式插入到第3步读取的 `Markdown` 文本中，组成最终要用于后续做 `text_units` 切分的原始文本，形式为：

代码块

```
1      <!-- METADATA
2      type: table
3      description: "下表展示了2023年各季度销售额对比..."
4      source_page: 5
5      parent_headings: ["## 实验结果"]
6      -->
7      该表格统计了2023年各季度的销售额（见下表）：
8      | 季度 | 销售额（万元） |
9      |-----|-----|
10     | Q1   | 120              |
```

2.4.4.3 GraphRAG切割数据策略设计与实现

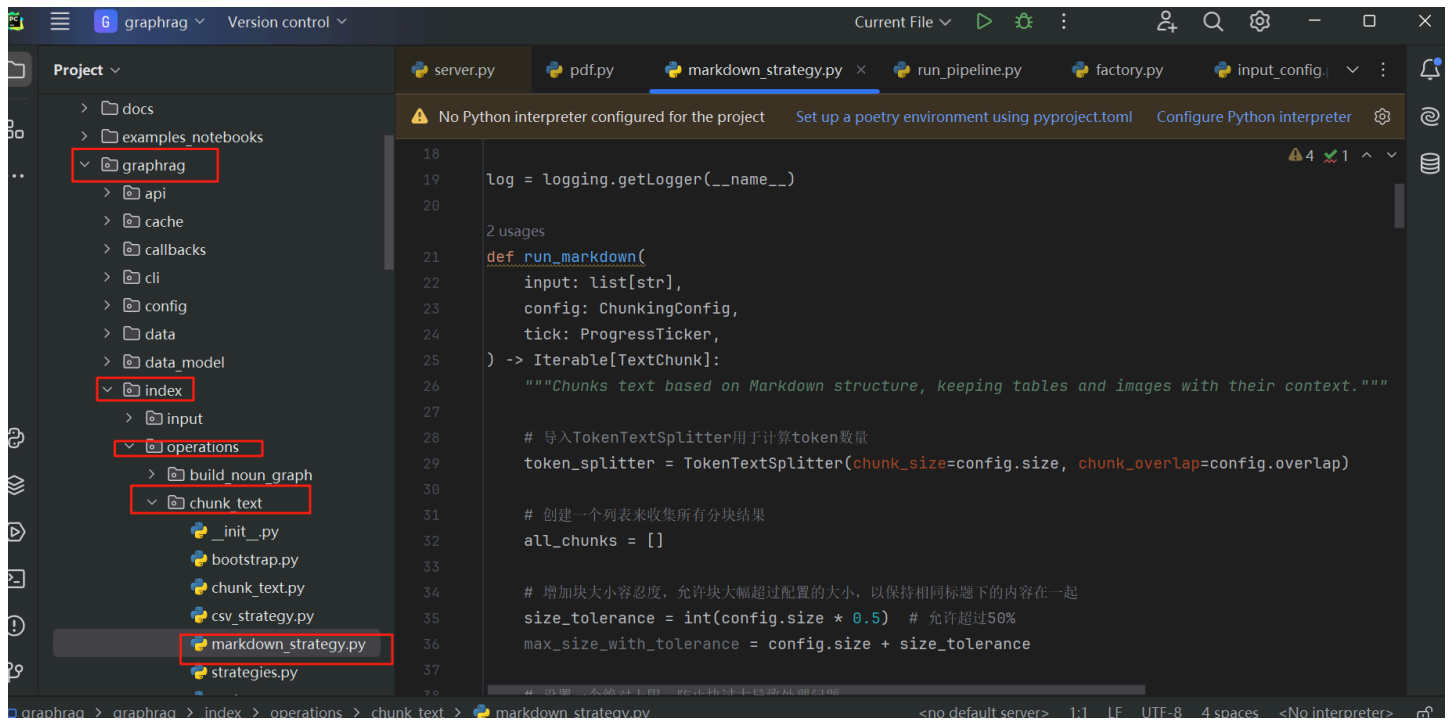
`PDF` 包含文本、图像和表格，所以在分块时也常被称多模式分块，涉及处理单个文档中的多种类型的数据（例如文本、图像和表格）。比如有一些常见的切分方法：

- 基本策略：将文档的连续部分组合起来以填充每个块，确保它们不超过指定的大小。其中表格被单独处理，如果太大，进行拆分。
- 按标题策略：保留章节边界，每当新章节或页面开始时，就从新区块开始。合并较小的章节，以避免区块过小。
- 按页面策略：这种方法可确保来自不同页面的内容不会出现在同一个块中。每个新页面都会开始一个新的块。
- 按相似度策略：使用大模型来识别和分组相似内容。所需的相似度级别可以调整。

不同的切分方法其实都涉及两个主要问题：如果处理表格，如何处理图片。传统 `RAG` 常用固定大小的分块，比如 `512` 个 `token`，但可能破坏表格结构，其次 `GraphRAG` 需要更语义化的分块，比如按章节或段落。同时，是否需要重叠块来保持上下文联系，最后，表格的处理是关键，因为表格的结构信息一旦破坏，模型可能无法正确理解。图片的话，转成 `Markdown` 后可能只是图片链接，需要 `OCR` 提取文字或添加描述。

而上面三个核心的问题，我们在 `PDF` 文档加载器中已经全部解决，因此，这里我们实现的是核心策略：结构感知分块（Structure-aware Chunking），具体来看：

- 首先按标题层级划分主块：将文档按一级标题（#）分割为顶级块，再按二级标题（##）划分子块，同时保持标题下的所有内容（文本、表格、图片）在同一个块中。
- 动态块大小调整：对于文本块，使用滑动窗口分割，并允许窗口间重叠，表格/图片块：单独成块，不分割，直接关联描述和上下文。



2.4.4.4 总体方案代码联调

步骤1: minerU里启动server.py服务

代码块

```
1 python server.py
```

步骤2: GraphRAG里执行解析任务

代码块

```
1 ## 在项目的根目录下执行
2 poetry run poe index --root ./data
```

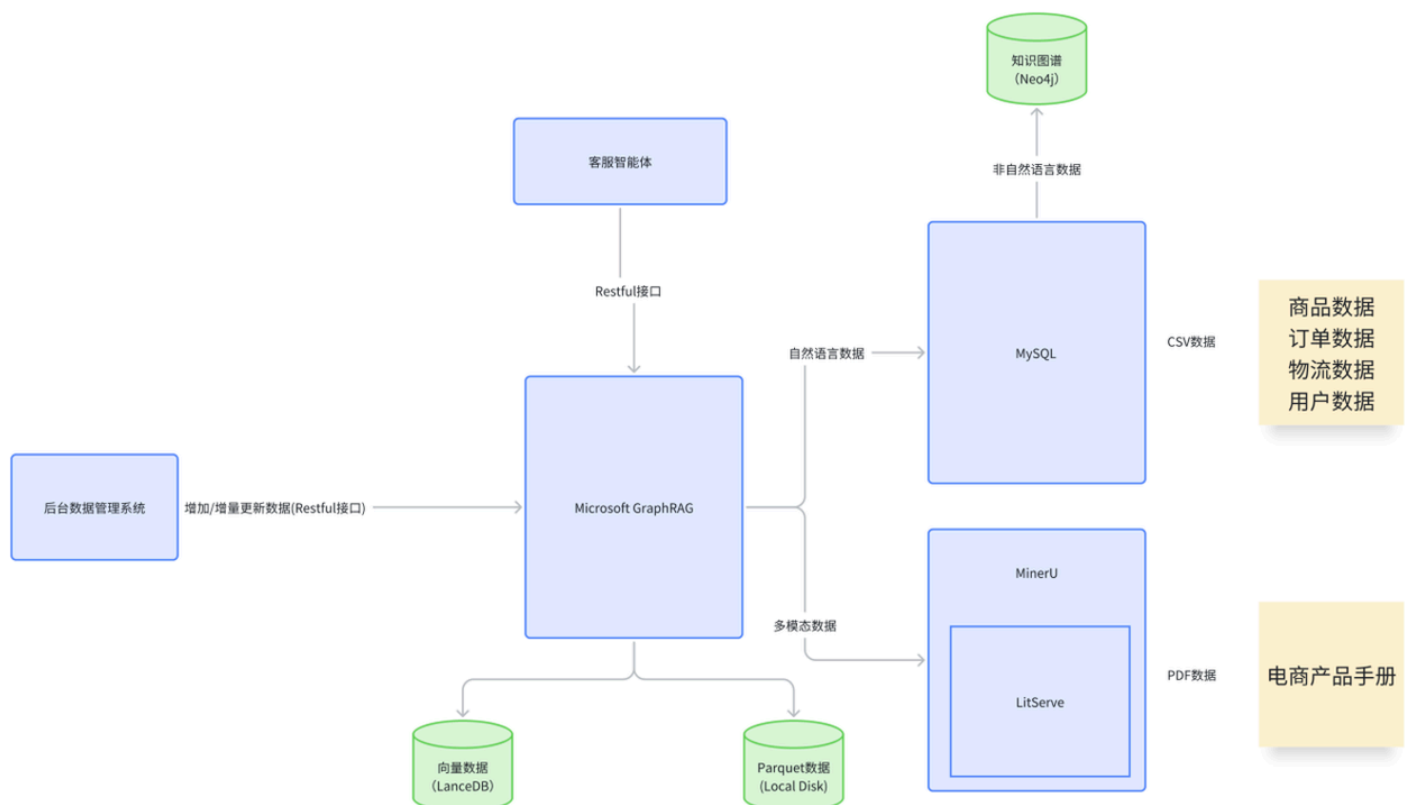
```

11 50cc3886508c84c624c854636cd14827a5e61138b46cc2... [0.041006166487932205, 0.008371501229703426, 0...
12 11c0cce383c73b3386a3fa474e3610265fc71b30f2278b... [-0.017882397398352623, -0.005298241972923279, ...
13 18d8eedfb658b8dd370851c39dc05a3abe13074540090f... [0.023006577044725418, 0.002436253475025296, 0...
14 46bb54b7b1eede337084056da36d511ede5a40d56c2cc... [0.029996396973729134, -0.0005542676080949605, ...
15 85a6365a6609c69e60582fbbdb894d6629dcd985bbef02... [0.04065461829304695, 0.02797679789364338, 0.0...
16 c018b5891b16a9fb90f589ae59115316723e2092f79b48... [0.0181626845151186, -0.0004242272407282144, 0...
17 58f5ee77b2acla63f16a7f9e7b6198e9b1e4576cff3413... [-0.013141832314431667, 0.007541987579315901, ...
18 0a3fed349334a5c8e15675b15d37cd9ee16a833c2dce03... [0.025558263063430786, -0.03763441741466522, 0...
19 3c89d3107140a7e0700e0d695116b6b52d023d40c2c152... [0.025619933381676674, 0.011341736651957035, 0...
20 239e9c619e239bee630601161f1252fbed62f260a6ae3c... [0.026366453617811203, 0.0019100334029644728, ...
21 b595ba0847d44babf8f2a5f239f2656685a388e4cf2350... [0.02268451824784279, 0.006478500552475452, 0...
'community.full_content': id
embedding
0 a2672622c22f4c6489effa4751736287 [0.01240979041904211, -0.0008442374528385699, ...
1 8731d96c6a414438b3ab16d75f72fa7a [-0.011999815702438354, -0.01517688855288315, ...
2 dd05ee4dc6e24e0d8c1leeabd97a35b0 [0.01086348481476307, -0.026265490800142288, 0...
3 1d088eb09afe46bddd277f88062ca9f [0.05357172340154648, 0.014841456897556782, 0...
4 3d9e887068a0405f99931fd680a00786 [-0.0008011786267161369, -0.006116761360317469...
5 25994744871240bdb18584e360e2c231 [0.012270873412489891, -0.04365288093686104, -...
6 0e9960b644da439eb0644bbd0a4565b2 [0.01175000425428152, -0.012989134527742863, 0...
7 01b405de47004fe584ca0830c1420c02 [-0.010054233483970165, -0.014710752293467522, ...
8 9b51f1bb965348e8a1bdd6db284a1f69 [0.011496291495859623, -0.014079354703426361, ...]
.: GraphRAG Indexer
.: GraphRAG Indexer
| Loading Input (InputFileType.pdf) - 2 files loaded (1 filtered) 100% 0:00:00 0:0
0:00
| create_base_text_units 100% 0:00:00 0:00:00
| create_final_documents 100% 0:00:00 0:00:00
| extract_graph 100% 0:00:00 0:00:00
| finalize_graph 100% 0:00:00 0:00:00
| create_communities 100% 0:00:00 0:00:00
| create_final_text_units 100% 0:00:00 0:00:00
| create_community_reports 100% 0:00:00 0:00:00
| generate_text_embeddings 100% 0:00:00 0:00:00
All workflows completed successfully.
(/root/autodl-tmp/data/graphrag) root@autodl-container-997946b97e-a0631a7e:~/autodl-tmp/kefu/graphrag#

```

2.5 企业级智能客服GraphRAG工程化方案落地

2.5.1 企业智能客服数据总体架构设计



2.5.2 GraphRAG工程化设计与实现

2.5.2.1 GraphRAG源码已有功能梳理

到目前为止，我们一共对GraphRAG做了如下的优化改造：

- 1. 对 `.csv` 文件的索引流程做了优化，不再使用简单的按照 `token` 数量切分，而是结合 `csv` 文件的预处理流程，构建分层分级动态切分，从而保证每个 `row` 可以保留完整的语义；
- 2. 自定义集成了支持并行解析的 `MinerU` 服务，支持对 `PDF` 格式、图像（`.jpg` 及 `.png`）、`Word`（`.doc`及`.docx`）、以及 `PowerPoint`（`.ppt`及`.pptx`）在内的多种文档格式的文档解析；
- 3. 支持对图像、表格类型数据的分析和处理，接入多模态（视觉）模型对图片和表格进行分析，并生成对应的文本描述，从而保证 `GraphRAG` 可以处理更多的数据类型；
- 4. 新增 `PDF` 等格式文件的文档切分器，通过结构感知动态分块策略，有效保留文本和表格/图谱多模式语义组合长文本及图像、表格等单独语义的隔离；

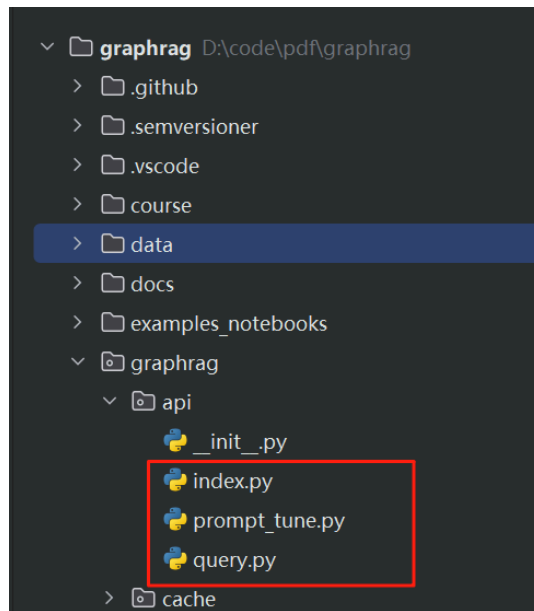
但是到目前，我们还是用的这两种方式提供的GraphRAG的服务：CLI 和 Poe

特性	CLI 启动方式	Poe 启动方式
安装方式	pip 安装	源码安装
启动方式	在控制台输入命令启动	在控制台输入命令启动
可用参数	根据 <code>--help</code> 提供的参数指定构建索引/检索流程	可以任意添加参数，满足定制化的需求
灵活度	灵活度低，无法满足一些特殊的需求	灵活度高，可以修改源码，任意添加参数

很明显，这样没办法跟我们的其它服务或者前端打通，所以接下来我们需要把GraphRAG的核心服务器封装成restful的接口。

但是这个大家也别担心，我们不需要写很多的代码逻辑，因为GraphRAG也考虑到了这个问题，所以提供好了接口，我们只需要基于这些接口再做简单的封装即可。

GraphRAG提供好的接口的源码位置是在这个地方：



文件	功能
prompt_tunne.py	1. 动态提示词模板生成
index.py	1. 构建索引 2. 增量更新索引
query.py	1. 支持4种查询方式： basic, local, global, drift

对于我们来说，我们核心也是提供这样的四个接口能力：

1. 动态提示词模板生成的接口
2. 索引构建的接口
3. 增量索引更新的接口
4. 查询的接口

2.5.2.2 动态提示词模块接口设计

参数名	描述	状态	默认值
--config	配置文件的路径。这是加载数据和模型设置所必需的。	必需	无
--root	数据项目根目录，包括配置文件（YML、JSON 或 .env）。	可选	当前目录
--domain	与输入数据相关的域，例如“空间科学”、“微生物学”或“环境新闻”。如果留空，则将从输入数据中推断出域。	可选	无
--selection-method	选择文档的方法。选项包括 all、random、auto 或 top。	可选	random
--limit	使用随机或顶部选择时要加载的文本单元的限制。	可选	15
--language	用于输入处理的语言。如果它与输入的语言不同，则将进行翻译。默认值为“”，表示将自动从输入中检测。	可选	自动检测
--max-tokens	提示生成的最大令牌数。	可选	2000
--chunk-size	用于从输入文档生成文本单元的标记大小。	可选	200
--n-subset-max	使用自动选择方法时嵌入的文本块数量。	可选	300
--k	使用自动选择方法时要选择的文档数量。	可选	15
--min-examples-required	实体提取提示所需的最小示例数。	可选	2
--discover-entity-types	允许自动发现和提取实体。当数据涵盖大量主题或高度随机化时，建议使用此选项。	可选	无
--output	保存生成的提示的文件夹。	可选	prompts

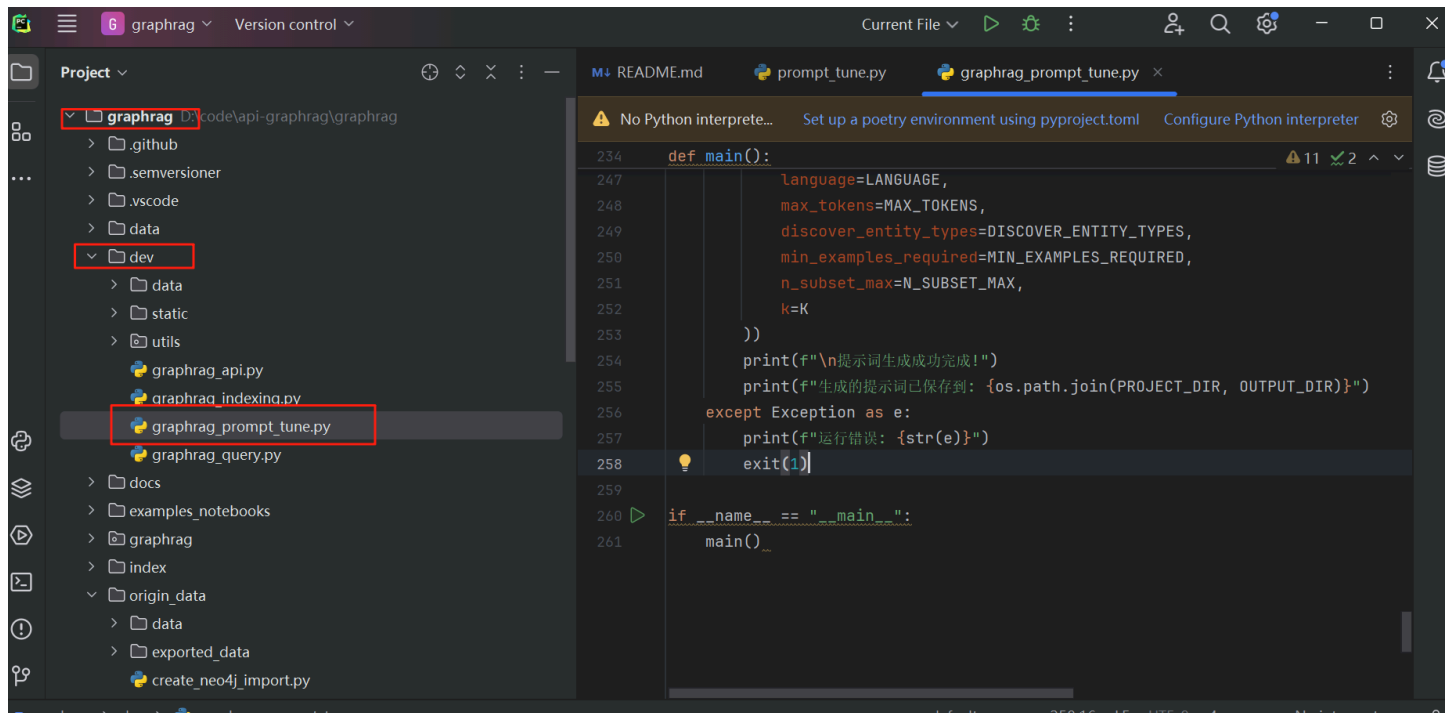
核心代码：

```

13
14 > import ...
49
50
51 4 usages
52 @validate_call(config={"arbitrary_types_allowed": True})
53 async def generate_indexing_prompts(
54     config: GraphRagConfig,
55     logger: ProgressLogger,
56     root: str,
57     chunk_size: PositiveInt = graphrag_config_defaults.chunks.size,
58     overlap: Annotated[
59         int, annotated_types.Gt(-1)
60     ] = graphrag_config_defaults.chunks.overlap,
61     limit: PositiveInt = 15,
62     selection_method: DocSelectionType = DocSelectionType.RANDOM,
63     domain: str | None = None,
64     language: str | None = None,
65     max_tokens: int = MAX_TOKEN_COUNT,
66     discover_entity_types: bool = True

```

其实核心接口源码里已经实现了，我们就是基于这个接口再完善一下即可。



核心的参数设置：

代码块

```
1  # GraphRAG项目根目录路径
2  PROJECT_DIR = "/root/autodl-tmp/kefu2/graphrag"
3
4  # 数据目录名称（相对于项目根目录）
5  DATA_DIR_NAME = "data"
6
7  # 文档块大小
8  CHUNK_SIZE = 500
9
10 # 文档块重叠大小
11 OVERLAP = 100
12
13 # 要加载的文档块数量限制
14 # 注意：如果文档较少，请设置较小的值
15 LIMIT = 5
16
17 # 文档选择方法: "random", "auto", "all", "top"
18 # 参考DocSelectionType枚举的有效值
19 SELECTION_METHOD = "random"
20
21 # 领域（为None则自动检测）
22 DOMAIN = None
23
24 # 语言（为None则自动检测）
25 LANGUAGE = None
26
27 # 提示词最大令牌数
```

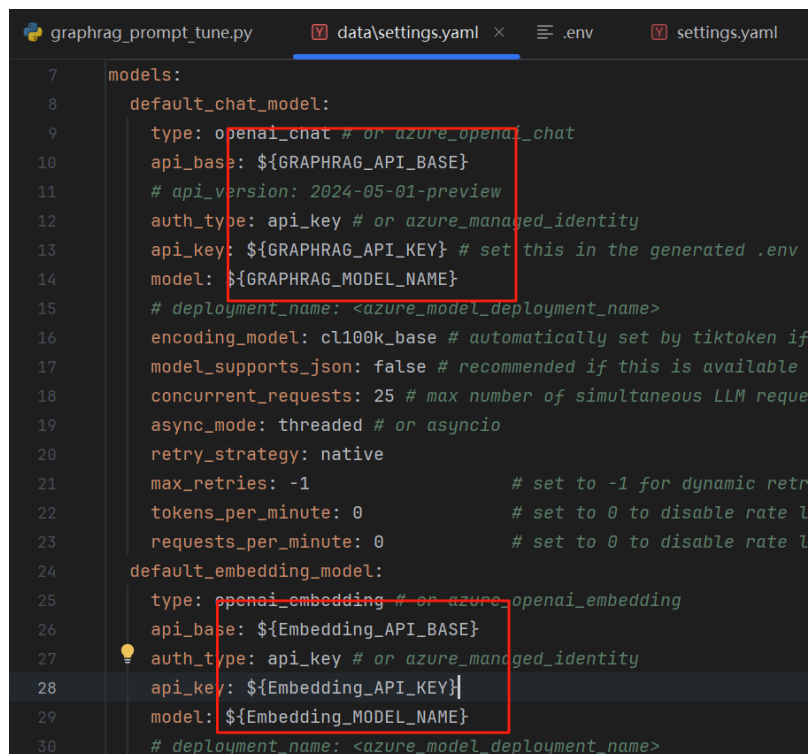
```

28 MAX_TOKENS = 4000
29
30 # 是否自动发现实体类型
31 DISCOVER_ENTITY_TYPES = True
32
33 # 最小所需的示例数
34 MIN_EXAMPLES_REQUIRED = 2
35
36 # 嵌入子集最大数量 (对auto选择方法有效)
37 N_SUBSET_MAX = 300
38
39 # 文档选择数量 (对auto选择方法有效)
40 K = 15
41
42 # 输出目录
43 OUTPUT_DIR = "/root/autodl-tmp/kefu2/graphrag/data/pdf_prompt_turn_output"

```

setting.yaml文件设置

1) 模型配置

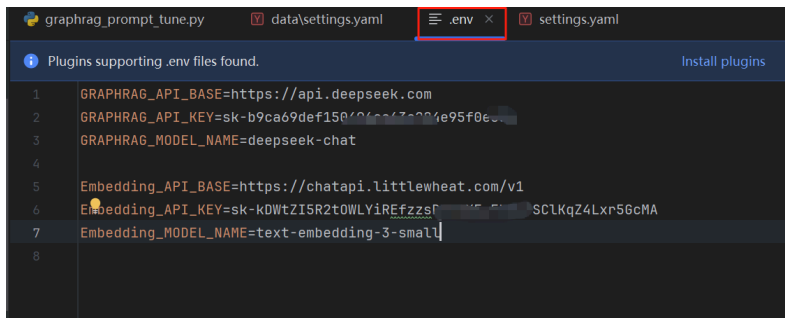


```

7 models:
8   default_chat_model:
9     type: openai_chat # or azure_openai_chat
10    api_base: ${GRAPHRAG_API_BASE}
11    # api_version: 2024-05-01-preview
12    auth_type: api_key # or azure_managed_identity
13    api_key: ${GRAPHRAG_API_KEY} # set this in the generated .env
14    model: ${GRAPHRAG_MODEL_NAME}
15    # deployment_name: <azure_model_deployment_name>
16    encoding_model: cl100k_base # automatically set by tiktoken if
17    model_supports_json: false # recommended if this is available
18    concurrent_requests: 25 # max number of simultaneous LLM requests
19    async_mode: threaded # or asyncio
20    retry_strategy: native
21    max_retries: -1 # set to -1 for dynamic retries
22    tokens_per_minute: 0 # set to 0 to disable rate limiting
23    requests_per_minute: 0 # set to 0 to disable rate limiting
24  default_embedding_model:
25    type: openai_embedding # or azure_openai_embedding
26    api_base: ${Embedding_API_BASE}
27    auth_type: api_key # or azure_managed_identity
28    api_key: ${Embedding_API_KEY}
29    model: ${Embedding_MODEL_NAME}
30    # deployment_name: <azure_model_deployment_name>

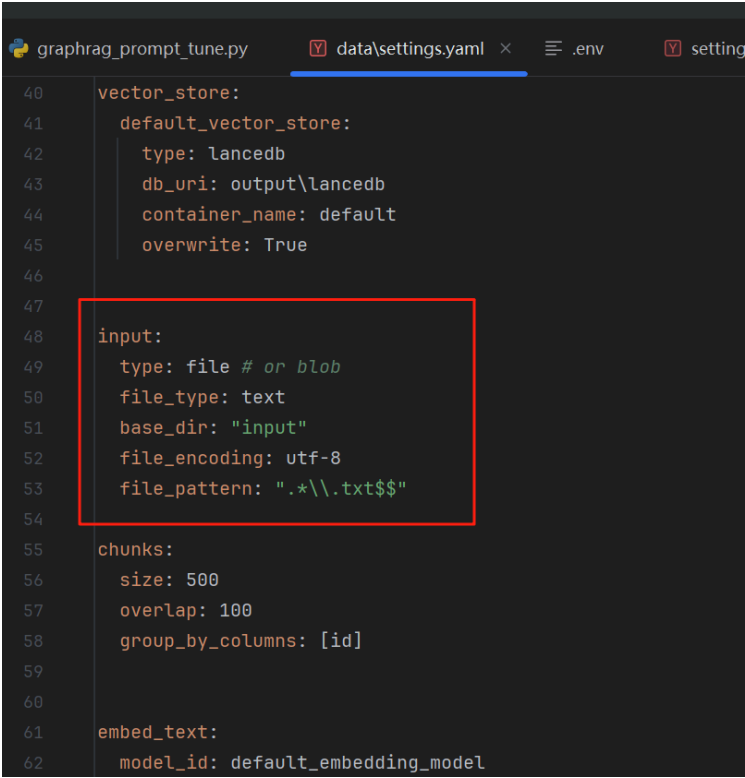
```

对应的.env文件



```
graphrag_prompt_tune.py data/settings.yaml .env x settings.yaml
Plugins supporting .env files found. Install plugins
1 GRAPHRA6_API_BASE=https://api.deepseek.com
2 GRAPHRA6_API_KEY=sk-b9ca69def150404c3a304e95f0e...
3 GRAPHRA6_MODEL_NAME=deepseek-chat
4
5 Embedding_API_BASE=https://chatapi.littleheat.com/v1
6 Embedding_API_KEY=sk-kDwtZISR2tOWLYiREfzzSF... SCLKqZ4LxrS6cMA
7 Embedding_MODEL_NAME=text-embedding-3-small
8
```

2) 读取文件配置



```
graphrag_prompt_tune.py data/settings.yaml x .env x setting
40 vector_store:
41   default_vector_store:
42     type: lancedb
43     db_uri: output\lancedb
44     container_name: default
45     overwrite: True
46
47   input:
48     type: file # or blob
49     file_type: text
50     base_dir: "input"
51     file_encoding: utf-8
52     file_pattern: ".*\\.txt$$"
53
54   chunks:
55     size: 500
56     overlap: 100
57     group_by_columns: [id]
58
59   embed_text:
60     model_id: default_embedding_model
61
62
```

数据准备：

代码块

```
1 # 小米、华为与阿里巴巴：中国科技巨头的崛起与全球化战略
2
3 在21世纪的中国商业史上，小米、华为和阿里巴巴无疑是三颗最耀眼的明星。它们分别代表了智能手机、通信设备和电子商务领域的中国力量，不仅在国内市场占据主导地位，更在全球范围内产生了深远影响。本文将深入剖析这三家科技巨头的发展历程、商业模式、全球化战略及未来挑战，展现中国科技企业的创新力量与全球竞争力。
4
5 ## 小米：从性价比之王到全球智能生态领导者
6
7 小米公司的创业故事堪称中国互联网时代的经典案例。2010年4月6日，北京中关村银谷大厦807室，雷军和几位联合创始人喝下一碗小米粥，一家名为"小米"的公司悄然成立。当时没人能想到，这碗"小米粥"会在未来十年搅动全球科技行业的格局。雷军曾是中国软件行业的传奇人物，金山软件的掌舵人，却在40岁这年选择"归零重启"，他坦言："我害怕被时代抛弃，害怕成为那个只会回忆过去的人。"这种危机感催生了小米的核心理念——"为发烧而生"。
8
```

商业模式革命：铁人三项战略

2011年8月16日，小米1发布，定价1999元，配置却堪比当时三四千元的旗舰机型。这不是简单的价格战，而是一场商业模式的革命。小米采用了"硬件+软件+互联网服务"的铁人三项模式：硬件贴近成本定价、MIUI系统通过用户反馈快速迭代、电商直销砍掉中间环节。这种模式让传统手机厂商措手不及，一位行业分析师回忆："当时很多厂商的高管都在拆解小米手机，试图理解为什么他们能卖这个价格。"

小米早期通过MIUI论坛与用户共创，这种深度互动塑造了产品，体现了"用户思维比技术更重要"的理念。2014年，小米售出6112万部智能手机，销售额达743亿元，超越三星成为中国市场第一。同年12月，小米获得11亿美元投资，估值达450亿美元。

生态链布局与国际化扩张

2014-2016年，小米遭遇"成长的烦恼"——供应链问题、产品同质化、线下渠道薄弱等问题集中爆发。雷军后来反思："我们跑得太快，有些基础没打牢。"关键时刻，小米做了几个关键决策：组建"特种部队"解决供应链问题、启动"新零售"战略三年开1000家小米之家、全面进军国际市场。

印度成为小米国际化的第一站。通过本土化运营和性价比策略，小米在印度市场占有率从2015年的3%飙升至2018年的28%，反超三星成为第一。同时，小米生态链可能是商业史上最成功的生态布局之一。从移动电源到空气净化器，从扫地机器人到电动滑板车，小米通过"投资+孵化"模式，培育了上百家生态链企业。这种"竹林效应"让小米在手机增长放缓时仍能保持品牌热度。

高端化转型与汽车业务突破

近年来，小米加大在高端智能手机、汽车和人工智能领域的投入。2025年8月，小米公布了固态电池技术专利，续航里程超过1200公里，10分钟快充可行驶800公里。这项技术采用固态电解质，能量密度达1000Wh/L以上，安全性大幅提升，即使在800度高温下也能保持稳定。同时，小米汽车YU7车型搭载了自研540°全景环视系统，结合360°全景影像与180°底盘透视功能，展现了小米在智能驾驶辅助技术方面的实力。

2018年，小米在香港上市，开启了新阶段的发展。如今的小米已从"性价比代表"向"高端科技品牌"转型，其愿景是"让全球每个人都能享受科技带来的美好生活"。正如雷军常说："永远相信美好的事情即将发生。"

华为：从通信设备商到全球科技领军者

华为的创业故事同样传奇。1987年，任正非在深圳创立华为，初始资本仅2.1万元。公司最初代理香港公司的PBX交换机，后来转向自主研发。1990年代，华为在农村市场积累经验，逐步攻克城市市场和国际市场。

通信设备领域的全球扩张

2000年左右，华为开始国际化，首先进入发展中国家市场，然后进军欧洲等发达市场。通过高性价比和本地化服务，华为逐渐成为全球通信设备市场的主要玩家。2012年，华为超越爱立信成为全球最大电信设备制造商。

在5G技术研发上，华为投入巨大。截至2020年，华为拥有全球最多的5G标准必要专利，成为5G技术的领导者。尽管面临国际政治压力，华为仍在全球获得了大量5G商用合同。

消费者业务的崛起与创新

2010年后，华为消费者业务迅速成长。Mate和P系列智能手机凭借卓越的拍照能力和长续航等特点，赢得了全球高端用户的青睐。华为还开发了自己的图像处理芯片和算法，与徕卡合作提升了手机摄影水平。

面对美国的制裁，华为加速推进自主创新。2019年，华为推出自研操作系统HarmonyOS（鸿蒙），旨在构建全场景智慧生态。2021年，HarmonyOS 2.0发布，装机量迅速突破亿级。华为还大力发展HMS（华为移动服务），以替代谷歌的GMS。

云计算与企业业务的发展

华为云近年来增长迅速，已成为中国第二大、全球前五的云服务提供商。华为在企业业务领域提供数字化转型解决方案，服务政府、金融、交通等多个行业。华为还大力发展人工智能芯片和计算框架，推出昇腾系列AI处理器。

任正非提出的"狼性文化"和"长期投入研发"的战略，使华为在全球科技竞争中保持韧性。华为每年将10%-15%的营收投入研发，在全球设立多个研究所，吸引顶尖人才。

阿里巴巴：构建数字经济生态系统

阿里巴巴的故事始于1999年马云在杭州的公寓里创办的Alibaba.com。最初的使命是"让天下没有难做的生意"，服务中小企业进行国际贸易。2003年，阿里巴巴推出淘宝网，开创了中国C2C电商的新纪元。

电商帝国的建立与创新

2004年，支付宝从淘宝分拆独立，解决了网络交易的信任问题。2008年，淘宝商城（后改名天猫）上线，开拓B2C市场。阿里巴巴不断创新电商模式，推出了"双11"购物节，创造了全球网购奇迹。

2014年，阿里巴巴在纽交所上市，融资250亿美元，成为当时全球最大IPO。阿里逐渐形成了以电商为核心，涵盖支付、物流、云计算、数字娱乐等业务的数字经济体。

数字经济生态的扩展

2009年，阿里云成立，如今已成为全球第三大云计算服务商。阿里云支撑了"双11"的海量交易，其技术能力得到全球认可。在物流领域，2013年成立的菜鸟网络通过智能物流骨干网，大幅提升了中国物流效率。

近年来，阿里巴巴加速整合本地生活服务。2025年8月，淘天集团正式更名，与饿了么、飞猪形成平行架构，强化协同效应。数据显示，淘宝闪购与饿了么资源互通后，日订单总量突破4000万单。阿里还推出了整合饿了么、飞猪等资源的大会员体系，覆盖多元生活场景。

技术创新与社会责任

阿里巴巴在人工智能、区块链、量子计算等前沿技术领域均有布局。达摩院成立于2017年，专注于基础科学和颠覆性技术创新。阿里还积极推动乡村振兴和中小企业数字化，践行"让天下没有难做的生意"的使命。

马云提出的"新零售"概念引领行业变革，盒马鲜生、银泰商业等成为线上线下融合的典范。阿里巴巴正从电商平台向数字经济基础设施提供商转变，推动整个社会的数字化转型。

三巨头的全球化战略比较

小米、华为和阿里巴巴采取了不同的全球化策略，但都取得了显著成就。

小米的国际化采取"先易后难"策略，首先进入印度、东南亚等新兴市场，再进军欧洲。小米在印度复制了中国的成功模式，通过高性价比和线上销售迅速占领市场。2020年，小米全球智能手机出货量达到1.46亿台，成为全球第三大手机厂商。

华为的全球化更加全面，在电信设备、智能手机和企业业务三个领域同时推进。华为在170多个国家和地区开展业务，研发中心遍布全球。华为坚持"全球化思维，本地化运营"，在不同市场采取差异化策略。

阿里巴巴的全球化聚焦于电子商务和云计算。通过收购Lazada、投资Tokopedia等，阿里巴巴加强了在东南亚的影响力。全球速卖通(AliExpress)服务全球消费者，阿里云在美国、欧洲、中东等地建立数据中心。阿里巴巴还提出"电子世界贸易平台"(eWTP)倡议，推动全球跨境电商规则制定。

面临的挑战与未来展望

尽管取得了巨大成功，这三家公司都面临着严峻挑战。

小米需要平衡性价比与高端化的矛盾，提升品牌溢价能力。造车业务投入巨大，可能影响公司盈利能力。国际政治经济环境的不确定性也给全球化带来挑战。

华为面临的地缘政治压力最大，芯片供应受限严重影响手机业务。华为需要加快核心技术自主创新，构建完整的生态系统。云计算和企业业务成为新的增长点，但竞争激烈。

阿里巴巴面临国内电商增长放缓的压力，拼多多、抖音电商等竞争者不断蚕食市场份额。反垄断监管趋严，要求平台更加规范发展。阿里需要找到新的增长引擎，持续创新商业模式。

展望未来，这三家中国科技巨头将继续在全球舞台上发挥重要作用。小米致力于打造全球最大的消费级IoT平台；华为坚持技术创新，构建万物互联的智能世界；阿里巴巴推动商业基础设施数字化，让天下没有难做的生意。它们的成功不仅是中国企业崛起的缩影，也为全球科技产业发展提供了中国方案。

结语：中国科技企业的创新之路

小米、华为和阿里巴巴代表了不同领域的中国科技企业崛起之路。它们的成功有共同点：强烈的危机意识、持续的技术创新、灵活的商业策略和对用户体验的极致追求。这三家公司都从中国市场的激烈竞争中脱颖而出，然后走向全球，成为各自领域的领导者。

- 97 它们的经验表明，中国企业完全可以在全球高科技领域占据重要地位。未来，随着中国科技实力的整体提升，我们有望看到更多像小米、华为和阿里巴巴这样的企业走向世界舞台，为全球科技进步和数字经济发展贡献中国智慧和力量。

执行脚本：

代码块

```
1 cd /root/autodl-tmp/kefu2/graphrag/dev
2 python graphrag_prompt_tune.py
3
```

```
/root/autodl-tmp/kefu2/graphrag/dev
(/root/autodl-tmp/data/graphrag) root@autodl-container-997946b97e-a0631a7e:~/autodl-tmp/kefu2/graphrag/dev# python graphrag_prompt_tune.py

INFO: 从目录加载配置: /root/autodl-tmp/kefu2/graphrag
INFO: 使用数据目录: /root/autodl-tmp/kefu2/graphrag/data
INFO: 输出目录: /root/autodl-tmp/kefu2/graphrag/data/pdf_prompt_turn_output
INFO: 文档块大小: 500
INFO: 文档块重叠大小: 100
INFO: 文档块数量限制: 5
INFO: 文档选择方法: random
INFO: 领域: 自动检测
INFO: 语言: 自动检测
* graphrag_prompt_tune: 开始生成提示词...
INFO: 找到 0 个输入文件
Chunking documents...
Retrieving language model configuration...
⚠max_retries not set, using default value: 5
Creating language model...
Generating domain...
Detecting language...
Generating persona...
Generating community report ranking description...
Generating entity types...
Generating entity relationship examples...
Generating entity extraction prompt...
```

提取出来的Prompt内容都跟我们的数据文件相关

The screenshot shows a file explorer on the left and a terminal window on the right. The file explorer displays a directory structure with files including 'community_report_graph.txt', 'extract_graph.txt' (highlighted), 'metadata.json', and 'summarize_descriptions.txt'. The terminal window shows the output of the 'python graphrag_prompt_tune.py' command, which generates a detailed prompt for entity extraction and relationship identification. The prompt includes instructions for identifying entities, extracting their attributes, and identifying relationships between them.

```
1 -Goal-
2 Given a text document that is potentially relevant to this activity and a list of entity types,
3 identify all entities of those types from the text and all relationships among the identified
4 entities.
5 -Steps-
6 1. Identify all entities. For each identified entity, extract the following information:
7 - entity_name: Name of the entity, capitalized
8 - entity_type: One of the following types: [company, business model, technology, product, market,
9 strategy, innovation, investment, ecosystem, geopolitical challenge, patent, service, competition,
10 regulation, globalization]
11 - entity_description: Comprehensive description of the entity's attributes and activities
12 Format each entity as ("entity"{tuple_delimiter}<entity_name>{tuple_delimiter}<entity_type>
13 {tuple_delimiter}<entity_description>)
14
15 2. From the entities identified in step 1, identify all pairs of (source_entity, target_entity)
16 that are *clearly related* to each other.
17 For each pair of related entities, extract the following information:
18 - source_entity: name of the source entity, as identified in step 1
19 - target_entity: name of the target entity, as identified in step 1
20 - relationship_description: explanation as to why you think the source entity and the target
21 entity are related to each other
22 - relationship_strength: an integer score between 1 to 10, indicating strength of the relationship
23 between the source entity and target entity
24 Format each relationship as ("relationship"{tuple_delimiter}<source_entity>{tuple_delimiter}<
25 target_entity>{tuple_delimiter}<relationship_description>{tuple_delimiter}<relationship_strength>{
```

2.5.2.3 索引构建接口设计

1. 我们直接先把IS_UPDATE = False，执行索引构建流程

模式	中文含义	工作原理	优点	缺点
Standard ("standard")	标准模式	完全依赖大语言模型（LLM）来完成所有步骤： 1. 识别实体和关系 2. 构建知识图谱3. 生成社区摘要	质量更高 · LLM理解能力强，能处理复杂的语言和上下文。 · 提取的实体和关系更准确、更符合语义。 · 生成的摘要质量更好，更连贯。	速度慢，成本高 · 需要调用大量LLM API，非常耗时。 · API调用费用昂贵。
Fast ("fast")	快速模式	混合使用传统NLP工具和LLM： 1. 使用传统NLP工具（如SpaCy）来快速识别实体和关系（这一步很快很便宜）。 2. 构建知识图谱3. 使用LLM 来生成社区摘要（这一步仍然需要LLM以保证质量）。	速度极快，成本低 · 传统NLP工具处理速度快如闪电。 · 大大节约成本	准确率不如大模型

代码块

```
1  ## 修改配置文件
2  CONFIG_FILE ="/root/autodl-tmp/kefu2/graphrag/data/settings.yaml"
3  ## 执行索引构建流程
4  python graphrag_indexing.py
5
```

```
(mygraphrag) root@autodl-container-74ea46b992-cbdd2dab:~/autodl-tmp/kefu2/graphrag/dev# python graphrag_indexing.py
```

```
INFO: 从目录加载配置: /root/autodl-tmp/kefu2/graphrag
INFO: 使用数据目录: /root/autodl-tmp/kefu2/graphrag/data
INFO: 日志保存在: /root/autodl-tmp/kefu2/graphrag/data/logs
INFO: 索引方法: Standard
INFO: 增量更新: 否
INFO: 内存分析: 否
INFO: 使用指定配置文件: /root/autodl-tmp/kefu2/graphrag/data/settings.yaml
```

```
graphrag-indexINFO: 开始构建索引...
```

```
Loading text files from input
```

```
Running standard indexing.
```

```
create_base_text_units
```

	id	n_tokens
0	f7d7c1d1ad15994d4615ea35099bf68bc906e50e0e39b2...	500
1	9425143a08076a2892ef009a1ac7693d32236ec0c924e2...	500
2	4c031cf6bca8615fdaa0adf5f400d8b33aace287eb699a...	500
3	03d6e9561df8b54d499016a752dbc44aacb3fa7cf5f301...	500
4	0bf42b29ada6c82e0172bf6c38d99a679b623c226a1279...	500
5	36caad71eef9018ecb71edc03c7c3a05fb98cd133689bf...	500
6	c8022fc6aa58d185c0aldebb214f50e013cf46e492373a...	500
7	dba6f91247589ea6e57d3e24255aafcb7732e9d887536d...	500
8	03da2617fb3e40e80a7fd61dc140c5310b96acd94f91f0...	500
9	1ca3c064cb3efe73e260b9c673819b12e13437df05b7e0...	500
10	df9b343655f4fd809aef8fa899455b22b8a1266a755323...	500
11	e9630763d01c822c8cac90448f4c2243b22a386c1b9171...	500
12	bd672db4549d654f36662826da73bf6d866a975c1a42cc...	500
13	8ce717adbbbd60b9a77e37907ba7c2f09c0796f8288ae3...	500
14	ffc20e8e1f6daa1df62d81f8efa195f4fafdf9cbb2afb5...	500
15	6e5a6eeac88f328651c20b1116ee33a89a6e28d1dc3cbb...	500
16	3fcfe3068965ae3fc8edafdae25705b88d3c76db198d6a...	500

代码块

```
1  ## 查询测试
2  poetry run poe query --root ./data --method local --query "华为的创始人是谁?"
```

```
6b992-cbdd2dab:~/autodl-tmp/kefu2/graphrag/dev# cd ..
(mygraphrag) root@autodl-container-74ea46b992-cbdd2dab:~/autodl-tmp/kefu2/graphrag# poetry run poe query --root ./data --method local --query "华为的创始人是谁?"
Poe => python -m graphrag query --root ./data --method local --query '华为的创始人是谁?'
```

```
INFO: Vector Store Args: {
  "default_vector_store": {
    "type": "lancedb",
    "db_uri": "/root/autodl-tmp/kefu2/graphrag/data/output\\lancedb",
    "url": null,
    "audience": null,
    "container_name": "==== REDACTED ====",
    "database_name": null,
    "overwrite": true
  }
}
```

```
SUCCESS: Local Search Response:
华为的创始人是任正非。他于1987年在深圳创立了华为公司，初始资本为2.1万元人民币 [Data: Entities (25); Relationships (26, 27, 29); Sources (3, 4, 14)]。任正非作为创始人和领导者，对华为的战略方向和发展产生了深远的影响 [Data: Relationships (25, 114)]。
```

2. 企业常见场景分析

企业常见需求一：如何构建多个索引？

多索引构建对应的开发场景大多存在于企业中存在多个数据源支撑多个业务线，彼此之间相对独立，且每个业务线之间存在一定的差异性，比如：数据格式、数据内容、数据规模等，冗余在一起会加重检索的效率和维护成本。（因为知识图谱越大，检索时需要检验的信息就越多，检索的效率就越低），亦或对不同的部门、人员之间的知识库进行隔离，避免知识库之间的相互污染等常见问题。

这类问题对应的解决方案在当前的流程中非常容易实现：即只需要在构建的时候，通过 `init` 的 `--root` 参数指定不同的数据源根目录，制定索引文件与数据源的对应关系（如通过字典构建映射关系），形式如下：

```
代码块 # 索引文件与数据源的对应关系
```

```
2 INDEX_FILE_MAP = {"index_1": "data_source_1", "index_2": "data_source_2", }
```

各个索引知识库独立构建，在进行问答检索时先通过 `INDEX_FILE_MAP` 字典找到对应的索引文件根路径，进行加载即可实现多知识库的隔离检索。

企业常见需求二：多个不同类型的数据源，如何共同创建一个索引？

多数据源共建一个知识库的场景，在企业中也比较常见。比如关系型数据库、`PDF`、`Word`、`Excel`、`CSV` 等不同类型的数据源，希望各个数据源中的有效信息可以实现整合，从而构建一个统一的知识库提供用户、或者下游的智能体使用。

回顾 `Microsoft GraphRAG` 的索引构建流程，筛选不同文件进入构建索引的 `Workflow` 流程，核心是在 `settings.yaml` 文件中 `input` 参数下借助 `file_type` 参数指定文件类型，以及 `file_parttern` 通过正则表达式的形式过滤符合标准的文件，比如通过 `(.*\\.txt$)` 过滤所有 `txt` 文件。

那么对于这种过滤文件的方式，大家可能想到的是：既然借助 `file_type` 和 `file_parttern` 参数可以过滤文件，那么是否可以借助 `file_type` 和 `file_parttern` 参数，将不同类型的数据源进行整合，从而构建一个统一的知识库？实际上确实可以的，比如指定 `file_pattern: ".*\\. (txt|csv)$"` ,即可同时读取 `input` 目录下所有 `txt` 和 `csv` 文件。

从执行流程上看是可以，但是存在一个非常严重的问题是：无法指定不同文件类型加载对应的优化切分策略。也就是说，`input` 可以通过 `file_type` 和 `file_pattern` 参数指定文件类型和文件过滤规则，但是 `chunk` 下的切分策略，将会用于所有文件的切分，也就是说无法实现不同文件类型加载对应的优化切分策略。

在现有流程上，是不能实现对文件格式和对应的切分策略进行灵活匹配的。因此，这里能得出的结论是：如果大家对所有格式的文件能够容忍使用同一种最简单的切分策略，那么可以直接用 `file_pattern` 进行文件过滤。而如果希望针对不同文件类型，加载对应的优化切分策略，则需要引入增量更新的解决方案。

增量更新（Incremental Update）指的是在已有知识图谱的基础上，快速更新新加入的知识，避免全量重建。一些核心的增量更新的实现方法是：

- a. **文档级增量更新**：这类方法会对文本内容生成MD5哈希值，在实现增量插入功能时，先检查是否存在相同内容，如果判定为新内容，则重新计算社区结构并生成新的社区报告。代表项目是 `nano-GraphRAG`。
- b. **知识库级增量更新**：这类方法会使用图结构管理实体和关系，每次有新文档加入时，会对其执行相同的图索引步骤，包括实体识别和关系提取。通过将新增的节点和边集与原有图的节点和边进行合并（集合并操作），实现新数据的无缝集成，代表项目是 `LightRAG`。

`Microsoft GraphRAG` 在 v0.4.0 版本中引入了增量索引支持，其实现的方法属于**知识库级增量更新**。

接下来我们测试一下增量更新的操作

代码块

```
1  ## 打开增量更新的开关
2  IS_UPDATE = True
3  ## 修改配置文件
4  CONFIG_FILE = "/root/autodl-tmp/kefu2/graphrag/data/settings_csv.yaml"
5  ## 执行索引构建流程
6  python graphrag_indexing.py
```

```
(mygraphrag) root@autodl-container-74ea46b992-cbdd2dab:~/autodl-tmp/kefu2/graphrag/dev# python graphrag_indexing.py
```

```
INFO: 从目录加载配置: /root/autodl-tmp/kefu2/graphrag
INFO: 使用数据目录: /root/autodl-tmp/kefu2/graphrag/data
INFO: 日志保存在: /root/autodl-tmp/kefu2/graphrag/data/logs
INFO: 索引方法: Standard
INFO: 增量更新: 是
INFO: 内存分析: 否
INFO: 使用指定配置文件: /root/autodl-tmp/kefu2/graphrag/data/settings_csv.yaml
```

```
graphrag-indexINFO: 开始构建索引...
```

```
Loading csv files from input
```

```
Running incremental indexing.
```

```
create_base_text_units
```

	id	n_tokens
0	9c6db06c807eb493893d01a0caea706cf5fa6dc9c58ba9...	433
1	9a655aa1a37e99928076234a9f08e25dac07ca6533cc17...	332
2	c554dd0572ee2ad02bc617668836b18d900931e40d8f49...	313
3	083f79c75eef18c942f8475c36685541eae1426be62e3...	447
4	b2a4e0d73fc8a1ea05c342c4ead71f32b331d13c0cd80...	148
5	e1b80dbd4d40076f41d3e1b49e26449e574dd4e6eb505d...	140
6	5691df6bdf84c1fb9e4f689607d62cb5a1a162e2d56de3...	488
7	84c8dbe03aa3e57d95fbb01e266dbb5a50984dd4a92aa1...	158
8	2c8c89cba18e2e2095334c4dc28210f06ad415da0497f8...	363
9	243ec1544d806c89813cae9064514faec5b7334a9ef45a...	430
10	5fd4df57ddc651320da17987783bce6126b88d917ad912...	133
11	4e440193b180dd54b8b9e08ca4edd8c0cb9296ff8b49d3...	366
12	a427fe70ca8c068a2a04ee5beafc725b75ca6226f06168...	297
13	8557407adb55d42bc9f48d186a6cef4f3397b58626af68...	473
14	61e7f6223740113101e4a43ff905dbc99116eba294df05...	175

```
[15 rows x 4 columns]
```

测试:

代码块

```
1  poetry run poe query --root ./data --method local --query "华为的创始人是谁?"
2
3  poetry run poe query --root ./data --method local --query "华成育卓科技有限公司电
   子商务都购买了哪些产品?"
```

```
(mygraphrag) root@autodl-container-74ea46b992-cbdd2dab:~/autodl-tmp/kefu2/graphrag# poetry run poe query --root ./data --method local --query "华成育卓科技有限公司电子商务都购买了哪些产品?"
Poe => python -m graphrag query --root ./data --method local --query '华成育卓科技有限公司电子商务都购买了哪些产品?'

INFO: Vector Store Args: {
  "default_vector_store": {
    "type": "lancedb",
    "db_uri": "/root/autodl-tmp/kefu2/graphrag/data/output\\lancedb",
    "url": null,
    "audience": null,
    "container_name": "==== REDACTED ====",
    "database_name": null,
    "overwrite": true
  }
}

SUCCESS: Local Search Response:
根据提供的客户数据，华成育卓科技有限公司电子商务部门（客户ID: QL111）购买了多款智能家居产品，包括智能冰箱、智能空气净化器和智能洗衣机。以下是具体的购买记录：

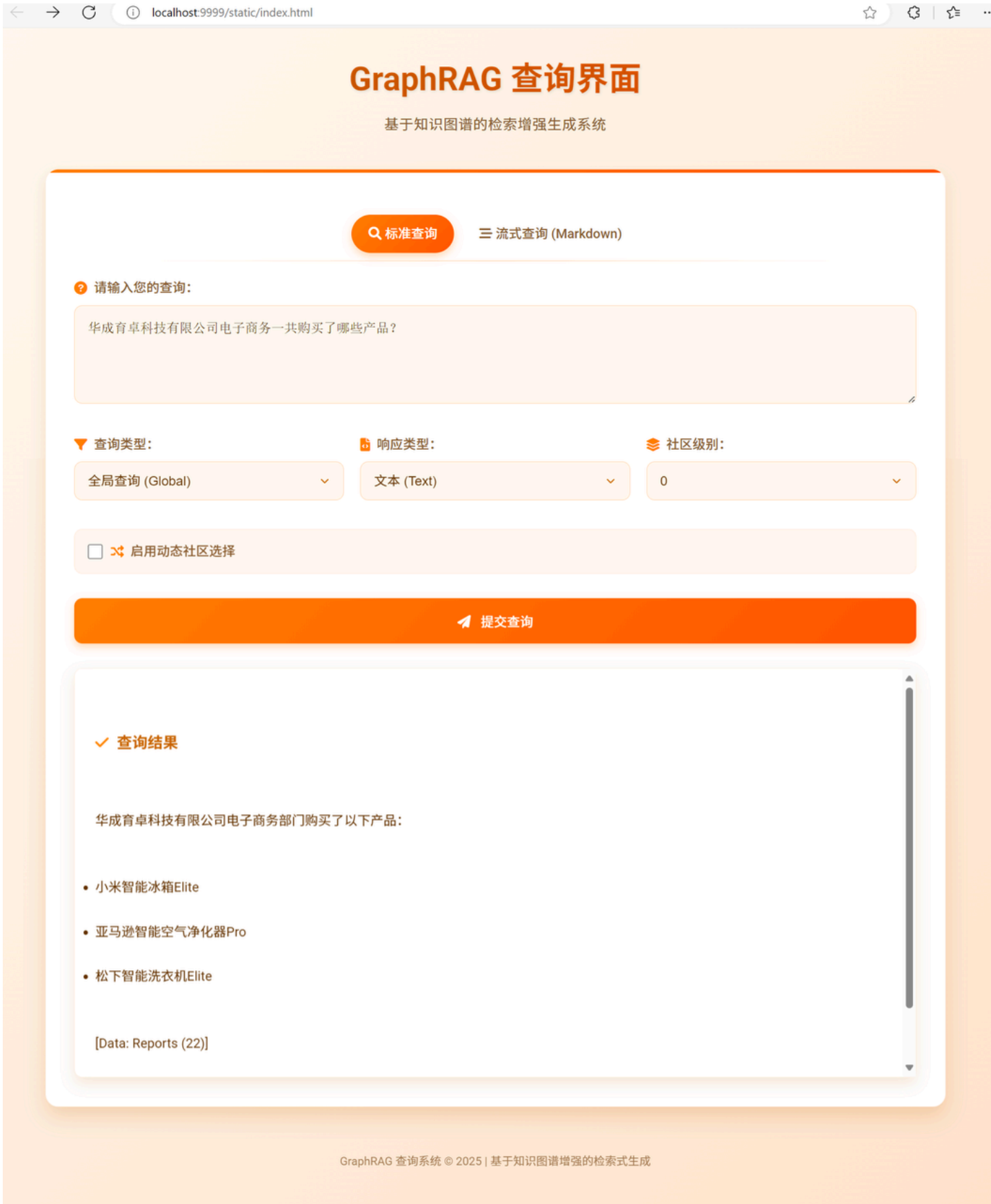
**智能冰箱产品**
客户购买了小米智能冰箱Elite和西门子智能冰箱Elite。小米智能冰箱Elite的价格为3425.86元，获得了4.7星的高评分，客户评价其语音识别功能实用且稳定性好。西门子智能冰箱Elite的价格为2969.73元，但评分较低，仅为1.6星，客户反馈存在故障、耗电量大和售后服务问题 [Data: Sources (25, 26); Relationships (210, 211, 212, 213, 214, 223, 224, 225, 226, 227)]。

**智能空气净化器产品**
客户购买了亚马逊智能空气净化器Pro，价格为2339.08元。该产品评分仅为1.7星，客户抱怨产品有严重问题、使用说明不清，并已申请退款 [Data: Sources (29); Relationships (261, 262, 263, 264, 265)]。
```

2.5.2.4 查询接口设计

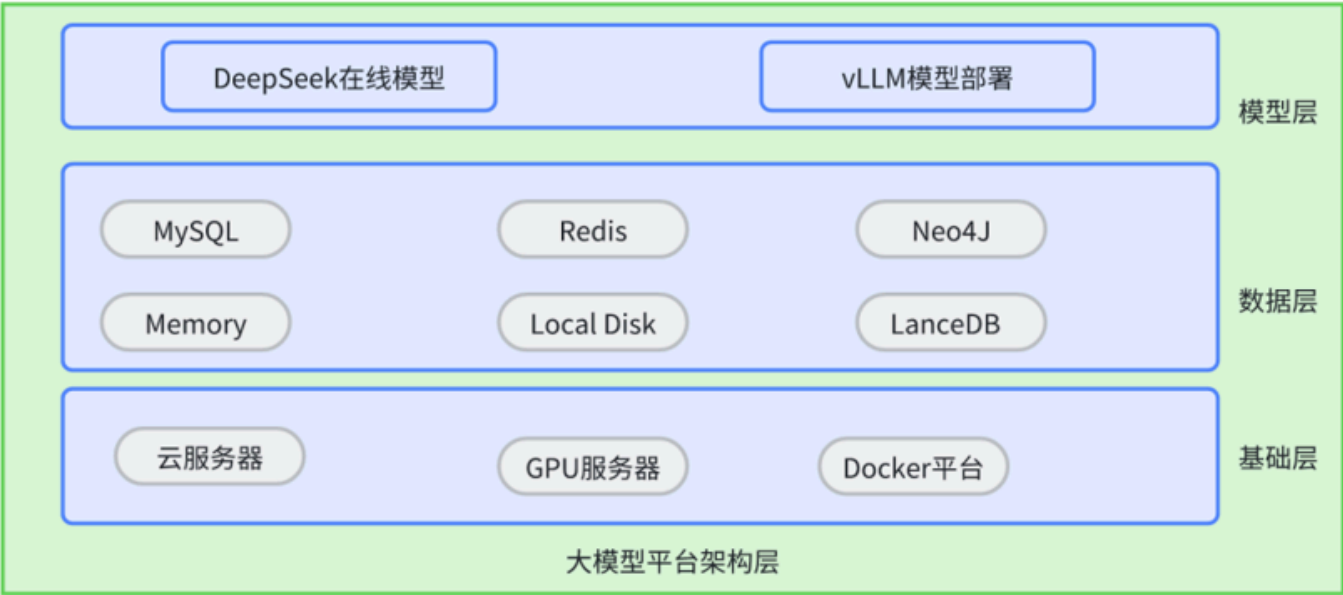
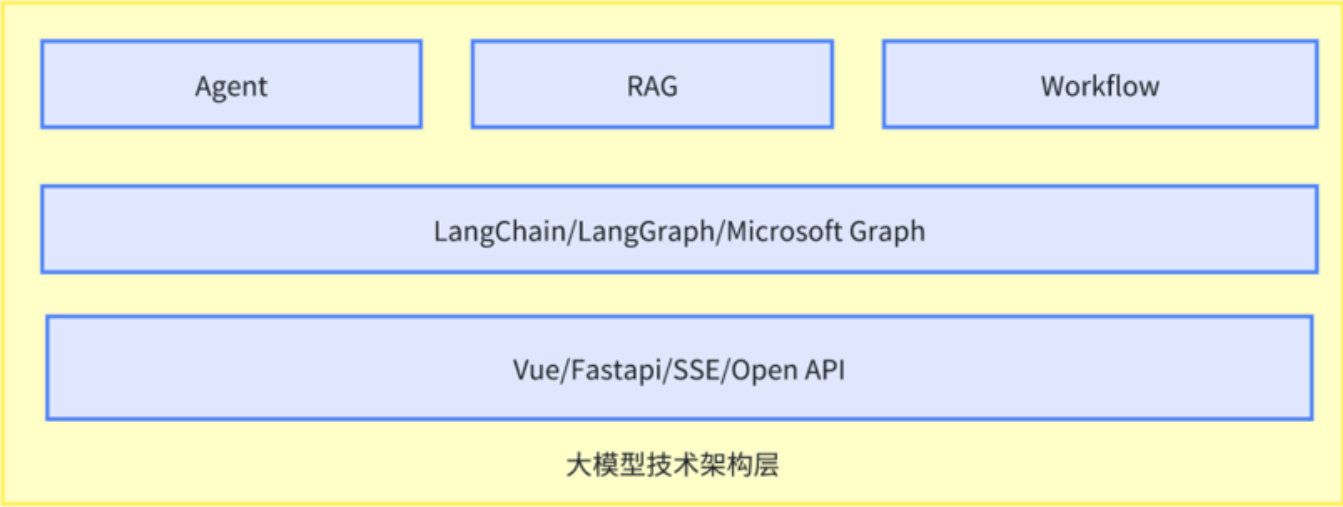
查询方式	查询流程	特点
Local	1. 从用户查询中抽取关键词或实体 2. 在图谱中找到相关的局部子图（与这些实体直接相连的节点与边） 3. 从局部子图召回文档片段 4. 将召回的内容与用户问题结合后，交给大模型生成答案	- 聚焦在“邻域”范围，效率高 - 回答更具体、更贴近用户问题 - 但容易受限于局部上下文，缺乏整体视角
Global	1. 从查询中提炼全局主题或核心问题 2. 在图谱全局范围中检索主题相关的社区/群落 3. 结合全局摘要或社区表示信息 4. 大模型基于全局信息生成答案	- 从整体出发，适合宏观、总结类问题 - 能提供更全面的视角 - 但可能缺少细节，回答偏笼统
Drift	1. 要求LLM基于用户查询生成一个假设性的答案（即使这个答案可能是错误的或未被证实的）。 2. 将这个假设性答案转换为向量，并用它去知识图谱中进行检索。 3. 检索出与这个假设最相关的实体和社区信息作为证据。 4. LLM根据检索到的证据来评估、修正或支持最初生成的假设，最终形成一个有根据的答案。	· “大胆假设，小心求证”：通过假设引导检索方向。 · 非常适合处理复杂、开放或需要推理的问题，传统检索方式可能找不到直接答案。 · 能探索“如果”式的问题和概念之间的新颖联系。 适用场景： · 复杂的因果分析（例如：“如果某事件没发生，会有什么影响？”）。 · 需要多步推理的开放域问题（例如：“某项技术未来将如何发展？”）。 · 发现非显而易见的见解和联系。
Basic	1. 不利用图结构，只在底层文档（chunks/向量库）中做语义检索 2. 召回若干相关文档片段 3. 交给大模型回答问题	- 最接近传统 RAG 的方式 - 简单直接，效果稳定 - 但无法利用知识图谱结构优势，缺少推理能力

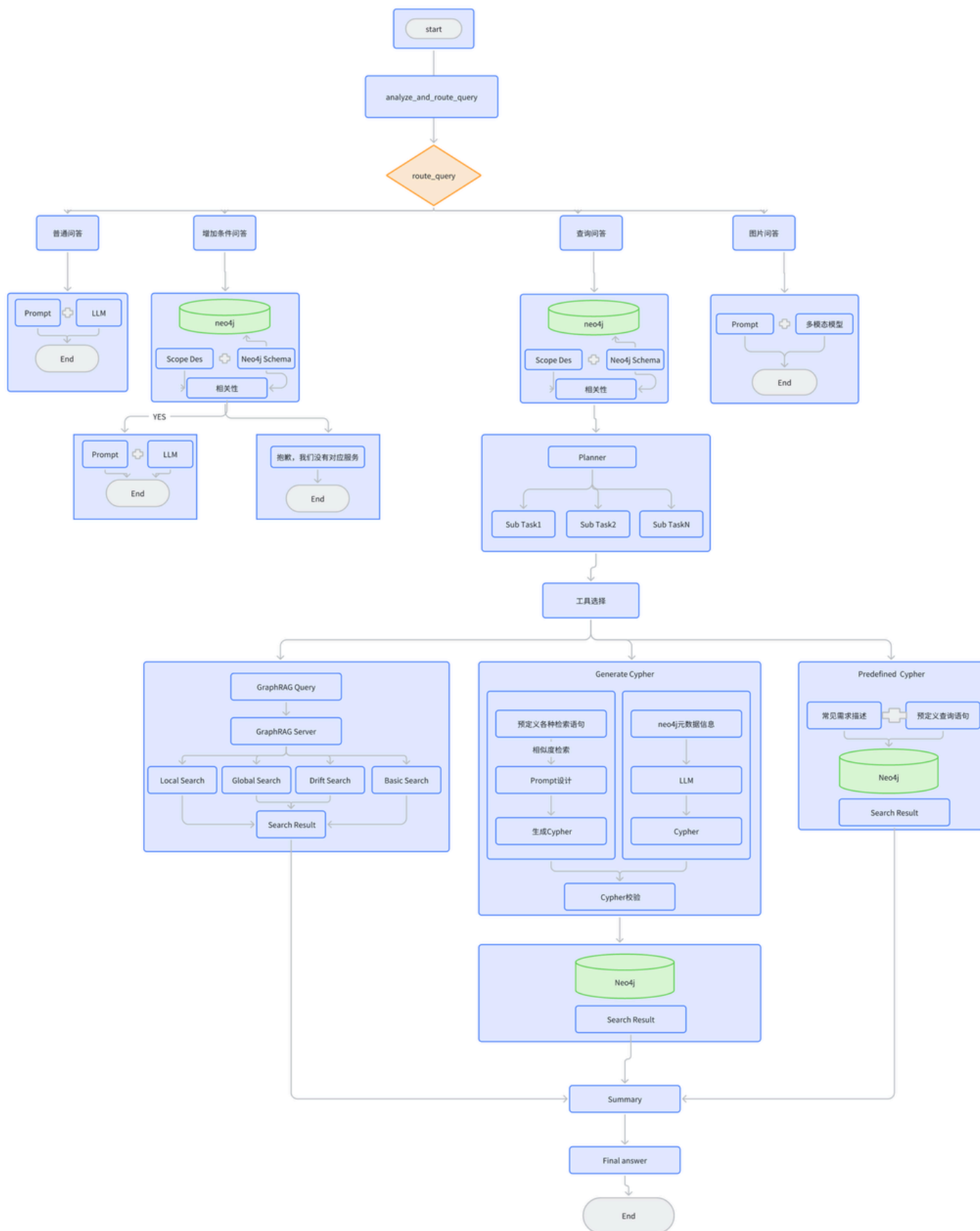
2.5.2.5 前后端接口联调测试



2.6 企业级智能客服整体架构全流程落地

2.6.1 智能客服 Multi Agent 架构设计





步骤	动作
1	配置DeepSeek模型
2	配置Ollama模型

3	配置GraphRAG参数
4	配置SerpAPI
5	配置MySQL
6	配置Neo4j数据库
7	配置Redis
8	配置GraphRAG位置
9	构建虚拟环境
10	初始化MySQL数据库
11	启动项目：python run.py

2.6.2 智能客服意图识别设计

代码块

```
1  ROUTER_SYSTEM_PROMPT = """你是一个电商领域的智能客服。你的工作是帮助用户解决与产品、订
   单、售后和技术支持相关的问题。
2
3  用户会向你提出询问。你的首要任务是对询问类型进行分类。你必须将用户询问的问题分类为以下类型
   之一：
4
5  ## `general-query`
6  如果是一般性问题，不需要查询知识库就能回答，请将其分类为此类。
7  包括但不限于：
8  - 与商品、订单、售后、技术支持无关的闲聊问题
9
10 ## `additional-query`
11 如果你需要更多信息才能帮助用户，请将用户询问分类为此类。例如：
12 - 用户询问商品但没有提供具体型号或规格
13 - 用户询问订单状态但没有提供订单号
14 - 用户描述问题不够具体，无法提供准确帮助
15
16 ## `graphrag-query`
17 如果通过查询本地知识库可以回答用户询问，请将其分类为此类。
18 包括但不限于：
19 - 商品价格、库存、规格等信息查询
20 - 订单状态、物流信息查询
21 - 会员积分、优惠活动查询
22 - 退换货政策咨询
```

```
23 - 商品使用指导及故障解决方法
24
25 ## `image-query`
26 如果用户提供了图片请求提供图片，请将其分类为此类。
27
28
29 ## `file-query`
30 如果用户上传了文件，请将其分类为此类。
31 """"
```

2.6.3 智能客服普通问答流程

代码块

```
1 # 一般查询提示，即用户的问题与电商、产品、订单、技术支持无关
2 GENERAL_QUERY_SYSTEM_PROMPT = """"你是一个电商领域的智能客服。你的工作是帮助用户解决与
  产品、订单、售后和技术支持相关的问题。
3
4 请以类似淘宝/京东等知名电商客服的风格回复用户，遵循以下规则：
5
6 ## 基本礼仪
7 1. 开场必须用"亲～"或"顾客您好～"问候
8 2. 使用积极、温暖的语气
9 3. 适当使用emoji表情（如 🙌 😊 ❤️）增加亲和力
10 4. 结尾必须表达感谢和继续服务的意愿
11
12 ## 回复策略
13 如果用户问题模糊不清：
14 1. 先表示理解和重视："感谢您的咨询～"
15 2. 友好地指出不明确之处："不过亲，为了能更好地帮助您..."
16 3. 引导用户提供更具体信息："您能告诉我具体是..."
17 4. 给出一些示例："比如说，您是想问..."
18
19 ## 如果问题与电商业务无关：
20 1. 先表示感谢："感谢您的咨询～"
21 2. 委婉说明情况："不好意思呢亲，这个问题可能不太属于我们的业务范围..."
22 3. 建议其他帮助渠道："您可以尝试咨询相关专业的平台/机构..."
23 4. 表达歉意和继续服务意愿："如果您有任何关于我们商品或服务的问题，随时欢迎询问哦～"
24
25 ## 回复要点
26 - 保持专业性：提供清晰、准确的信息
27 - 给出建议：在可能的情况下提供实用的解决方案
28 - 态度温和：即使是拒绝也要保持友好
29 - 预设期待：为后续可能的咨询留下互动空间
30
31 ## 示例回复格式
```

```

32
33   ### 模糊问题示例：
34   "亲～感谢您的咨询～😊 为了能更准确地帮助您，麻烦您能告诉我具体是哪款商品呢？这样我才能...
    （根据具体情况询问） "
35
36   ### 无关问题示例：
37   "亲～感谢您的咨询～😊 非常抱歉呢，这个问题可能不太属于我们的业务范围，建议您...（给出
    建议）。如果您有任何关于我们商品或服务的问题，随时欢迎询问哦～❤️"
38
39   系统已确定用户正在提出一般性问题，不需要查询特定数据库即可回答。以下是分类理由：
40
41   <logic>
42   {logic}
43   </logic>
44
45   记住：无论是哪种情况，都要保持亲切、专业的语气，让用户感受到被重视和理解。
46   同时，尽量保持回复的简洁性，不要过于冗长，尽量限制在20个字以内。
47   """

```

2.6.4 智能客服安全护栏设计

代码块

```

1  GUARDRAILS_SYSTEM_PROMPT = """
2  你是企业产品信息与订单管理系统中的范围检查组件。
3  你的职责是确定用户的问题是否在系统的处理范围内。
4
5  请遵循以下规则：
6  1. 如果问题与企业商品信息与订单管理相关，包括但不限于：
7      - 商品信息查询
8      - 商品分类信息
9      - 供应商信息
10     - 客户信息
11     - 订单信息和状态
12     - 员工信息
13     - 物流信息
14
15     如果相关，请仅输出："continue"
16  2. 如果问题明显与企业商品信息与订单管理无关（如政治、娱乐、天气等），请仅输出："end"
17  3. 如产生疑问，请假设问题可能相关，宁可错误地接受也不要错误地拒绝
18  4. 在做决定时，请严格按照系统的数据库结构来判断，如果不相关，则不属于经营范围，请输
    出："end"
19  5. 仅输出指定的结果："continue"或"end"
20  """

```

```

1 GET_ADDITIONAL_SYSTEM_PROMPT = """你是一个专业的电商客服助手。你的工作是帮助用户解决各
   种电商相关问题。
2
3 你应该使用亲切、专业的语气，类似淘宝和京东的客服风格：
4 1. 称呼用户为"亲"或"顾客"
5 2. 使用礼貌用语，如"请问"、"麻烦"、"感谢"等
6 3. 保持积极正面的语气
7 4. 适当使用emoji表情，让对话更亲和
8 5. 结尾要表达感谢和继续服务的意愿
9
10
11 系统确定在为用户提供帮助前需要更多信息。以下是分类理由：
12
13 <logic>
14 {logic}
15 </logic>
16
17 请友好地向用户询问所需的额外信息。表达理解用户需求的意愿，并明确说明需要哪些信息才能更好地
   提供帮助。提问要具体，一次只问一个问题，避免让用户感到困惑。
18 注意：同样要保持简短，尽量限制在20个字以内。
19 """

```

2.6.5 智能客服混合知识库设计

1. 问题拆解

代码块

```

1 PLANNER_SYSTEM_PROMPT = """
2 你是一个电商平台智能客服系统中的任务规划组件。
3 你的职责是分析用户的查询，并将其拆分为独立的子任务。
4
5 请遵循以下规则：
6 1. 如果问题可以拆分为多个独立子任务，返回这些子任务列表
7 2. 如果问题很简单，不需要拆分，则返回一个只包含原问题的列表
8 3. 子任务之间应该是独立的，不要相互依赖
9 4. 确保子任务不会返回重复或相似的信息
10 5. 将相互依赖的任务合并为一个问题
11 6. 将返回相同信息的任务合并为一个问题
12
13 示例：
14 - 问题："北风商贸有哪些饮料类产品？它们的价格是多少？"
15   子任务：["北风商贸有哪些饮料类产品？", "北风商贸饮料类产品的价格是多少？"]
16
17 - 问题："谁是处理了订单10248的员工？这个订单发往哪里？"
18   子任务：["谁是处理了订单10248的员工？", "订单10248的配送信息是什么？"]
19

```

```

20 - 问题: "供应商Exotic Liquids提供了哪些产品? 这些产品的库存情况如何? "
21   子任务: ["供应商Exotic Liquids提供了哪些产品?", "Exotic Liquids供应的产品库存情况
22   如何? "]
23 - 问题: "订单10248包含哪些产品? 这些产品是哪家供应商提供的? "
24   子任务: ["订单10248包含哪些产品?", "订单10248中的产品分别由哪些供应商提供? "]
25
26 - 问题: "茶叶产品的使用说明手册在哪里可以找到? "
27   子任务: ["茶叶产品的使用说明手册在哪里? "]
28
29 - 问题: "客户对北风商贸巧克力产品的评价如何? "
30   子任务: ["客户对北风商贸巧克力产品的评价? "]
31   ""
32

```

2. 工具选择

代码块

```

1  TOOL_SELECTION_SYSTEM_PROMPT = ""
2  你是一个电商平台智能客服系统中的工具选择组件。你的职责是根据用户的问题选择合适的工具。
3  你默认应该选择text2cypher工具，除非其他的工具完全匹配用户输入的问题。
4  ""

```

代码块

```

1  class cypher_query(BaseModel):
2      """如果用户问的是关于产品价格、库存、规格等，则使用这个工具，生成Cypher查询语句进行查
3      询"""
4      task: str = Field(..., description="The task the Cypher query must
5      answer.")

```

代码块

```

1  class predefined_cypher(BaseModel):
2      """这个工具包含预定义的Cypher查询语句，用于快速响应各种电商场景的查询需求。
3
4      根据用户问题的类型，可以选择以下类别的查询：
5
6      1. 产品类查询：
7          - product_by_name: 通过产品名称查询产品信息
8          - product_by_category: 通过类别名称查询该类别下的所有产品
9          - product_by_supplier: 查询特定供应商提供的所有产品
10         - products_low_stock: 查询库存不足的产品

```

```
11         - products_popular: 查询最受欢迎(评论最多)的产品
12
13     2. 客户类查询:
14         - customer_by_name: 通过名称查询客户信息
15         - customer_orders: 查询特定客户的所有订单
16         - customer_purchase_history: 查询特定客户的购买历史
17
18     3. 订单类查询:
19         - order_by_id: 通过订单ID查询订单信息
20         - order_details: 查询特定订单的详细信息(包含的产品)
21         - recent_orders: 查询最近的订单
22         - delayed_orders: 查询延迟发货的订单
23
24     4. 供应商类查询:
25         - supplier_by_country: 查询特定国家的供应商
26         - supplier_products: 查询特定供应商提供的所有产品
27
28     5. 类别类查询:
29         - all_categories: 查询所有产品类别
30         - category_products: 查询特定类别下的所有产品
31         - category_product_count: 查询每个类别包含的产品数量
32
33     6. 员工类查询:
34         - employee_by_name: 通过姓名查询员工信息
35         - employee_processed_orders: 查询特定员工处理的所有订单
36
37     7. 评论类查询:
38         - product_reviews: 查询特定产品的所有评论
39         - top_rated_products: 查询评分最高的产品
40
41     8. 销售分析类查询:
42         - product_sales: 查询特定产品的总销售额
43         - category_sales: 查询各类别的总销售额
44         - monthly_sales: 查询每月的销售情况
45
46     9. 智能家居相关查询:
47         - smart_home_products: 查询所有智能家居产品
48         - smart_speakers: 查询智能音箱类产品
49         - smart_lighting: 查询智能照明类产品
50
51     请根据用户的问题选择最合适的查询, 并根据需要替换查询中的参数值 (如$product_name,
52     $category_name等)。
53     """
54     query: str = Field(..., description="query the graph must include the
question")
```

```
55     parameters: dict = Field(..., description="parameters for the query to Neo4j")
```

代码块

```
1 class microsoft_graphrag_query(BaseModel):
2     """如果用户问的问题是关于产品的故障、售后、保修、维修、退换货以及评价等，则使用这个工具"""
3     query: str = Field(..., description="query the graph must include the question")
```

2.6.6 智能客服Text2Code功能设计

1. 整体的图

```
# 定义状态图
builder = StateGraph(AgentState, input=InputState)
# 添加节点
builder.add_node(analyze_and_route_query)
builder.add_node(respond_to_general_query)
builder.add_node(get_additional_info)
builder.add_node("create_research_plan", create_research_plan) # 这里是子图
builder.add_node(create_image_query)
builder.add_node(create_file_query)

# 添加边
builder.add_edge(START, "analyze_and_route_query")
builder.add_conditional_edges("analyze_and_route_query", route_query)
```

2. 核心关注路由

```
2
3
4 # 构建日志记录器
5 logger = get_logger(service="lg_builder")
6
7 async def analyze_and_route_query(
8     state: AgentState, *, config: RunnableConfig
9 ) -> dict[str, Router]:
10     """Analyze the user's query and determine the appropriate
```

分类提示词

```
5
6 # 路由系统提示
7 ROUTER_SYSTEM_PROMPT = """你是一个电商领域的智能客服。你的工作是帮助用户解决与产品、订单、售后和技术支持相关的问题。
8
9 用户会向你提出询问。你的首要任务是对询问类型进行分类。你必须将用户询问的问题分类为以下类型之一：
10
11 ## `general-query`
12 如果是一般性问题，不需要查询知识库就能回答，请将其分类为此类。
13 包括但不限于：
14 - 与商品、订单、售后、技术支持无关的闲聊问题
15
16 ## `additional-query`
17 如果你需要更多信息才能帮助用户，请将用户询问分类为此类。例如：
18 - 用户询问商品但没有提供具体型号或规格
19 - 用户询问订单状态但没有提供订单号
20 - 用户描述问题不够具体，无法提供准确帮助
21
22 ## `graphrag-query`
23 如果通过查询本地知识库可以回答用户询问，请将其分类为此类。
24 包括但不限于：
25 - 商品价格、库存、规格等信息查询
```

返回结果：

```
class Router(TypedDict):
    """Classify user query."""
    logic: str
    type: Literal["general-query", "additional-query", "graphrag-query", "image-query", "file-query"]
    question: str = field(default_factory=str)
```

3. 关注条件边

```
106
107 # 检查配置中是否有图片路径，如果有，优先处理为图片查询
108 if hasattr(state, "config") and state.config and state.config.get("configurable", {}).get("image_
109     logger.info("检测到图片路径，转为图片查询处理")
110     return "create_image_query"
111
112 if _type == "general-query":
113     return "respond_to_general_query"
114 elif _type == "additional-query":
115     return "get_additional_info"
116 elif _type == "graphrag-query":
117     return "create_research_plan"
118 elif _type == "image-query":
119     return "create_image_query"
120 elif _type == "file-query":
121     return "create_file_query"
122 else:
123     raise ValueError(f"Unknown router type {_type}")
124
125 async def respond_to_general_query(
```

4. 第一种逻辑：普通问答

respond_to_general_query

```
37 """
38
39 # 一般查询提示，即用户的问题与电商、产品、订单、技术支持无关
40 GENERAL_QUERY_SYSTEM_PROMPT = """你是一个电商领域的智能客服。你的工作是帮助用户解决与产品、订单、售后和技术支持相关的问
41
42 请以类似淘宝/京东等知名电商客服的风格回复用户，遵循以下规则：
43
44 ## 基本礼仪
45 1. 开场必须用"亲~"或"顾客您好~"问候
46 2. 使用积极、温暖的语气
47 3. 适当使用emoji表情（如 🙌 😊 ❤️）增加亲和力
48 4. 结尾必须表达感谢和继续服务的意愿
49
50 ## 回复策略
51 如果用户问题模糊不清：
52 1. 先表示理解和重视："感谢您的咨询~"
53 2. 友好地指出不明确之处："不过亲，为了更好地帮助您..."
```

5. 第二种逻辑：get_additional_info

```
09     else ""
10 )
11
12 message = scope_context + graph_context + "\nQuestion: {question}"
13
14 # 拼接提示模版
15 full_system_prompt = ChatPromptTemplate.from_messages(
16     [
17         (
18             "system",
19             GUARDRAILS_SYSTEM_PROMPT,
20         ),
21         (
22             "human",
23             (message),
24         ),
25     ]
26 )
27
```

你的职责是确定用户的问题是否在系统的处理范围内。

请遵循以下规则：

1. 如果问题与企业商品信息与订单管理相关，包括但不限于：

- 商品信息查询
- 商品分类信息
- 供应商信息
- 客户信息
- 订单信息和状态
- 员工信息
- 物流信息

如果相关，请仅输出：`"continue"`

2. 如果问题明显与企业商品信息与订单管理无关（如政治、娱乐、天气等），请仅输出：`"end"`

3. 如产生疑问，请假设问题可能相关，宁可错误地接受也不要错误地拒绝

4. 在做决定时，请严格按照系统的数据库结构来判断，如果不相关，则不属于经营范围，请输出：`"end"`

5. 仅输出指定的结果：`"continue"`或`"end"`

"""

根据格式化输出的结果，返回不同的响应

if guardrails_output.decision == "end":

logger.info("-----Fail to pass guardrails check-----")

return {"messages": [AIMessage(content="抱歉，我家暂时没有这方面的商品，可以在别家看看哦~")]}

else:

logger.info("-----Pass guardrails check-----")

system_prompt = GET_ADDITIONAL_SYSTEM_PROMPT.format(

logic=state.router["logic"]

)

messages = [{"role": "system", "content": system_prompt}] + state.messages

response = await model.ainvoke(messages)

return {"messages": [response]}

async def create_image_query(

state: AgentState, *, config: RunnableConfig

) -> Dict[str, List[BaseMessage]]:

"""处理图片查询并生成描述词"""

```
88 # 补充信息查询：即用户的意图不明显，需要补充额外信息
89 GET_ADDITIONAL_SYSTEM_PROMPT = """你是一个专业的电商客服助手。你的工作是帮助用户解决各种电商相关问题。
90
91 你应该使用亲切、专业的语气，类似淘宝和京东的客服风格：
92 1. 称呼用户为"亲"或"顾客"
93 2. 使用礼貌用语，如"请问"、"麻烦"、"感谢"等
94 3. 保持积极正面的语气
95 4. 适当使用emoji表情，让对话更亲和
96 5. 结尾要表达感谢和继续服务的意愿
97
98
99 系统确定在为用户提供帮助前需要更多信息。以下是分类理由：
100
101 <logic>
102 {logic}
103 </logic>
104
105 请友好地向用户询问所需的额外信息。表达理解用户需求的意愿，并明确说明需要哪些信息才能更好地提供帮助。提问要具体，一次只问一个问题
106 注意：同样要保持简短，尽量限制在20个字以内。
107 """
```

6. 任务拆解：create_research_plan

```
"""
# 创建多工具工作流
multi_tool_workflow = create_multi_tool_workflow(
    llm=model,
    graph=neo4j_graph,
    tool_schemas=tool_schemas,
    predefined_cypher_dict=predefined_cypher_dict,
    cypher_example_retriever=cypher_retriever,
    scope_description=scope_description,
    llm_cypher_validation=True,
)

# return multi_tool_workflow
```

安全护栏

```
"""
# 1. 创建guardrails节点
# Guardrails 节点决定传入的问题是否在检索的范围内（比如是否和电商（自家的产品相关））。如果不在，则提供默认
guardrails = create_guardrails_node(
    llm=llm, graph=graph, scope_description=scope_description
)
```

拆解子任务

```
# 2. 如果通过guardrails, 则会针对用户的问题进行任务分解
```

```
planner = create_planner_node(llm=llm)
```

```
33
34 # 分析用户问题并规划任务的提示词
35 PLANNER_SYSTEM_PROMPT = """
36 你是一个电商平台智能客服系统中的任务规划组件。
37 你的职责是分析用户的查询，并将其拆分为独立的子任务。
38
39 请遵循以下规则：
40 1. 如果问题可以拆分为多个独立子任务，返回这些子任务列表
41 2. 如果问题很简单，不需要拆分，则返回一个只包含原问题的列表
42 3. 子任务之间应该是独立的，不要相互依赖
43 4. 确保子任务不会返回重复或相似的信息
44 5. 将相互依赖的任务合并为一个问题
45 6. 将返回相同信息的任务合并为一个问题
46
47 示例：
48 - 问题: "北风商贸有哪些饮料类产品？它们的价格是多少？"
49   子任务: ["北风商贸有哪些饮料类产品？", "北风商贸饮料类产品的价格是多少？"]
50
51 - 问题: "谁是处理了订单10248的员工？这个订单发往哪里？"
52   子任务: ["谁是处理了订单10248的员工？", "订单10248的配送信息是什么？"]
53
54 - 问题: "供应商Exotic Liquids提供了哪些产品？这些产品的库存情况如何？"
```

并行查询

```

2 usages
def map_reduce_planner_to_tool_selection(state: OverallState) -> List[Send]:
    """Map each identified task in the planner stage to a tool_selection node."""
    return [
        Send(
            "tool_selection",
            {
                "question": task.question,
                "parent_task": task.parent_task,
            },
        )
        for task in state.get("tasks", list())
    ]

```

```

57 def create_multi_tool_workflow(
137     main_graph_builder.add_node(planner)
138     main_graph_builder.add_node("cypher_query", cypher_query)
139     main_graph_builder.add_node(predefined_cypher)
139     main_graph_builder.add_node("customer_tools", customer_tools)
140     main_graph_builder.add_node(summarize)
141     main_graph_builder.add_node(tool_selection)
142     main_graph_builder.add_node(final_answer)
143
144
145     # 添加边
146     main_graph_builder.add_edge(START, "guardrails")
147     main_graph_builder.add_conditional_edges(
148         "guardrails",
149         guardrails_conditional_edge,
150     )
151     main_graph_builder.add_conditional_edges(
152         "planner",
153         map_reduce_planner_to_tool_selection, # type: ignore[arg-type, unused-ignore]
154         ["tool_selection"],
155     )

```

工具选择节点

```

💡 # 工具选择节点，根据用户的问题选择合适的工具
tool_selection = create_tool_selection_node(
    llm=llm,
    tool_schemas=tool_schemas,
    default_to_text2cypher=default_to_text2cypher,
)

```

```
TOOL_SELECTION_SYSTEM_PROMPT = """
```

你是一个电商平台智能客服系统中的工具选择组件。你的职责是根据用户的问题选择合适的工具。
你默认应该选择text2cypher工具，除非其他的工具完全匹配用户输入的问题。

```
"""
```

```
# step 3. 定义工具模式列表
```

```
from app.lg_agent.kg_sub_graph.kg_tools_list import cypher_query, predefined_cypher, microsoft_graphrag_query  
tool_schemas: List[type[BaseModel]] = [cypher_query, predefined_cypher, microsoft_graphrag_query]
```

```
# 3. 预定义的Cypher查询 - 为电商场景定义有用的查询
```

```
from app.lg_agent.kg_sub_graph.agentic_agents.components.predefined_cypher.cypher_dict import
```

```
class cypher_query(BaseModel):
```

```
    """如果用户问的是关于产品价格、库存、规格等，则使用这个工具，生成Cypher查询语句进行查询"""
```

```
    task: str = Field(..., description="The task the Cypher query must answer.")
```

```
2 messages
```

```
class predefined_cypher(BaseModel):
```

```
    """这个工具包含预定义的Cypher查询语句，用于快速响应各种电商场景的查询需求。
```

```
    根据用户问题的类型，可以选择以下类别的查询：
```

```
    1. 产品类查询：
```

- product_by_name: 通过产品名称查询产品信息
- product_by_category: 通过类别名称查询该类别下的所有产品
- product_by_supplier: 查询特定供应商提供的所有产品
- products_low_stock: 查询库存不足的产品
- products_popular: 查询最受欢迎(评论最多)的产品

```
    2. 客户类查询：
```

- customer_by_name: 通过名称查询客户信息
- customer_orders: 查询特定客户的所有订单
- customer_purchase_history: 查询特定客户的购买历史

```
    query: str = Field(..., description="query the graph must include the question")
```

```
    parameters: dict = Field(..., description="parameters for the query to Neo4j")
```

```
1
```

```
class microsoft_graphrag_query(BaseModel):
```

```
    """如果用户问的问题是产品的故障、售后、保修、维修、退换货以及评价等，则使用这个工具"""
```

```
    query: str = Field(..., description="query the graph must include the question")
```

工具节点1: predefined_cypher

```

07         llm=llm, graph=graph, scope_description=scope_description
08     )
09
10     # 2. 如果通过guardrails, 则会针对用户的问题进行任务分解
11     planner = create_planner_node(llm=llm)
12
13     # 3. 创建cypher_query节点, 用来根据用户的问题生成Cypher查询语句
14     cypher_query = create_cypher_query_node()
15
16     predefined_cypher = create_predefined_cypher_node(
17         graph=graph, predefined_cypher_dict=predefined_cypher_dict
18     )
19
20     customer_tools = create_graphrag_query_node()
21
22     # 工具选择节点, 根据用户的问题选择合适的工具
23     tool_selection = create_tool_selection_node(
24         llm=llm,
25         tool_schemas=tool_schemas,
26         default_to_text2cypher=default_to_text2cypher,

```

```

v predefined_cypher_dict: Dict[str, str] = {
    # 产品类查询
    "product_by_name": "MATCH (p:Product) WHERE p.ProductName CONTAINS $product_name RETURN p.ProductName",
    "product_by_category": "MATCH (p:Product)-[:BELONGS_TO]->(c:Category) WHERE c.CategoryName = $category_name RETURN p.ProductName",
    "product_by_supplier": "MATCH (p:Product)-[:SUPPLIED_BY]->(s:Supplier) WHERE s.CompanyName = $supplier_name RETURN p.ProductName",
    "products_low_stock": "MATCH (p:Product) WHERE toInteger(p.UnitsInStock) < 10 RETURN p.ProductName",
    "products_popular": "MATCH (p:Product)-[:ABOUT]-(r:Review) RETURN p.ProductName, count(r) as ReviewCount",

    # 客户类查询
    "customer_by_name": "MATCH (c:Customer) WHERE c.CompanyName CONTAINS $customer_name RETURN c.CompanyName",
    "customer_orders": "MATCH (c:Customer)-[:PLACED]->(o:Order) WHERE c.CompanyName = $customer_name RETURN o.orderId",
    "customer_purchase_history": "MATCH (c:Customer)-[:PLACED]->(o:Order)-[:CONTAINS]->(p:Product) WHERE c.CompanyName = $customer_name RETURN p.ProductName",

    # 订单类查询
    "order_by_id": "MATCH (o:Order) WHERE o.orderId = $order_id RETURN o.OrderDate, o.RequiredDate, o.CustomerName",
    "order_details": "MATCH (o:Order)-[:CONTAINS]->(p:Product) WHERE o.orderId = $order_id RETURN p.ProductName",
    "recent_orders": "MATCH (o:Order) RETURN o.orderId, o.OrderDate, o.CustomerName ORDER BY o.OrderDate DESC",
    "delayed_orders": "MATCH (o:Order) WHERE o.RequiredDate < o.ShippedDate OR (o.RequiredDate < date()) RETURN o.orderId, o.OrderDate, o.CustomerName"
}

```

工具节点2: cypher_query

```
# 2. 如果通过guardrails, 则会针对用户的问题进行任务分解
```

```
planner = create_planner_node(llm=llm)
```

```
# 3. 创建cypher_query节点, 用来根据用户的问题生成Cypher查询语句
```

```
cypher_query = create_cypher_query_node()
```

```
predefined_cypher = create_predefined_cypher_node(
```

```
    graph=graph, predefined_cypher_dict=predefined_cypher_dict
```

```
)
```

```
# step 2. 创建自定义检索器实例, 根据 Graph Schema 创建 Cypher 示例, 用来引导大模型生成正确的Cypher 查询
```

```
cypher_retriever = NorthwindCypherRetriever()
```

```
# step 2. 创建自定义检索器实例, 根据 Graph Schema 创建 Cypher 示例, 用来引导大模型生成正确的Cypher 查询
```

```
cypher_retriever = NorthwindCypherRetriever()
```

```
# Step 3. 根据自定义的 Cypher 示例, 引导大模型生成 当前输入 问题的 Cypher 查询语句
```

```
cypher_generation = create_text2cypher_generation_node(
```

```
    llm=model, graph=neo4j_graph, cypher_example_retriever=cypher_retriever
```

```
)
```

```
# step 4. 验证生成的 Cypher 查询语句是否正确
```

```
validate_cypher = create_text2cypher_validation_node(
```

```
    llm=model,
```

```
    graph=neo4j_graph,
```

```
    llm_validation=True,
```

```
    cypher_statement=cypher_result
```

```
)
```

```
errors = []
```

```
mapping_errors = []
```

```
# 1. 语法校验: 检查Cypher查询的语法是否正确, 例如括号匹配、关键字使用等。
```

```
syntax_error = validate_cypher_query_syntax(
```

```
    graph=graph, cypher_statement=cypher_statement
```

```
)
```

```
errors.extend(syntax_error)
```

```
errors.extend(syntax_error)
```

```
# 检查Cypher查询中是否包含写操作(如CREATE、DELETE、SET等), 防止大模型意外修改数据库,  
write_errors = validate_no_writes_in_cypher_query(cypher_statement=cypher_statement)  
errors.extend(write_errors)]
```

```
# Neo4j的关系是有方向性的。这一步会检查关系方向是否正确, 如果不正确, 会尝试自动修复。这对提高查询成功率很重要。
```

```
errors.extend(write_errors)
```

```
# Neo4j的关系是有方向性的。这一步会检查关系方向是否正确, 如果不正确, 会尝试自动修复。这对提高查询成功率很重要。
```

```
corrected_cypher = correct_cypher_query_relationship_direction(  
    graph=graph, cypher_statement=cypher_statement  
)
```

```
validate_cypher_system = """
```

```
你是一位Cypher专家, 正在审查一位初级开发者编写的语句。
```

```
"""
```

```
validate_cypher_user = """你必须检查以下内容:
```

- * Cypher语句中是否有任何语法错误?
- * Cypher语句中是否有任何缺失或未定义的变量?
- * Cypher语句是否包含足够的信息来回答问题?
- * 确保所有节点、关系和属性都存在于提供的模式中。

```
好的错误示例:
```

- * 标签(:Foo)不存在, 你是否指的是(:Bar)?
- * 属性bar对标签Foo不存在, 你是否指的是baz?
- * 关系F00不存在, 你是否指的是F00_BAR?

```
模式:
```

```

)

# 如果启用了大模型验证，会使用语言模型检查Cypher查询的更高级错误，
# 例如语义上是否符合用户问题、属性映射是否正确等。这是一种更智能的验证方式。
if llm is not None and llm_validation:
    llm_errors = await validate_cypher_query_with_llm(
        validate_cypher_chain=validate_cypher_chain,
        question=state.get("task", ""),
        graph=graph,
        cypher_statement=cypher_statement,
    )
    errors.extend(llm_errors.get("errors", []))
    mapping_errors.extend(llm_errors.get("mapping_errors", []))

```

```

# step 6. 执行 Cypher 查询语句
execute_cypher = create_text2cypher_execution_node(
    graph=neo4j_graph, cypher=execute_info
)

```

工具节点3: customer_tools

```

customer_tools = create_graphrag_query_node()

```

总结节点

```

goal_and_conditions_page,
)
main_graph_builder.add_conditional_edges(
    "planner",
    map_reduce_planner_to_tool_selection, # type: ignore[arg-type, unused-ignore]
    ["tool_selection"],
)

main_graph_builder.add_edge("cypher_query", "summarize")
main_graph_builder.add_edge("predefined_cypher", "summarize")
main_graph_builder.add_edge("customer_tools", "summarize")
main_graph_builder.add_edge("summarize", "final_answer")

main_graph_builder.add_edge("final_answer", END)

return main_graph_builder.compile()

```

No Python interpreter configured for the project Create a virtual environment using requirements.txt Configure Python interp...

```

4  def create_summarization_prompt_template() -> ChatPromptTemplate:
16      (
17          "system",
18          """你是一个专业的电商智能客服助手，擅长将复杂信息整理成简洁明了的回答。
19
20      请以类似淘宝/京东等知名电商客服的风格回复用户：
21      - 开场要亲切，使用"亲~"或"顾客您好~"等问候语
22      - 保持积极、专业的语气
23      - 适当使用emoji表情（如 🍷 😊 ❤️）增加亲和力
24      - 结尾表达感谢和继续服务的意愿"""
25      ),
26      (
27          "human",
28          (
29              """事实信息: {results}
30
31      问题: "{question}"
32
33      请按照以下要求回答：
34      * 根据上述事实信息，以亲切的电商客服口吻回答用户问题
35      * 当事实不为空时，只使用这些信息构建回答
36
37      """
38      ),
39      ),
40  )

```

```

summarize = create_summarization_node(llm=llm)

final_answer = create_final_answer_node()

# 创建状态图
main_graph_builder = StateGraph(OverallState, input=InputState, output=OutputState)

main_graph_builder.add_node(quandnails)

```

2.6.7 总体问答效果

您好！我是智能客服助手，有什么可以帮您的吗？😊

你好

亲~您好呀！😊 有什么可以帮您的吗？

我想购买一个小米汽车，帮我介绍一下车型。

抱歉，我家暂时没有这方面的商品，可以在别家看看哦~

帮我查询一下我的订单

亲~麻烦提供一下订单号，我马上帮您查询哦！😊

我的订单号是51，帮我查询一下

亲~您好呀！😊 您的订单号51包含以下商品哦：

📦 商品清单：

- 博世 智能洗衣机 Basic
- 小米 智能手环 Mini
- 三星 智能门锁 Advanced

• 美的 智能音箱 Advanced

• 松下 智能空调 Basic

目前订单中部分价格和数量信息暂未更新，您可以稍后查看或联系专员补充详情~ ❤️

还有其他需要帮忙的随时找我哦！

介绍一下关于电子冰箱的评价

亲~您好呀！😊 关于电子冰箱的评价，目前暂时没有相关的具体信息和用户反馈记录哦~ ❤️

建议您可以参考专业的产品评测或者查看其他用户的真实使用体验，这样能更全面地了解产品呢！🌟

如果还有其他需要帮助的地方，随时问我哦~ 🙋

帮我查询一下订单号51和订单号32的订单情况

亲~您好呀！😊 我来为您查询订单情况啦~

📦 订单信息如下：

1 订单日期： 2024-02-17 22:34:55

要求送达日期： 2024-02-22 22:34:55

发货状态： 尚未发货

客户名称： 菊风公司传媒有限公司移动应用开发

2 订单日期： 2023-08-31 01:23:30

要求送达日期： 2023-09-11 01:23:30

发货状态： 已于2023-09-04 01:23:30
发货

客户名称： 精芯网络有限公司移动应
用开发

如果有其他需要帮助的地方，随时问
我哦~ ❤️

请输入您想咨询的问题...



发送